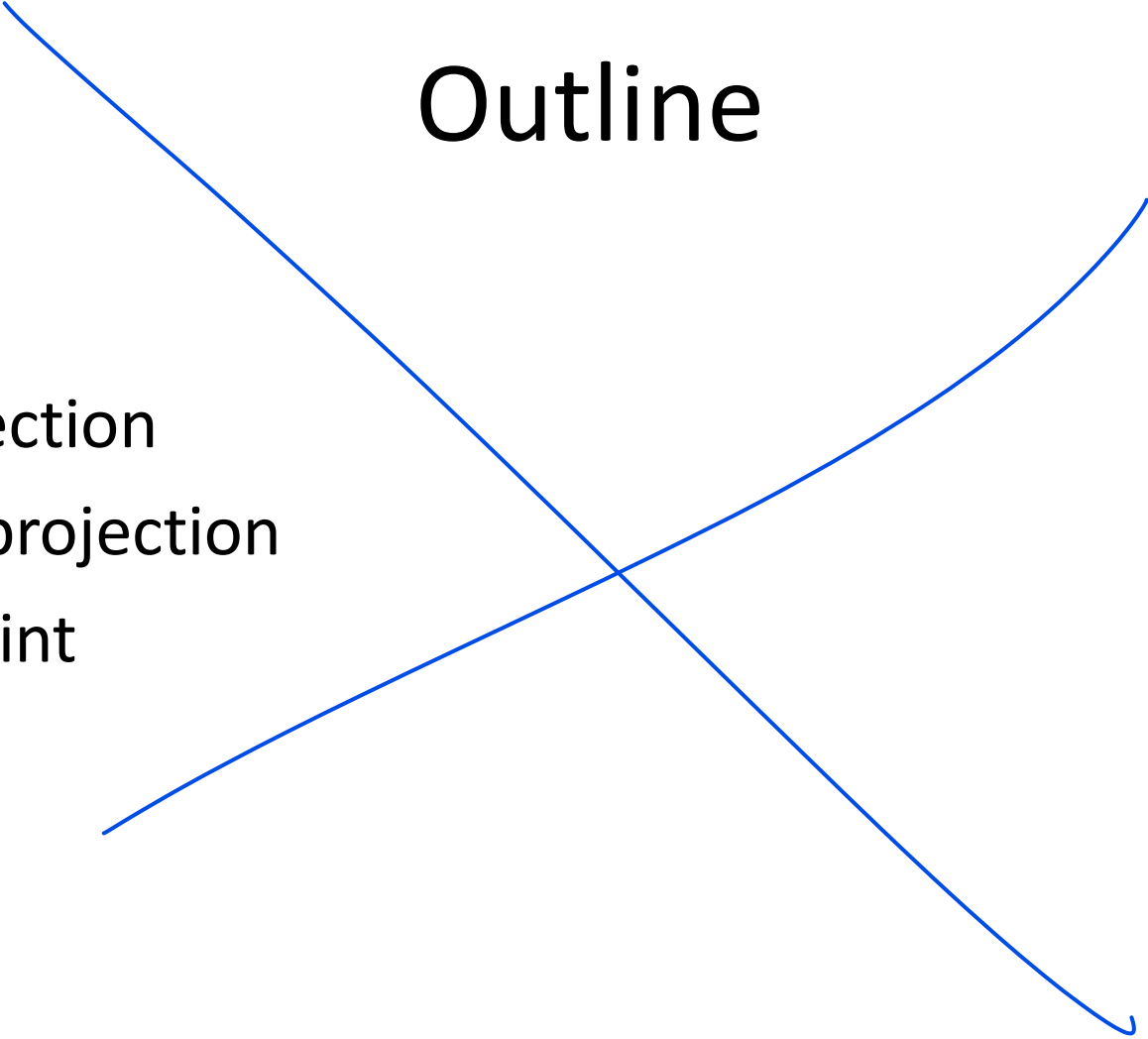


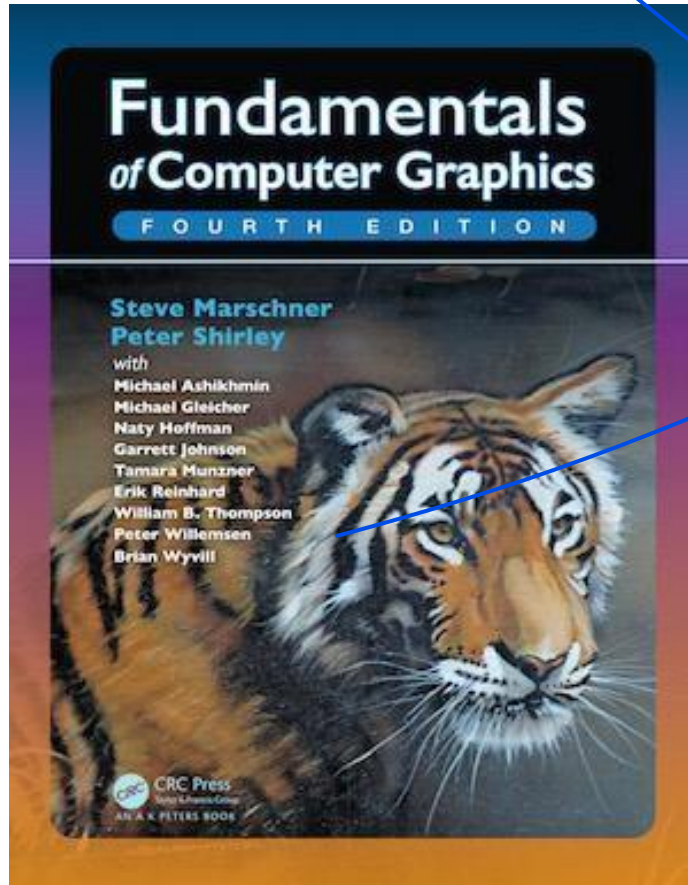
CSE4203: Computer Graphics
Chapter – 4 (part - A)
Ray Tracing

Outline



- Projection
- Parallel projection
- Perspective projection
- Vanishing point

Credit



CS4620: Introduction to Computer Graphics

Cornell University

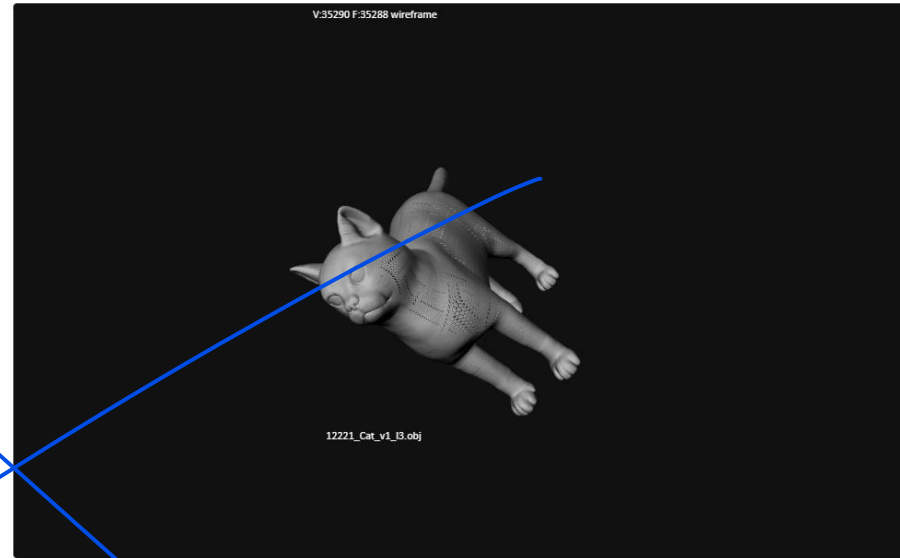
Instructor: Steve Marschner

<http://www.cs.cornell.edu/courses/cs4620/2019fa/>

3D → 2D

12221_Cat_v1_l3.obj

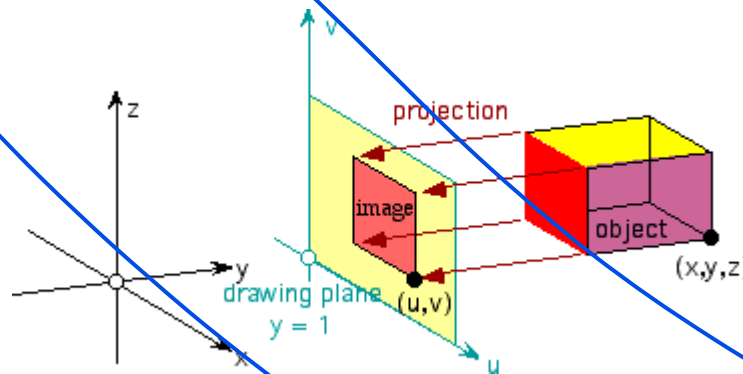
10	v	0.6134	-21.9357	31.4441
11	v	0.6205	-21.8541	31.4935
12	v	0.5534	-21.8682	31.5159
13	v	0.5464	-21.9516	31.4643
14	v	0.6207	-21.7610	31.5371
15	v	0.5544	-21.7724	31.5613
16	v	0.4816	-21.8847	31.5310
17	v	0.4743	-21.9703	31.4781
18	v	0.4838	-21.7858	31.5776
19	v	0.6156	-21.6618	31.5749
20	v	0.6070	-21.5619	31.6064
21	v	0.5439	-21.5675	31.6330
22	v	0.5507	-21.6702	31.6005
23	v	0.4772	-21.5744	31.6514
24	v	0.4819	-21.6802	31.6178
25	v	0.3320	-21.5913	31.6699
26	v	0.3330	-21.7025	31.6343
27	v	0.4094	-21.6911	31.6286
28	v	0.4067	-21.5824	31.6631
29	v	0.3317	-21.8143	31.5926
30	v	0.4094	-21.8001	31.5875
31	v	0.3275	-21.9189	31.5447



3D model Source: <https://free3d.com>
Model viewer: <http://masc.cs.gmu.edu/wiki/ObjViewer>

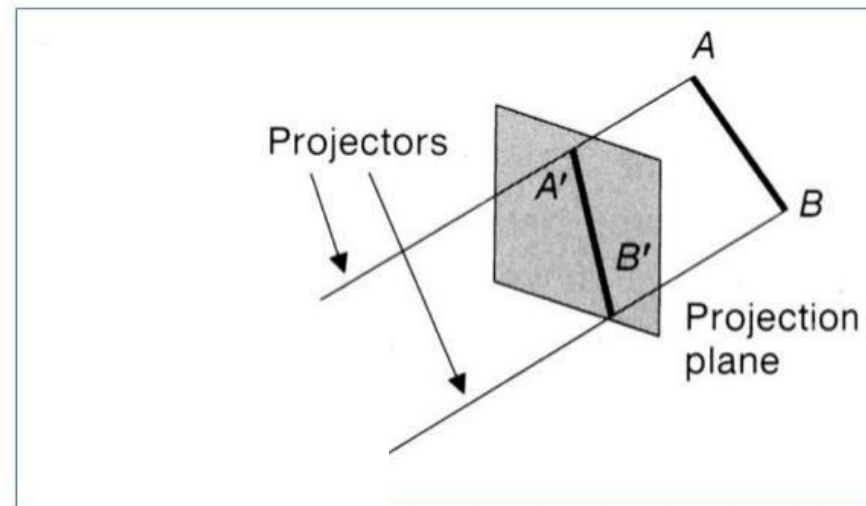
Projection (1/2)

- Representing a 3D object
 - Photographs also represent 3D scenes with 2D images.
- In computer graphics, **Projection** is used.



Projection (2/2)

- 3D points are mapped to 2D image plane by moving them along a *projection direction*
 - *until they hit the image plane*

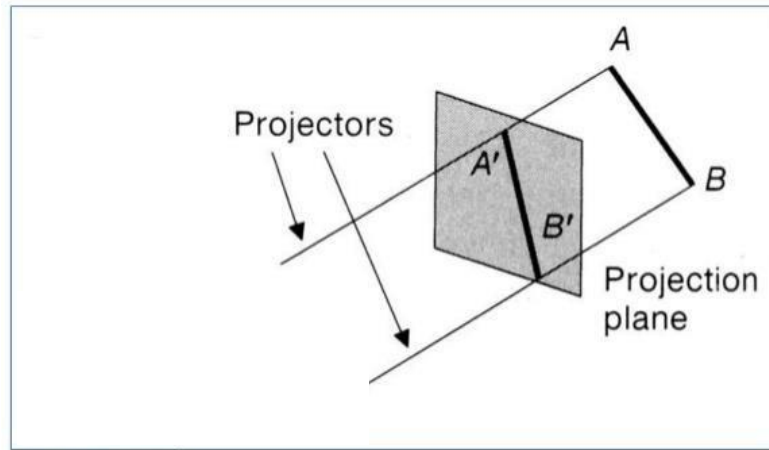


Credit: Fundamentals of Computer Graphics 4th Edition by Peter Shirley, Steve Marschner | <http://www.cs.cornell.edu/courses/cs4620/2019fa/>

Types of Projection (1/1)

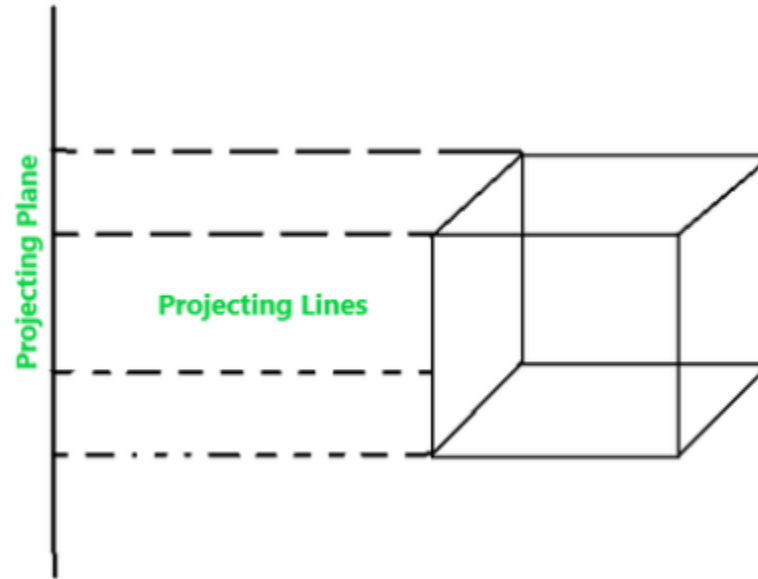
Main types:

- Parallel
- Perspective



Parallel Projection (1/3)

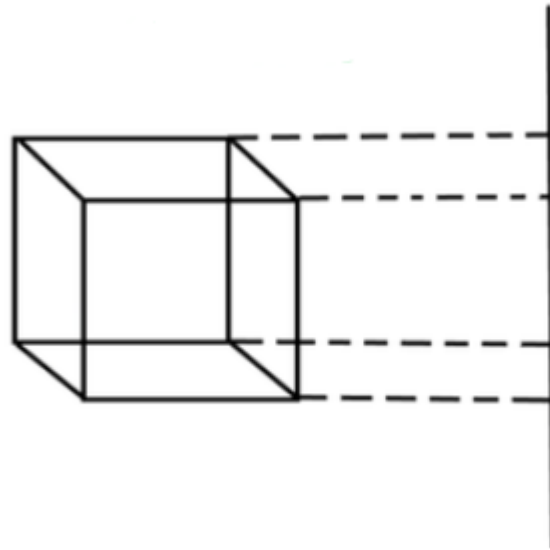
- Projectors are parallel
 - Center of Projection (CoP) lies at infinity ★



Credit: Fundamentals of Computer Graphics 4th Edition by Peter Shirley, Steve Marschner | <http://www.cs.cornell.edu/courses/cs4620/2019fa/>
Image: <https://www.geeksforgeeks.org/parallel-orthographic-oblique-projection-in-computer-graphics/?ref=rp>

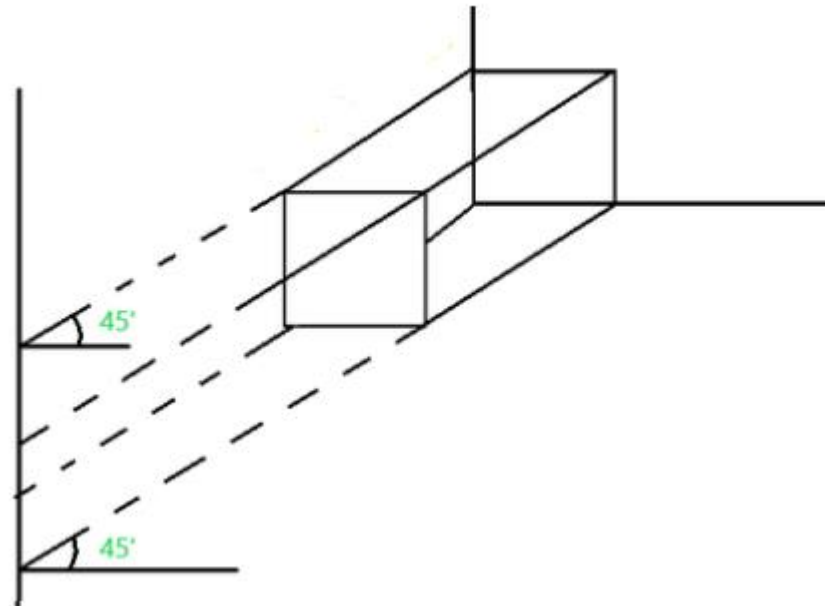
Parallel Projection (2/3)

- Orthographic
 - Image plane $\perp$ projector



Parallel Projection (3/3)

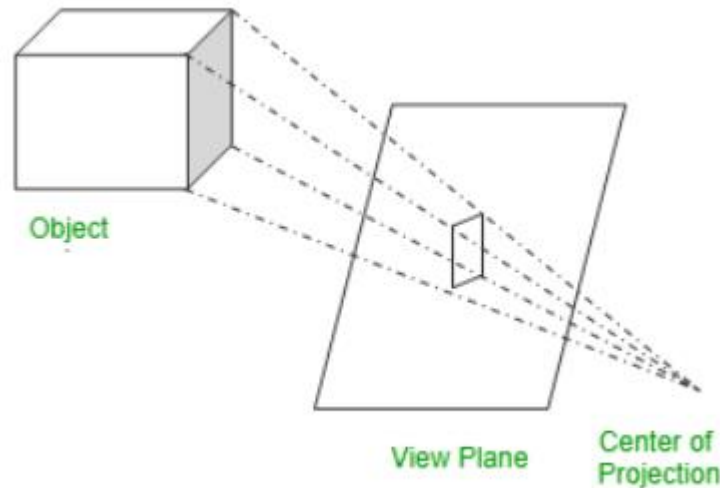
- Oblique
 - Image plane $\nparallel$ projector



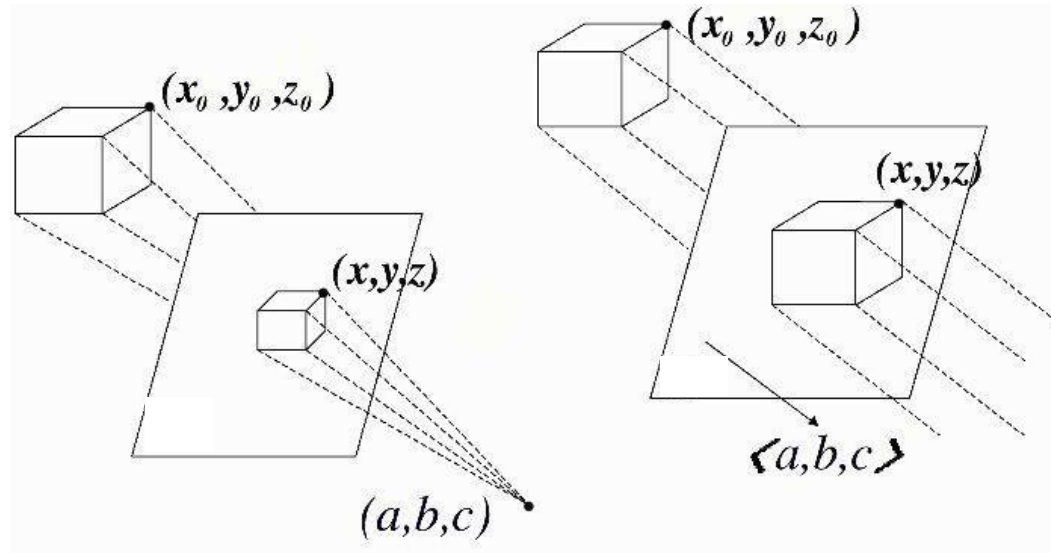
Credit: Fundamentals of Computer Graphics 4th Edition by Peter Shirley, Steve Marschner | <http://www.cs.cornell.edu/courses/cs4620/2019fa/>

Perspective Projection (1/2)

- Projectors do not remain parallel
- Projector meet at a point finite point



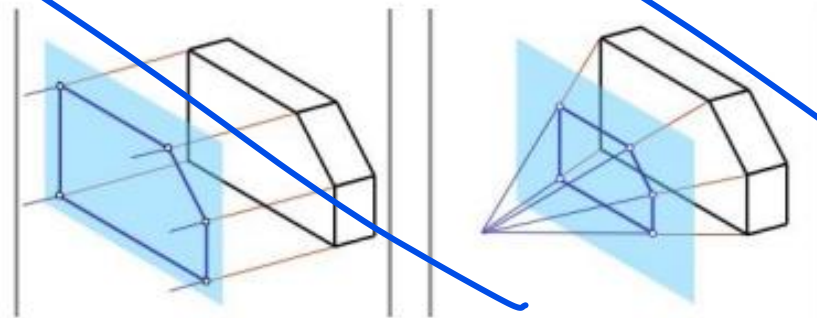
Perspective Projection (2/2)



- ✓ Does parallel project have a CP?
- ✓ What will happen if the object moves near/ far?
- Play around:
http://www.cs.cornell.edu/courses/cs4620/2017sp/demos/view_explore/view_explore.html

Parallel vs Perspective (1/2)

- Parallel projections are often used for mechanical and architectural drawings because they keep parallel lines parallel
- preserve the size and shape of planar objects that are parallel to the image plane



Credit: Fundamentals of Computer Graphics 4th Edition by Peter Shirley, Steve Marschner | <http://www.cs.cornell.edu/courses/cs4620/2019fa/>

Parallel vs Perspective (2/2)

- In our everyday experience (and in photographs)
 - objects look smaller $\leftrightarrow$ farther away

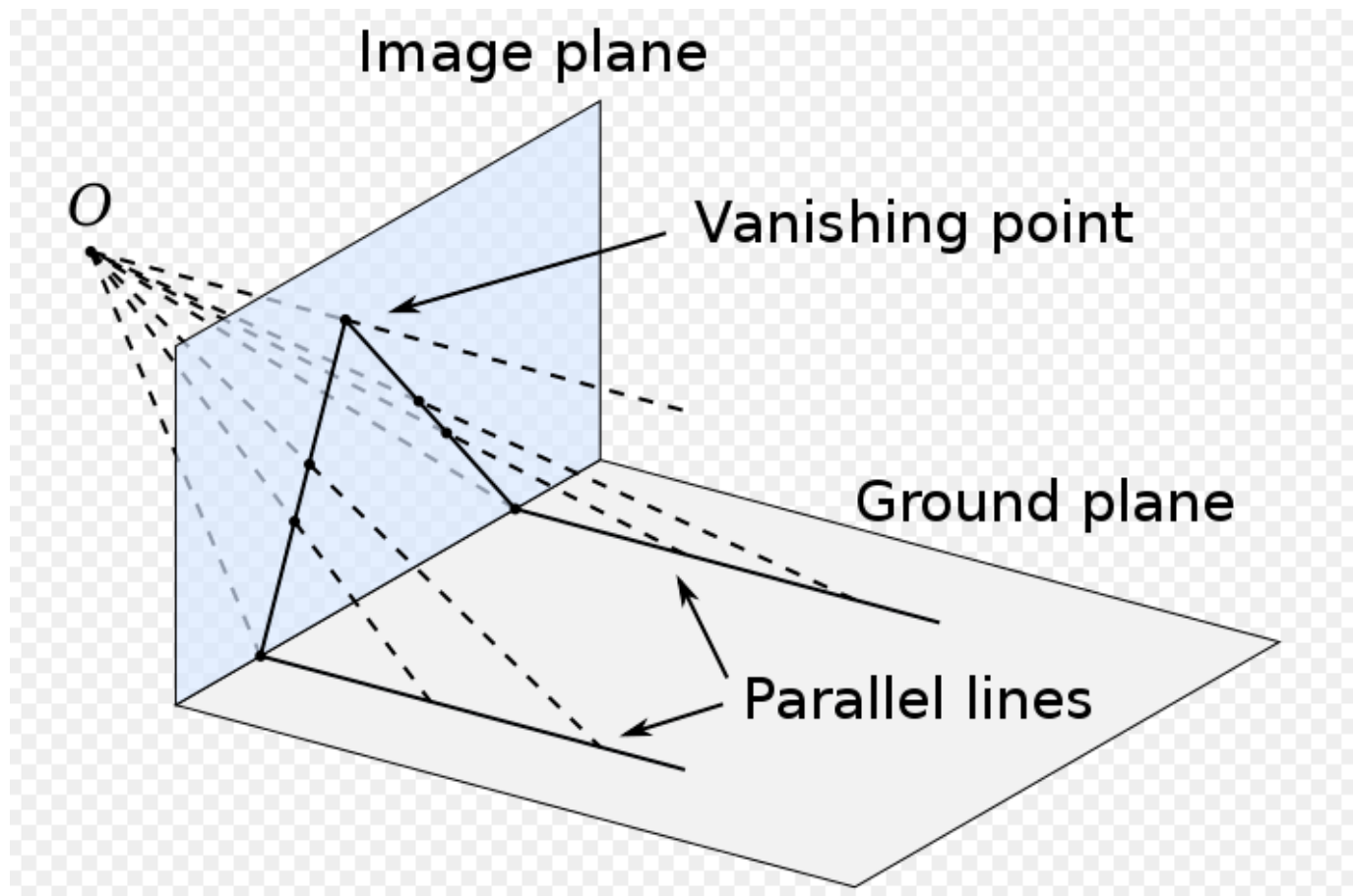


Credit: Fundamentals of Computer Graphics 4th Edition by Peter Shirley, Steve Marschner | <http://www.cs.cornell.edu/courses/cs4620/2019fa/>

Vanishing Point (1/2)



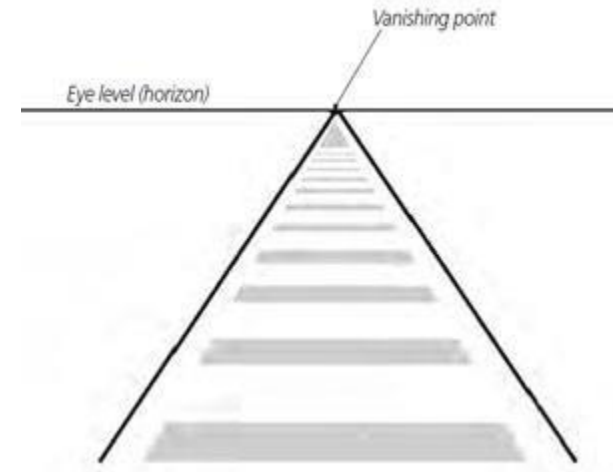
Q: Where is the VP here?



Vanishing Point (2/2)

Projection Plane = Image plane

- Vanishing points:
 - Any set of parallel lines that are not parallel to the projection plane will converge to a vanishing point.
 - Parallel horizontal lines meet at a point on the horizon.
 - Every set of parallel lines has own VP



Lines parallel to the other two axes will not form vanishing points as they are parallel to the image plane.

Q: Which type of parallel lines does not meet at VP?

Image source: <https://www.artistsnetwork.com/art-terms/vanishing-point-perspective/>

Credit: Fundamentals of Computer Graphics 4th Edition by Peter Shirley, Steve Marschner | <http://www.cs.cornell.edu/courses/cs4620/2019fa/>

Perspective Projection (Examples)

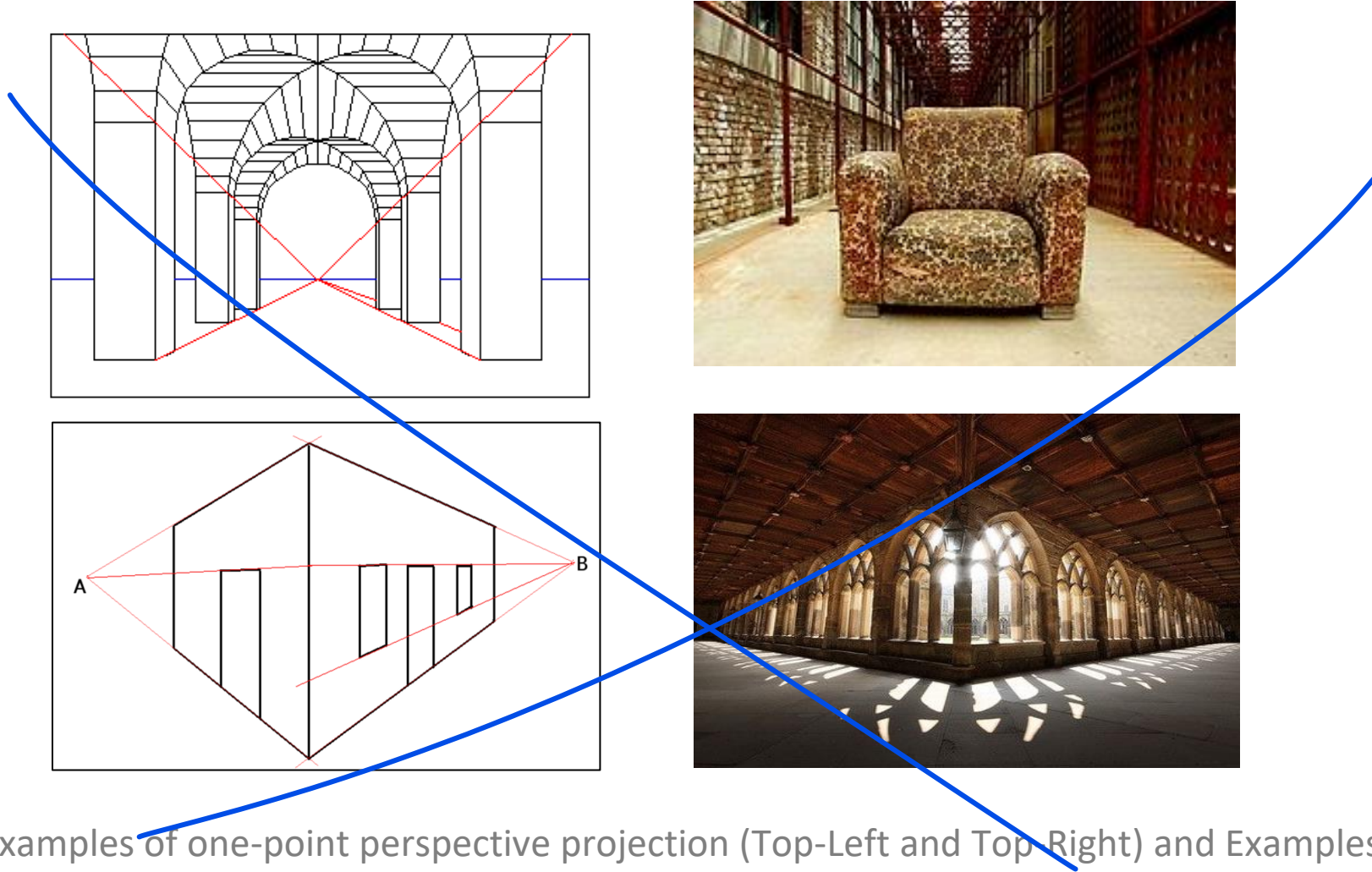
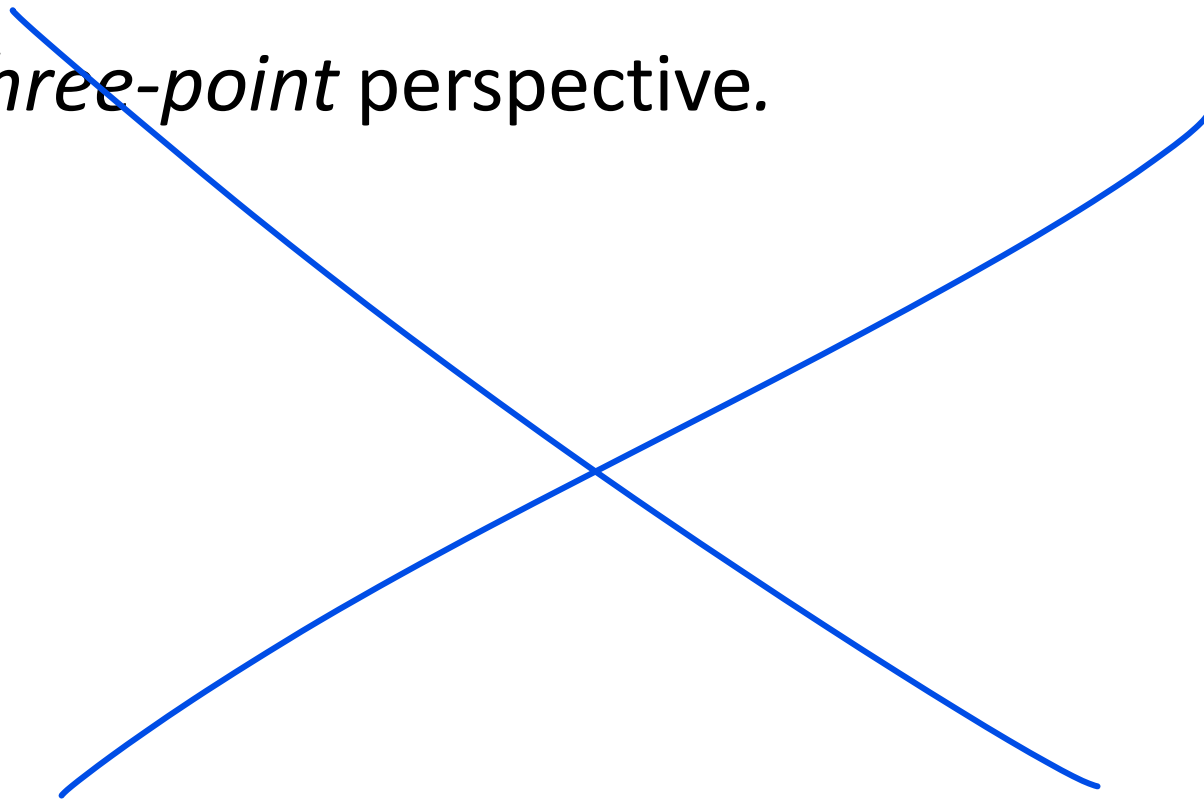


Fig: Examples of one-point perspective projection (Top-Left and Top Right) and Examples of two-point perspective projection (Bottom-Left and Bottom-Right)

Additional Reading

- The *three-point* perspective.



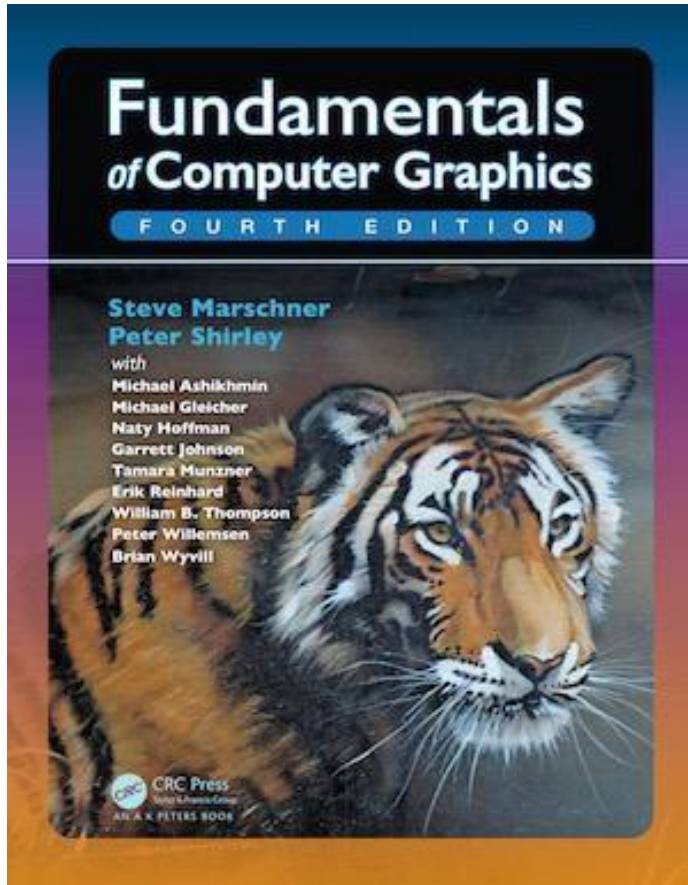
Thank You

CSE4203: Computer Graphics
Chapter – 4 (part - B)
Ray Tracing

Outline

- Ray-tracing

Credit



CS4620: Introduction to Computer Graphics

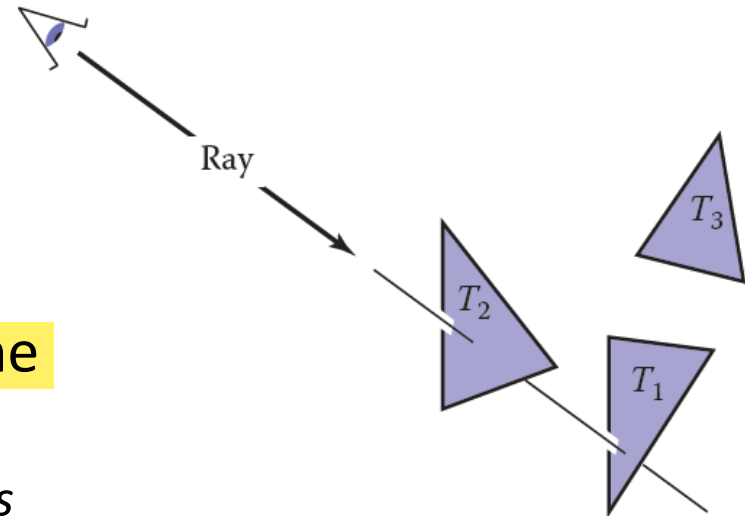
Cornell University

Instructor: Steve Marschner

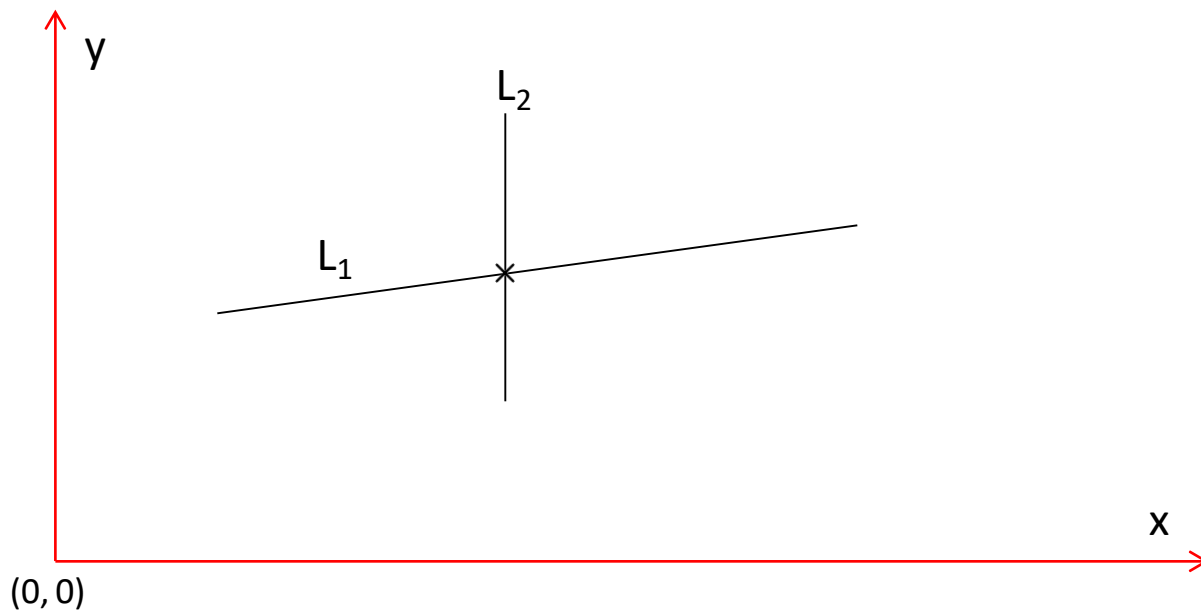
<http://www.cs.cornell.edu/courses/cs4620/2019fa/>

3D $\rightarrow$ 2D

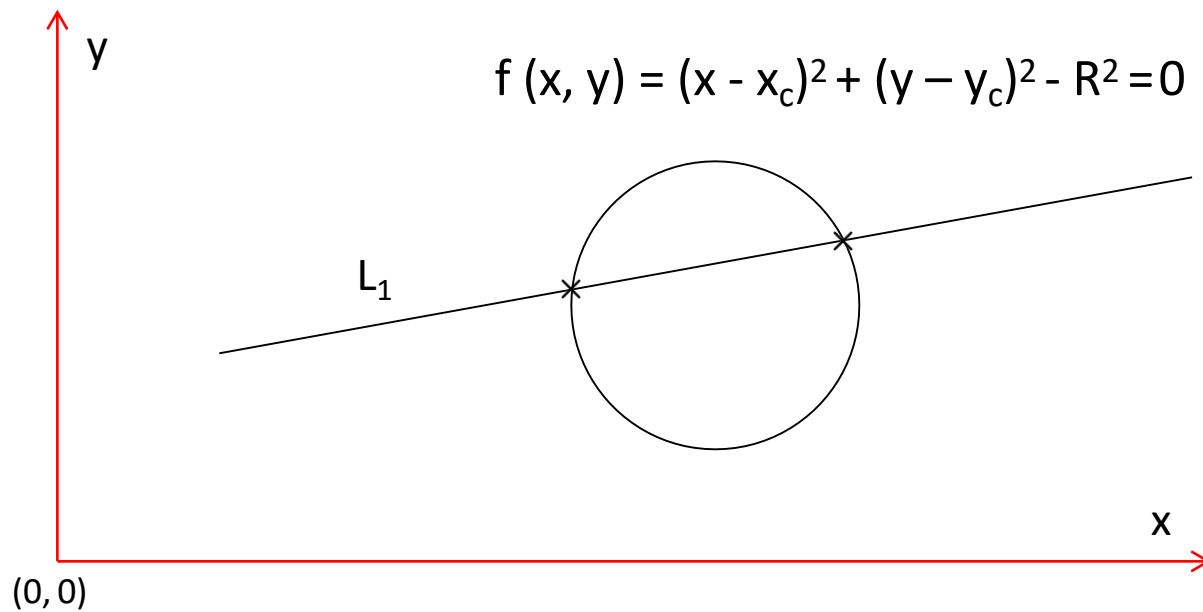
- Implementing projection: (3D $\rightarrow$ 2D)
 - Ray-tracing technique
- Motivation:
 - From how we see!
 - The **ray** is “**traced**” into the **scene**
 - the first **object** hit is the one seen.
 - In this case, the triangle T_2 is returned.



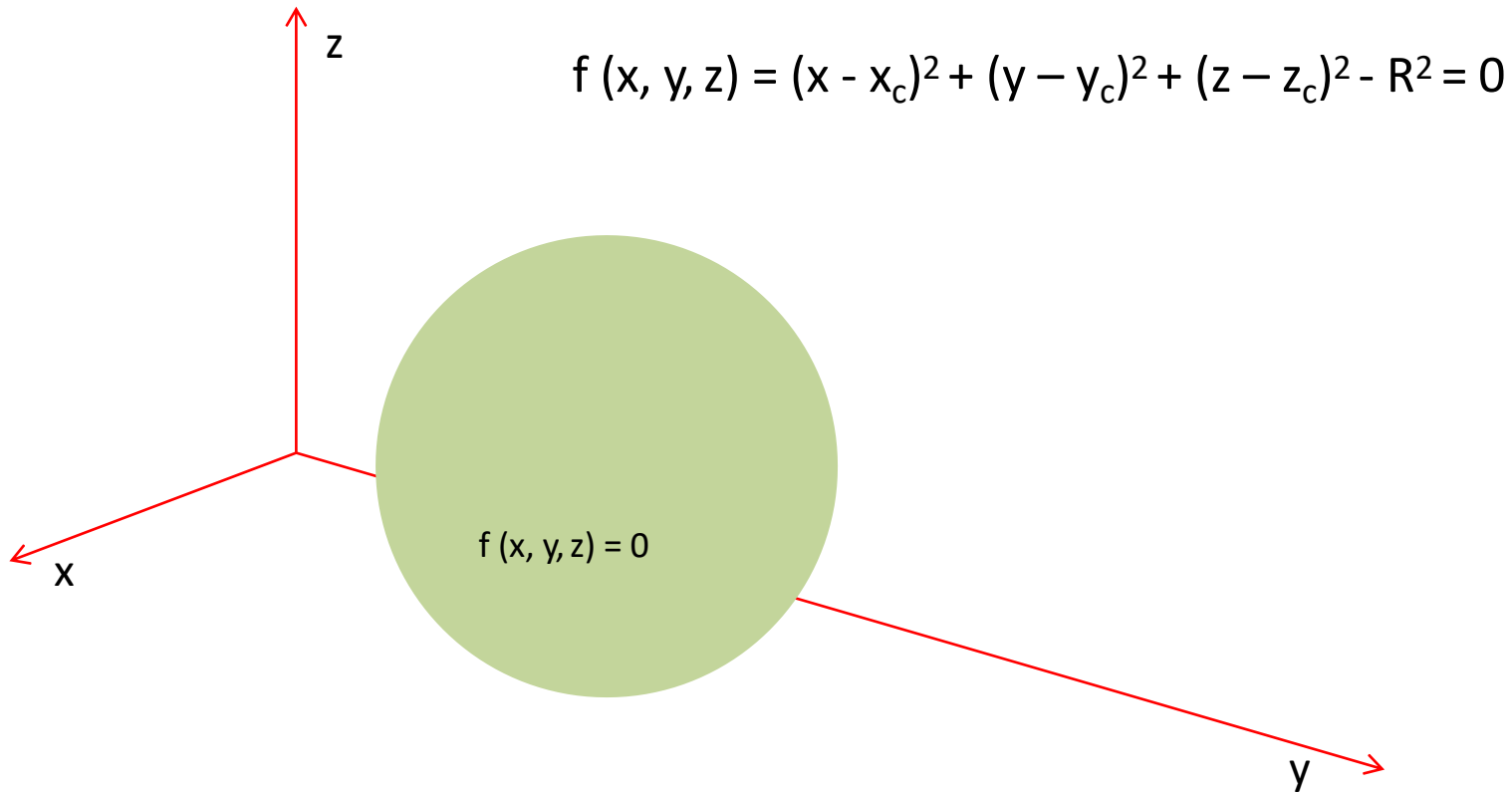
Warm-up (1/9)



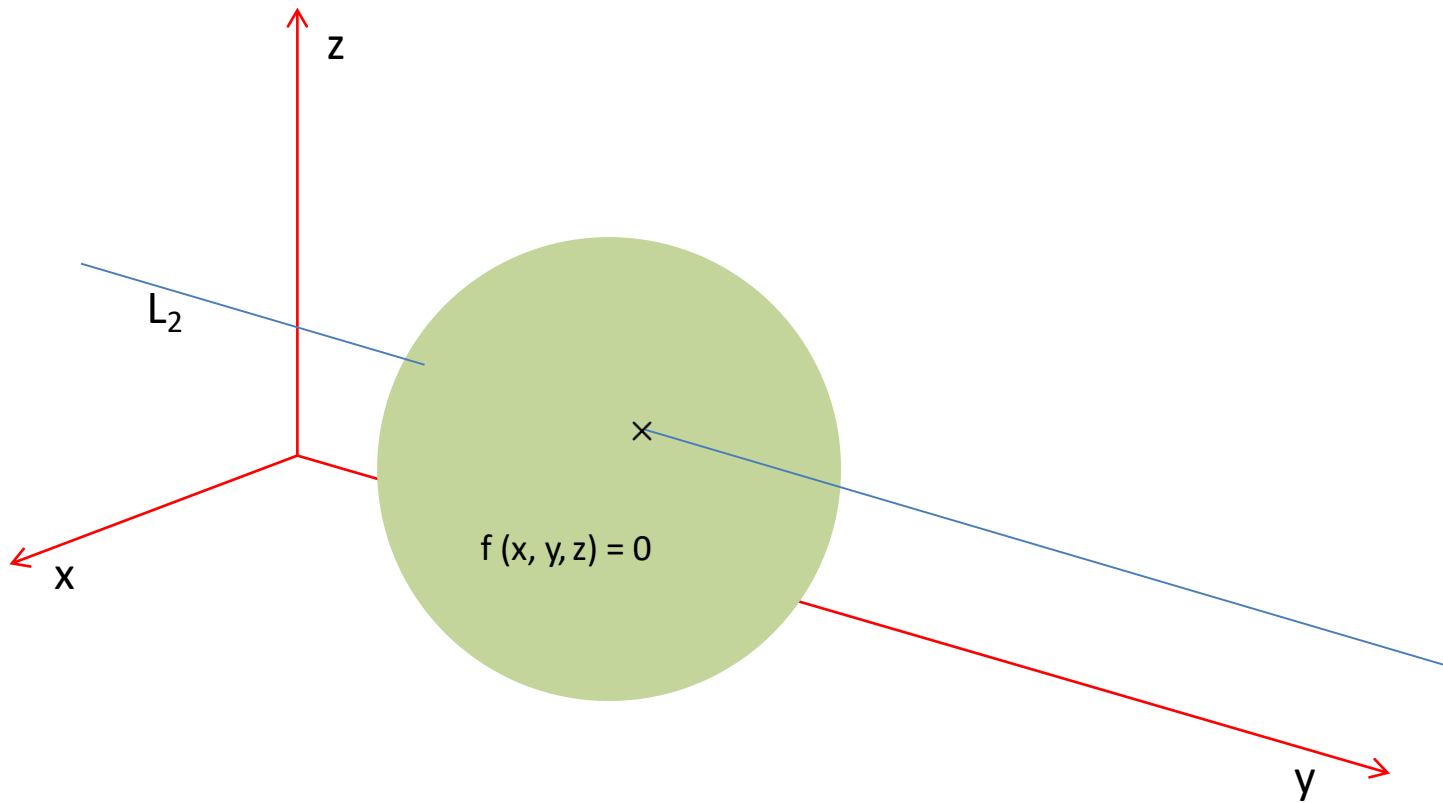
Warm-up (2/9)



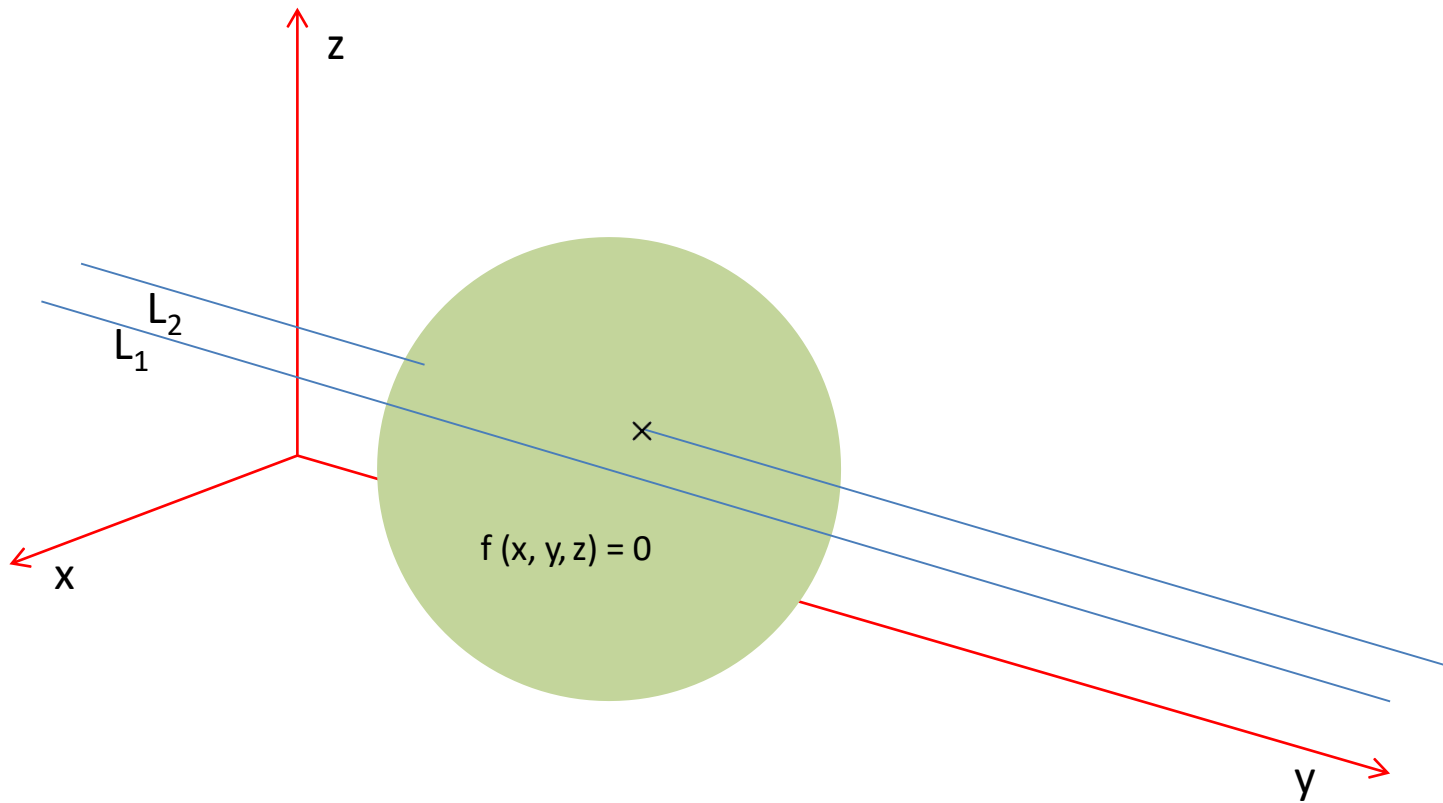
Warm-up (3/9)



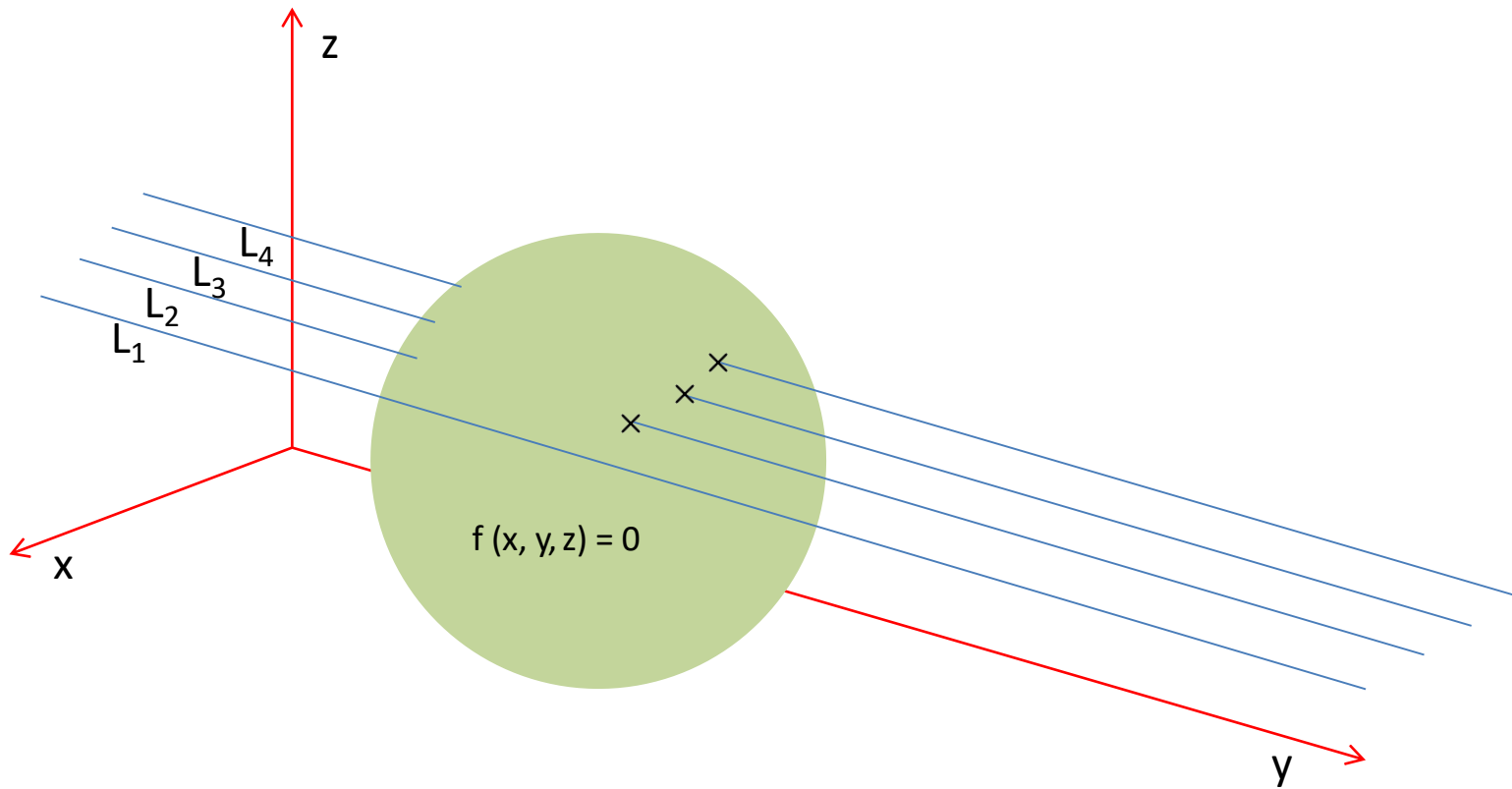
Warm-up (4/9)



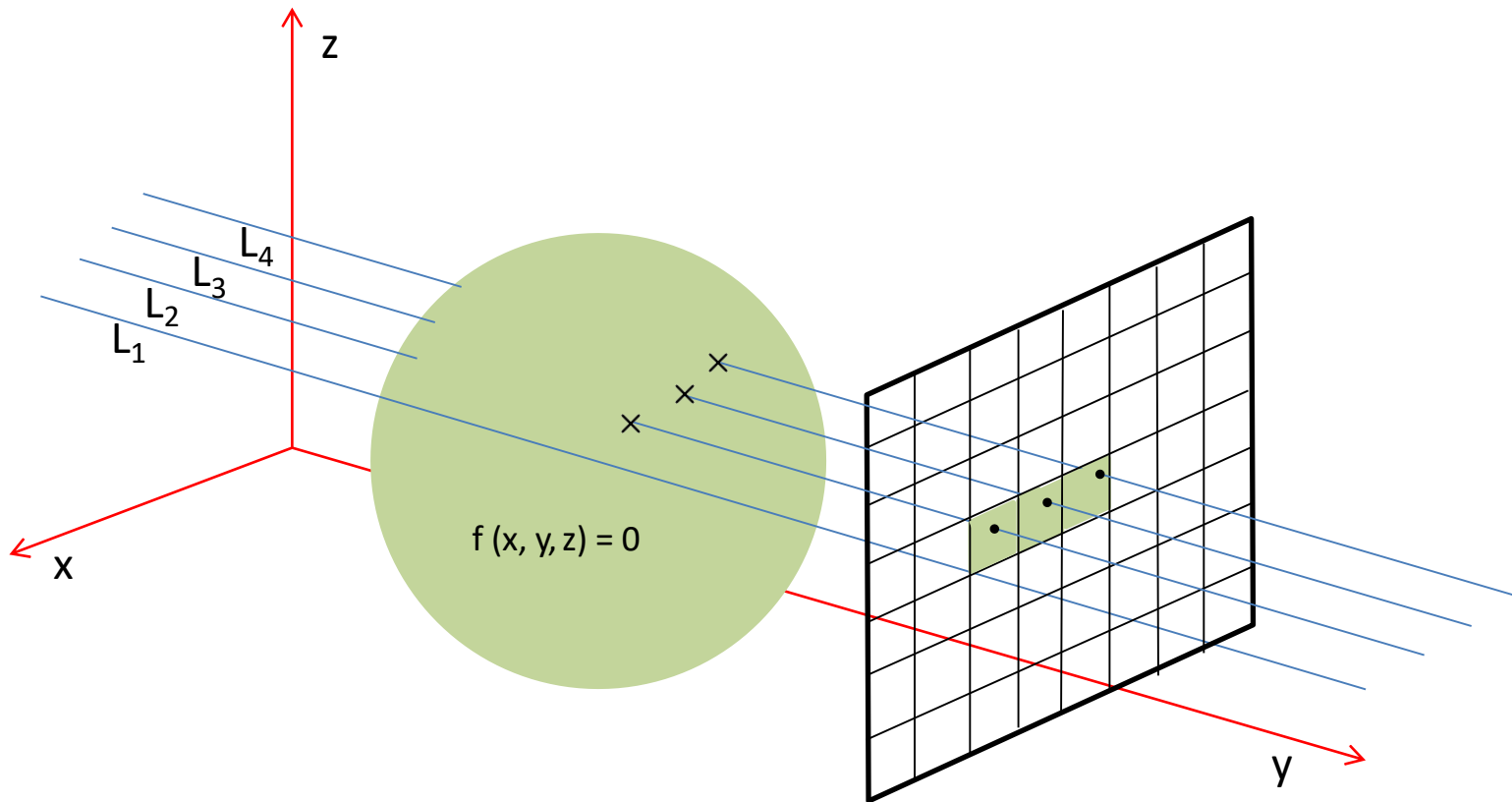
Warm-up (5/9)



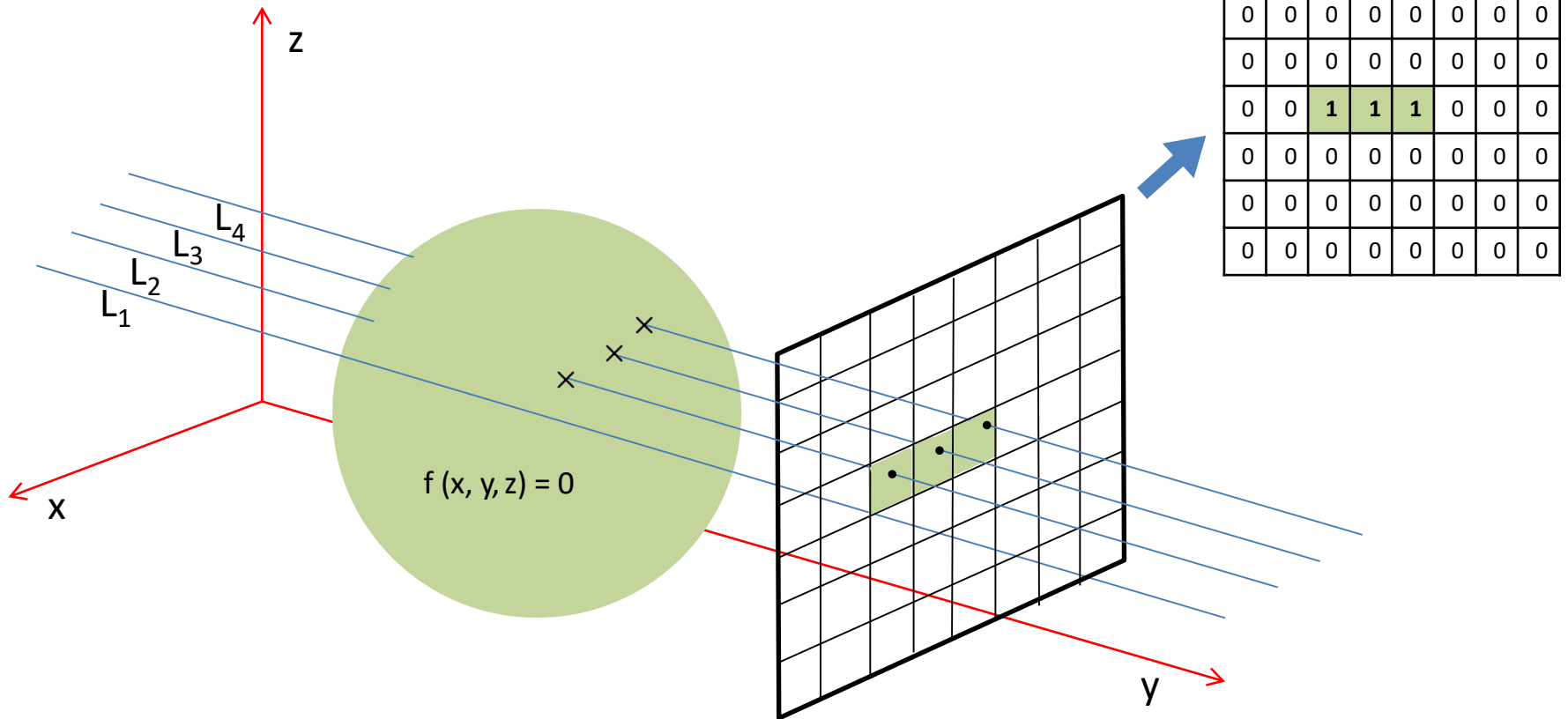
Warm-up (6/9)



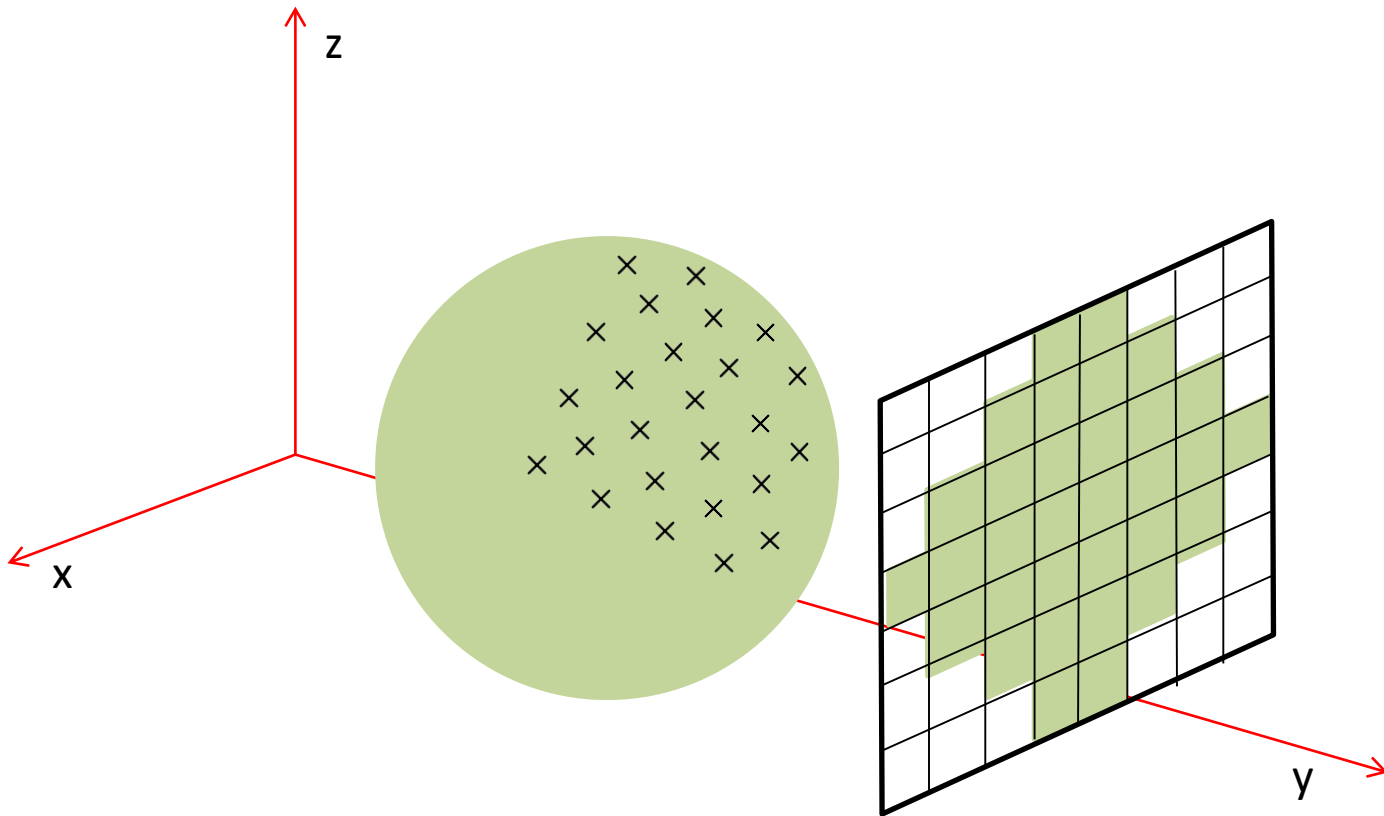
Warm-up (7/9)



Warm-up (8/9)



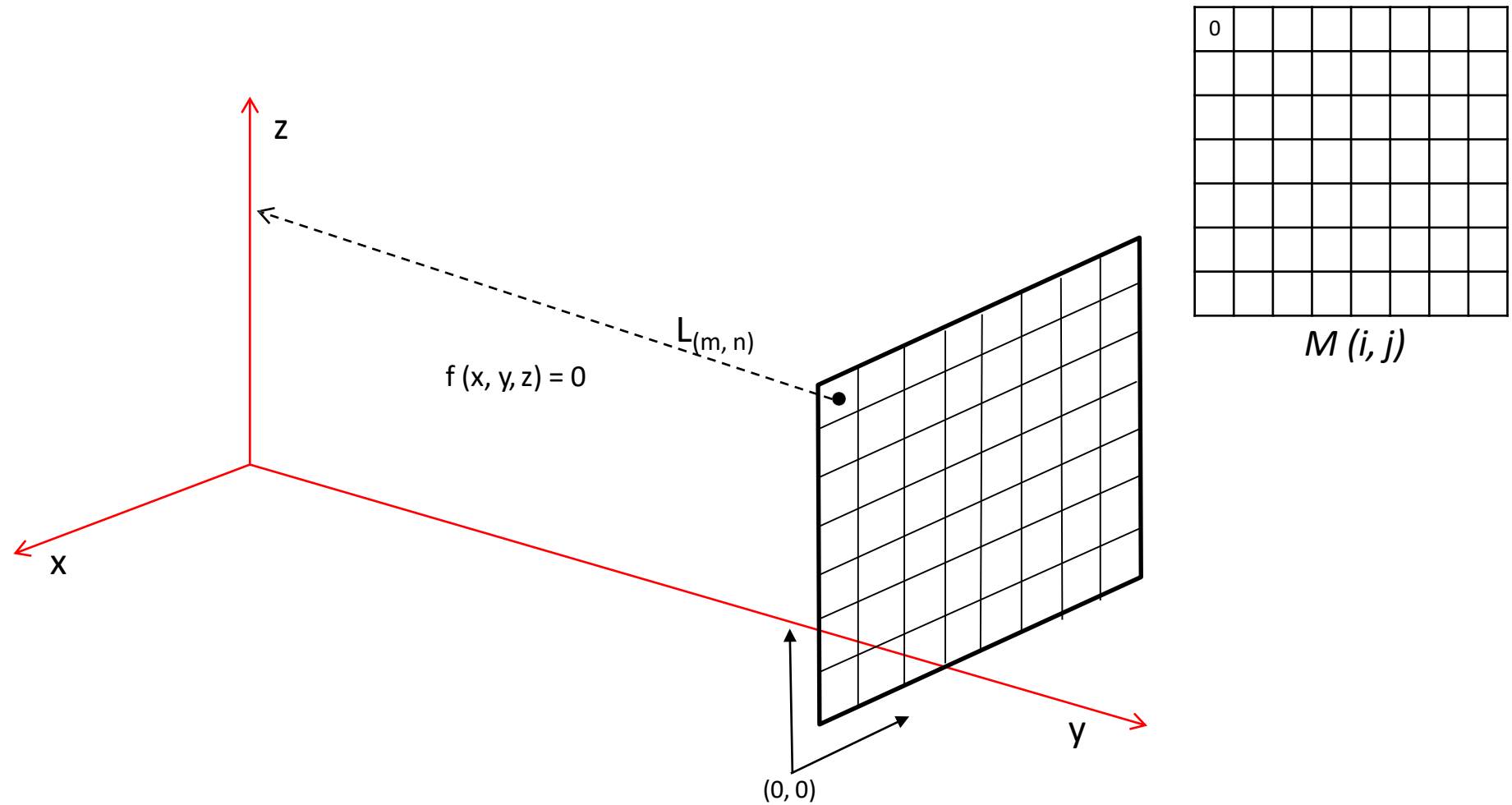
Warm-up (9/9)



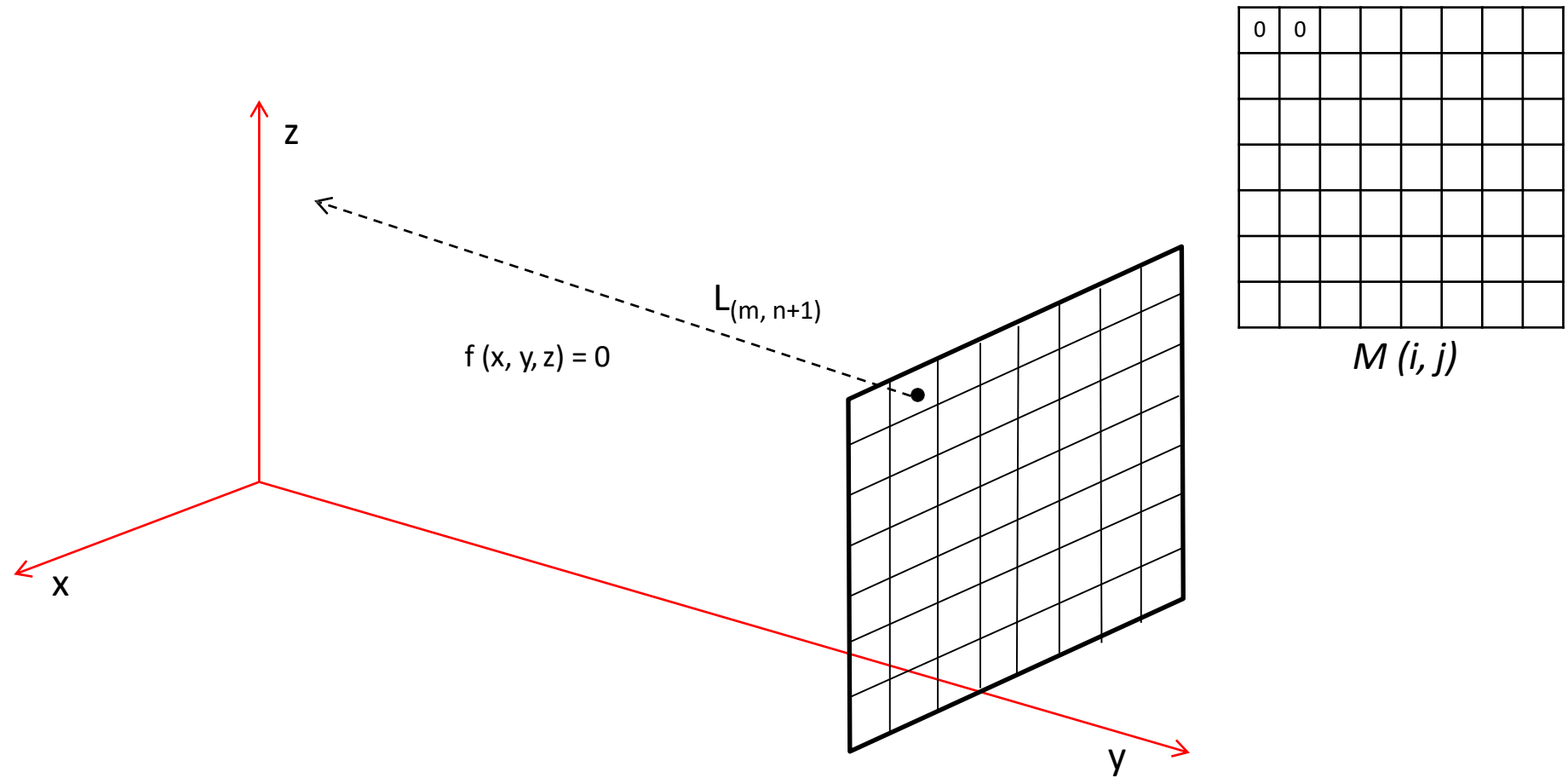
$M(i, j)$

0	0	0	1	1	0	0	0
0	0	1	1	1	1	0	0
0	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	0
0	0	1	1	1	1	0	0
0	0	0	1	1	0	0	0

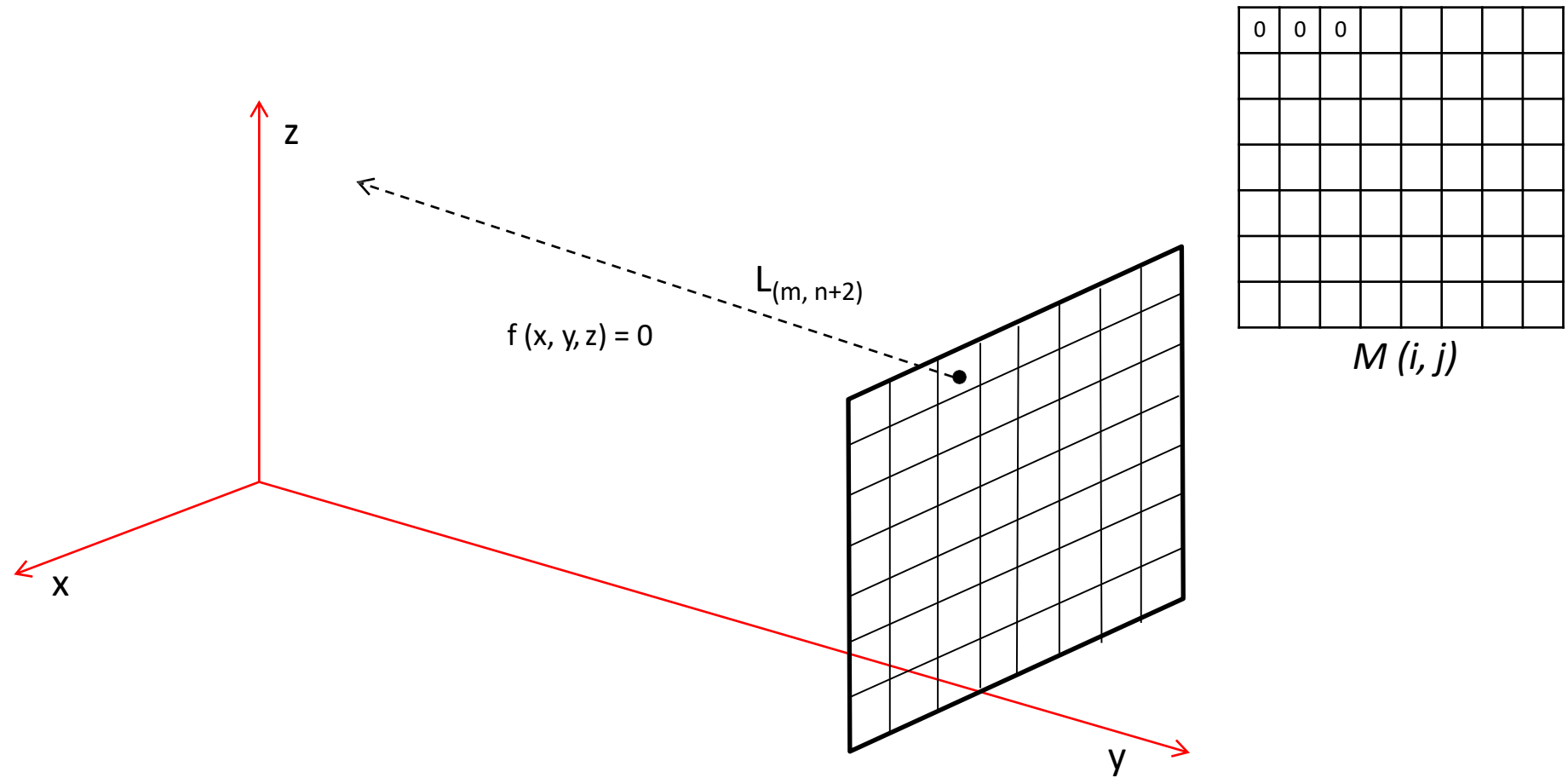
Ray-tracing Basics (1/15)



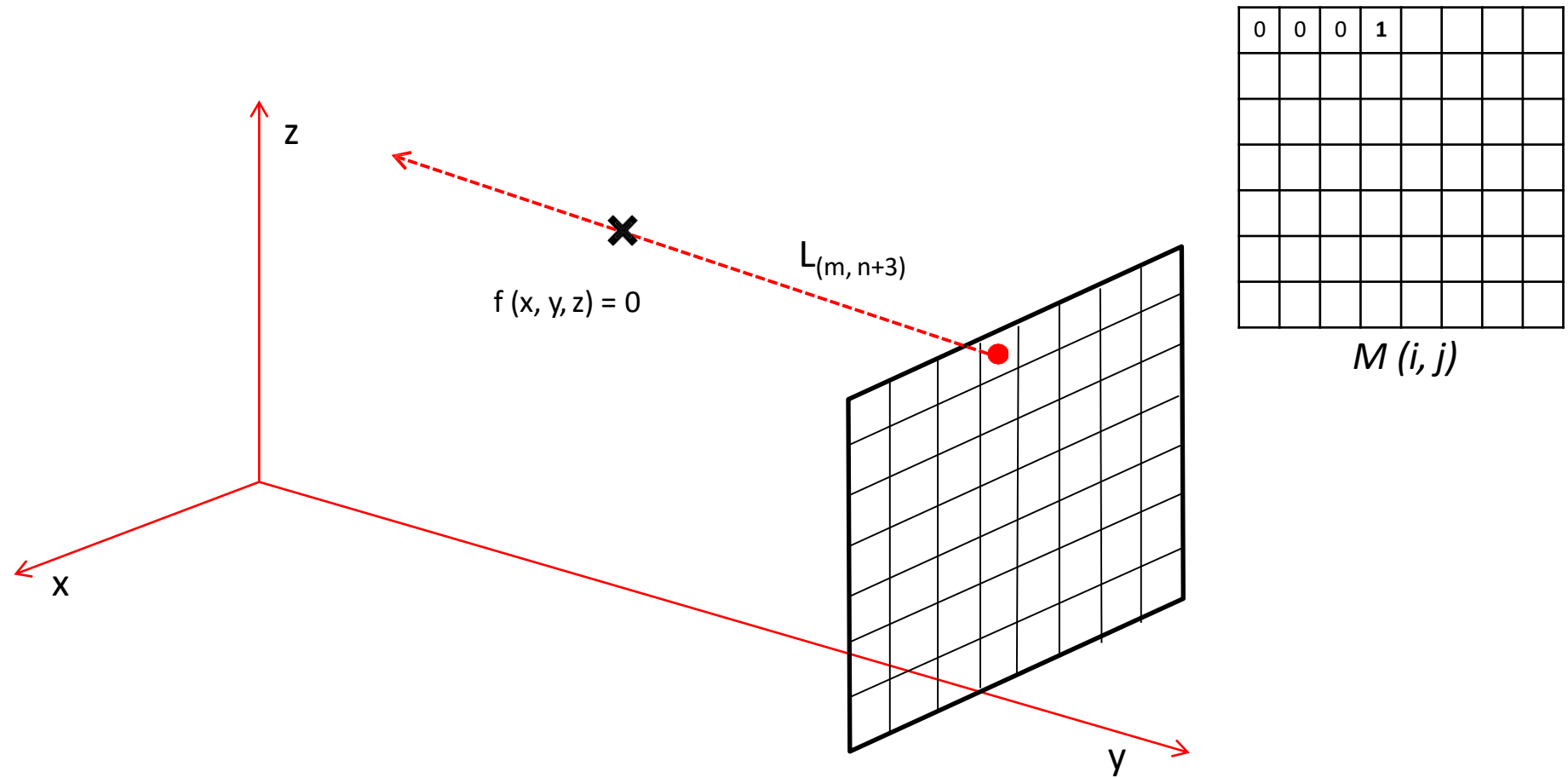
Ray-tracing Basics (2/15)



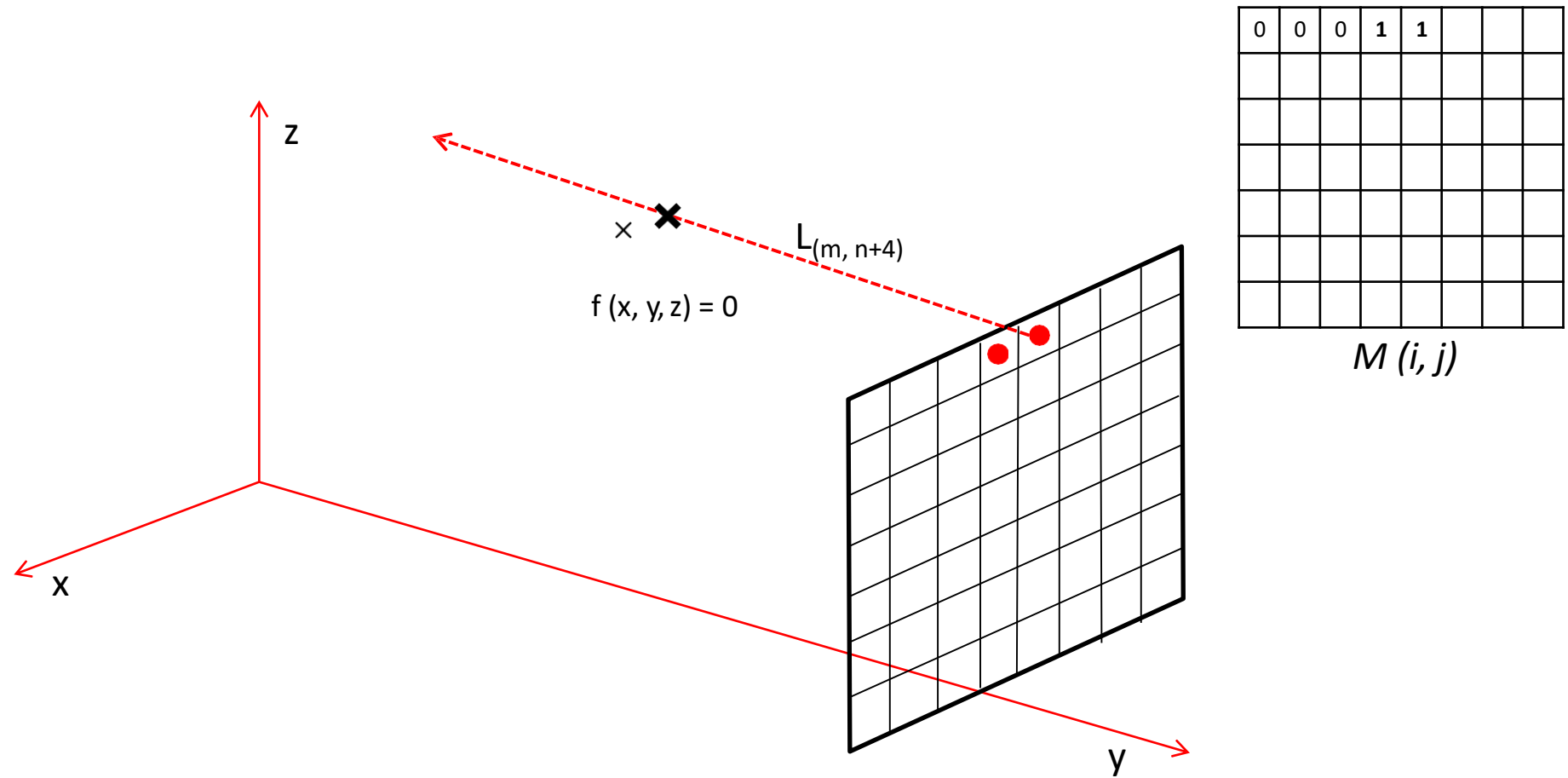
Ray-tracing Basics (3/15)



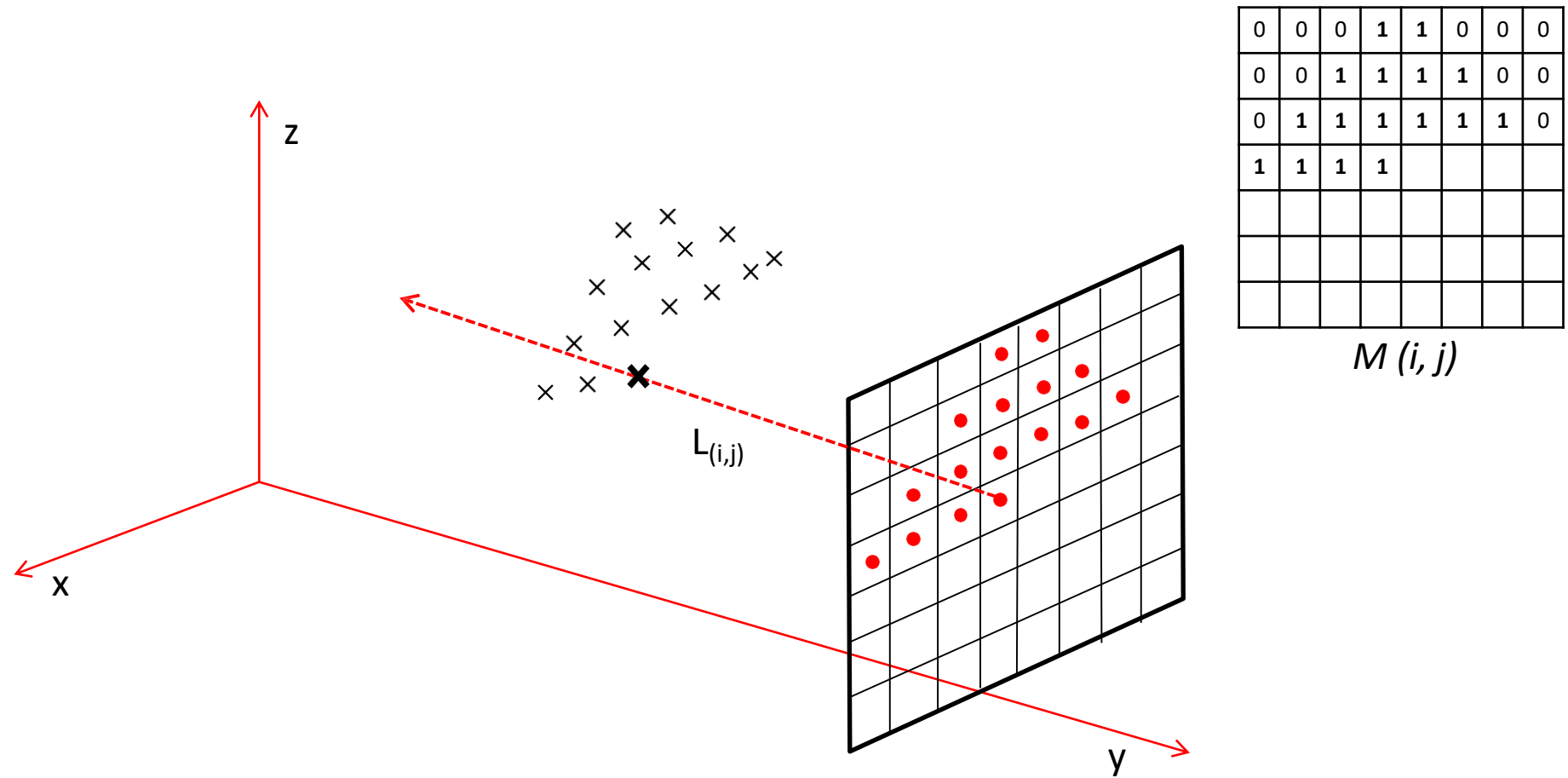
Ray-tracing Basics (4/15)



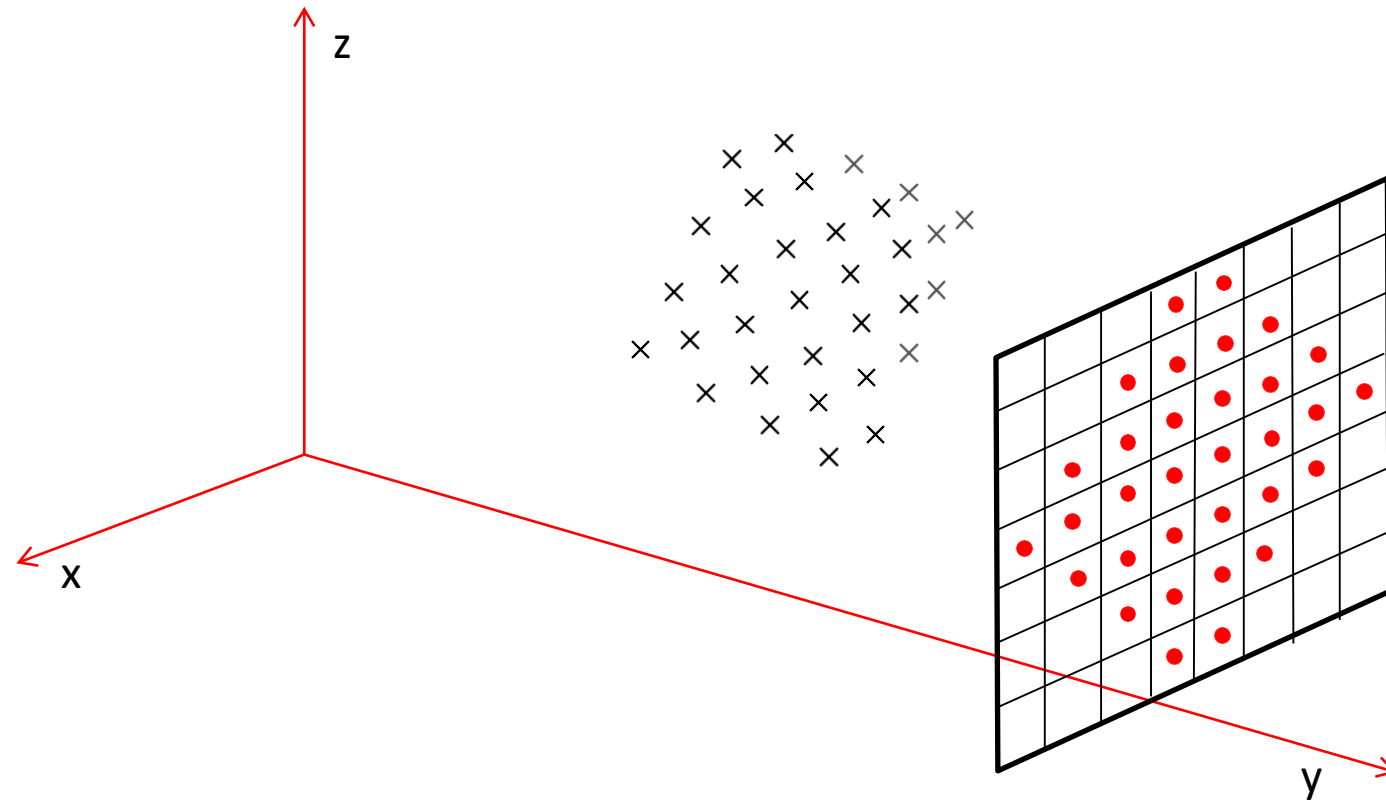
Ray-tracing Basics (5/15)



Ray-tracing Basics (6/15)



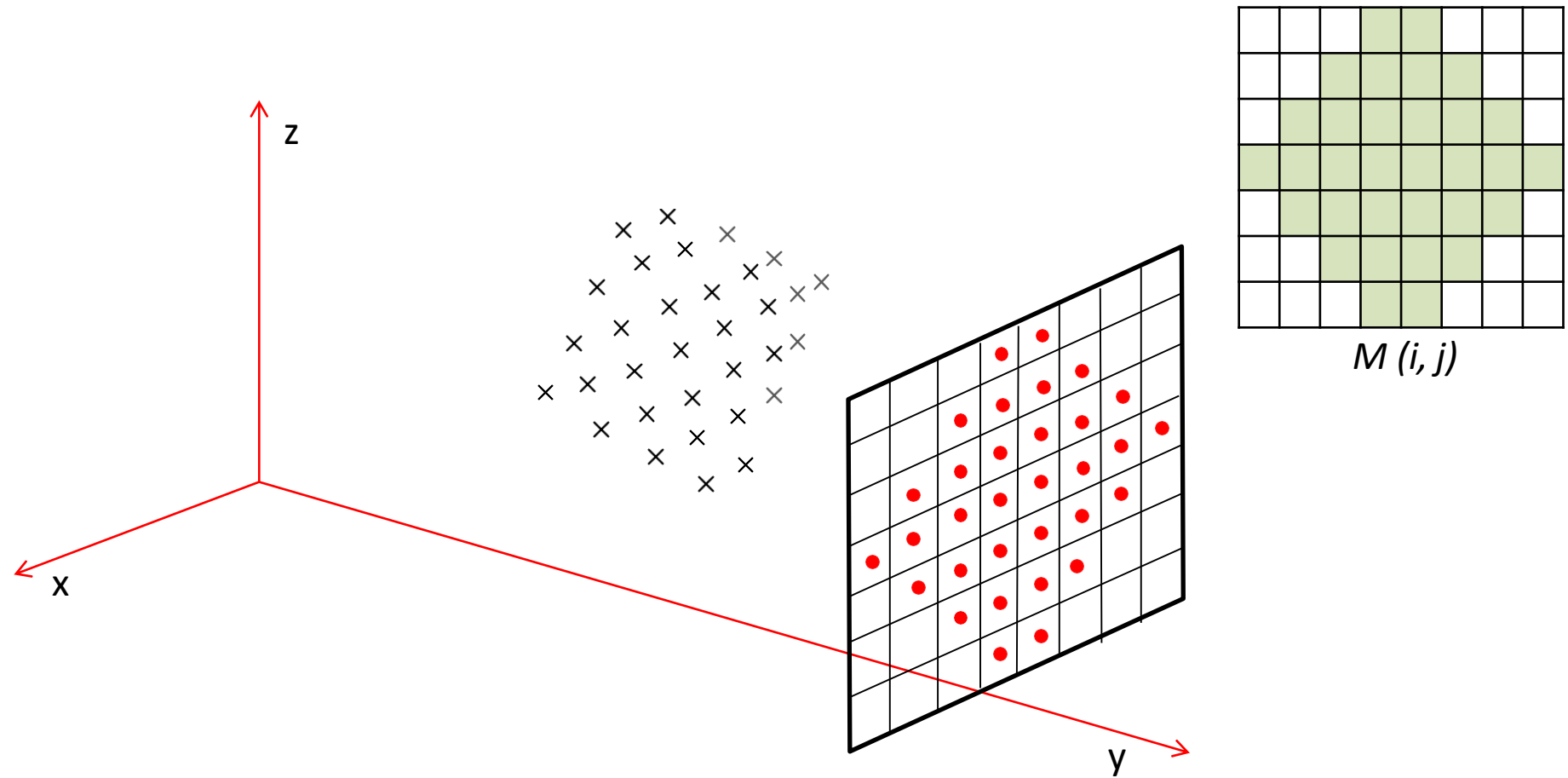
Ray-tracing Basics (7/15)



0	0	0	1	1	0	0	0
0	0	1	1	1	1	0	0
0	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	0
0	0	1	1	1	1	0	0
0	0	0	1	1	0	0	0

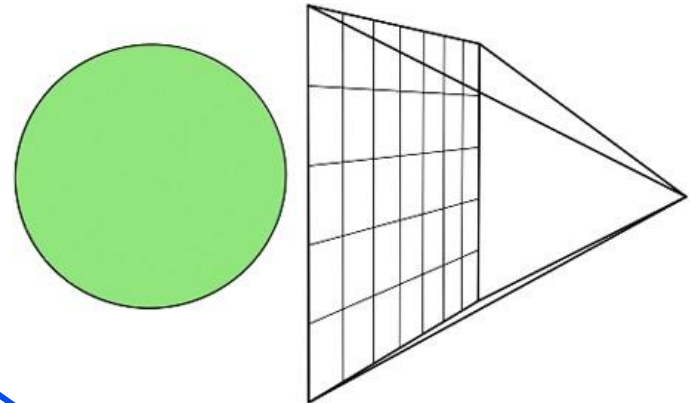
$M(i, j)$

Ray-tracing Basics (8/15)



Ray-tracing Basics (9/15)

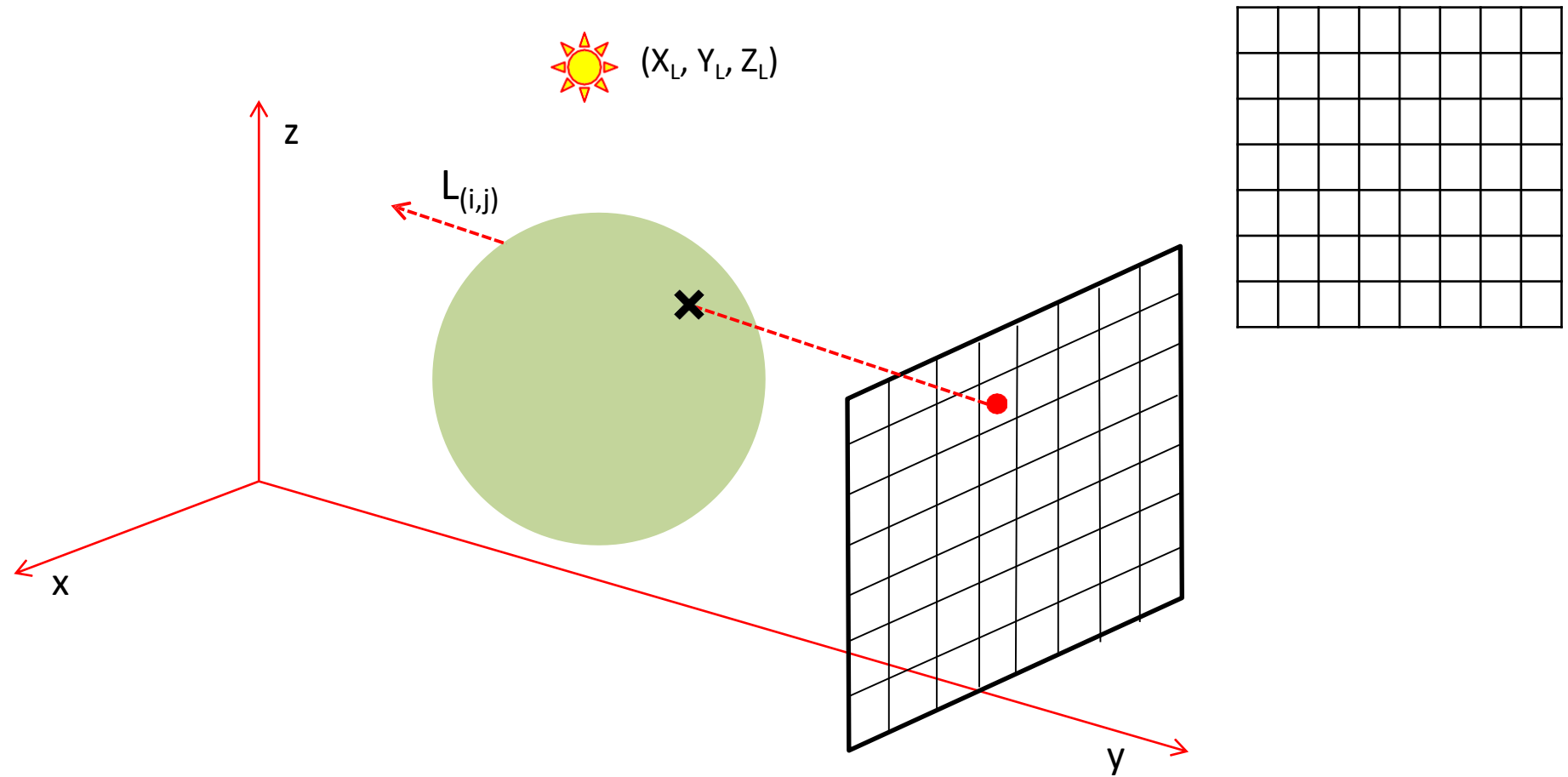
- Computing one pixel at a time
 - Each pixel “**looks**” in a **direction**
- Any object that is seen by a pixel
 - intersect “**viewing ray**”
 - viewing ray: line through that pixel is looking



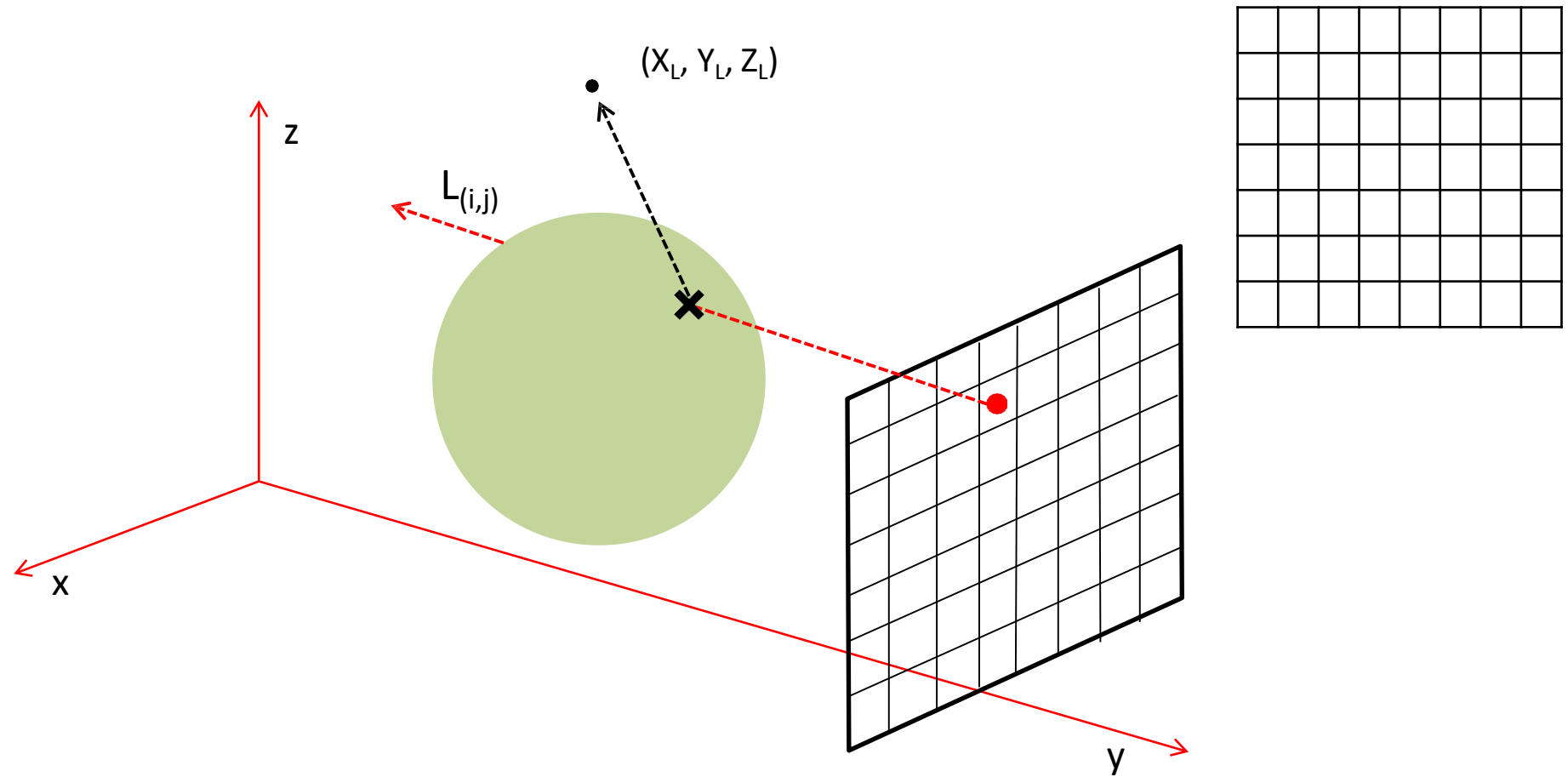
Ray-tracing Basics (10/15)

- ✓ ✱ Once that object is found, determine the color of the pixel.
 - a **shading computation** is need, that uses
 - the intersection point
 - surface normal (n)
 - other information

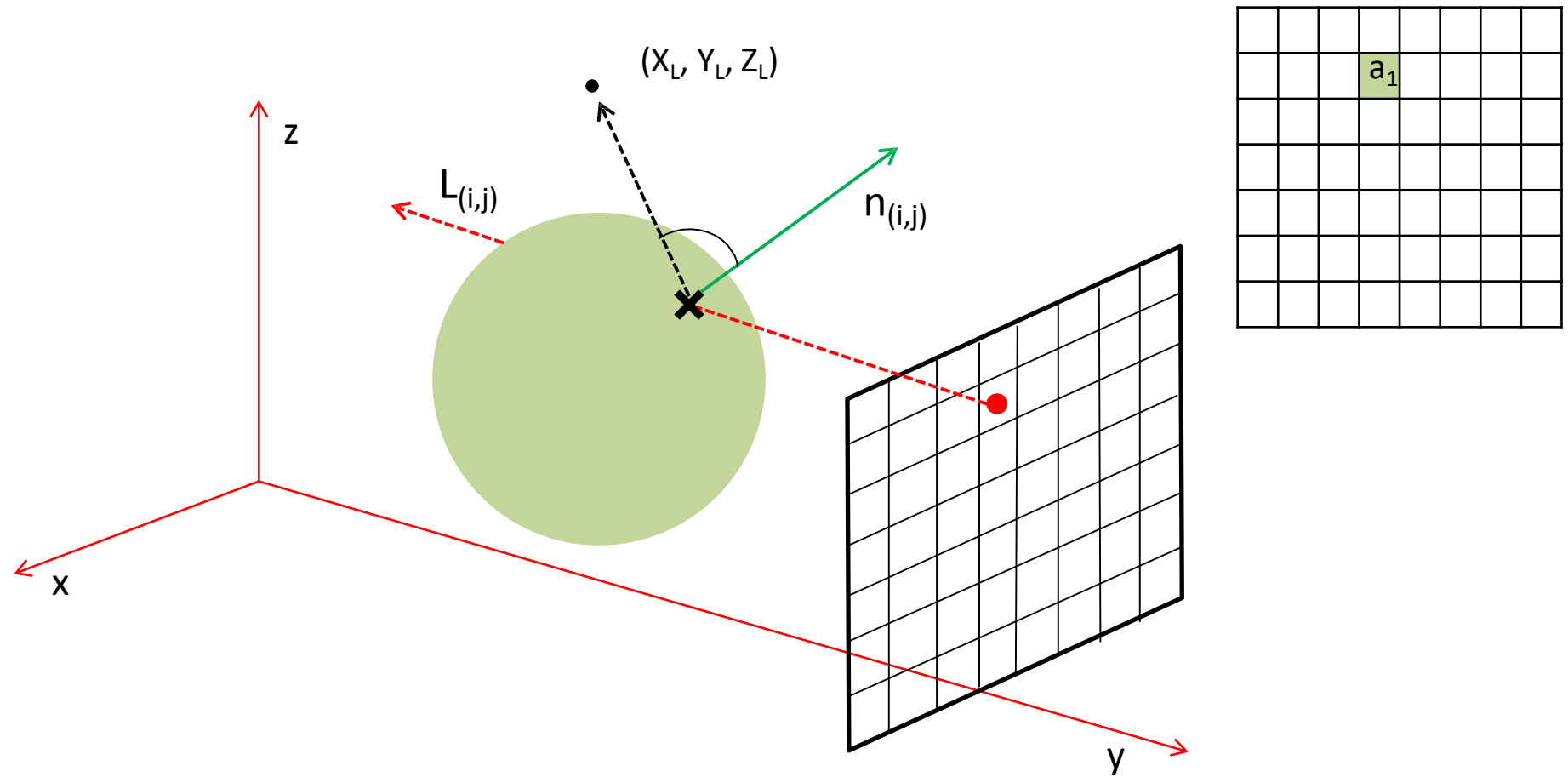
Ray-tracing Basics (11/15)



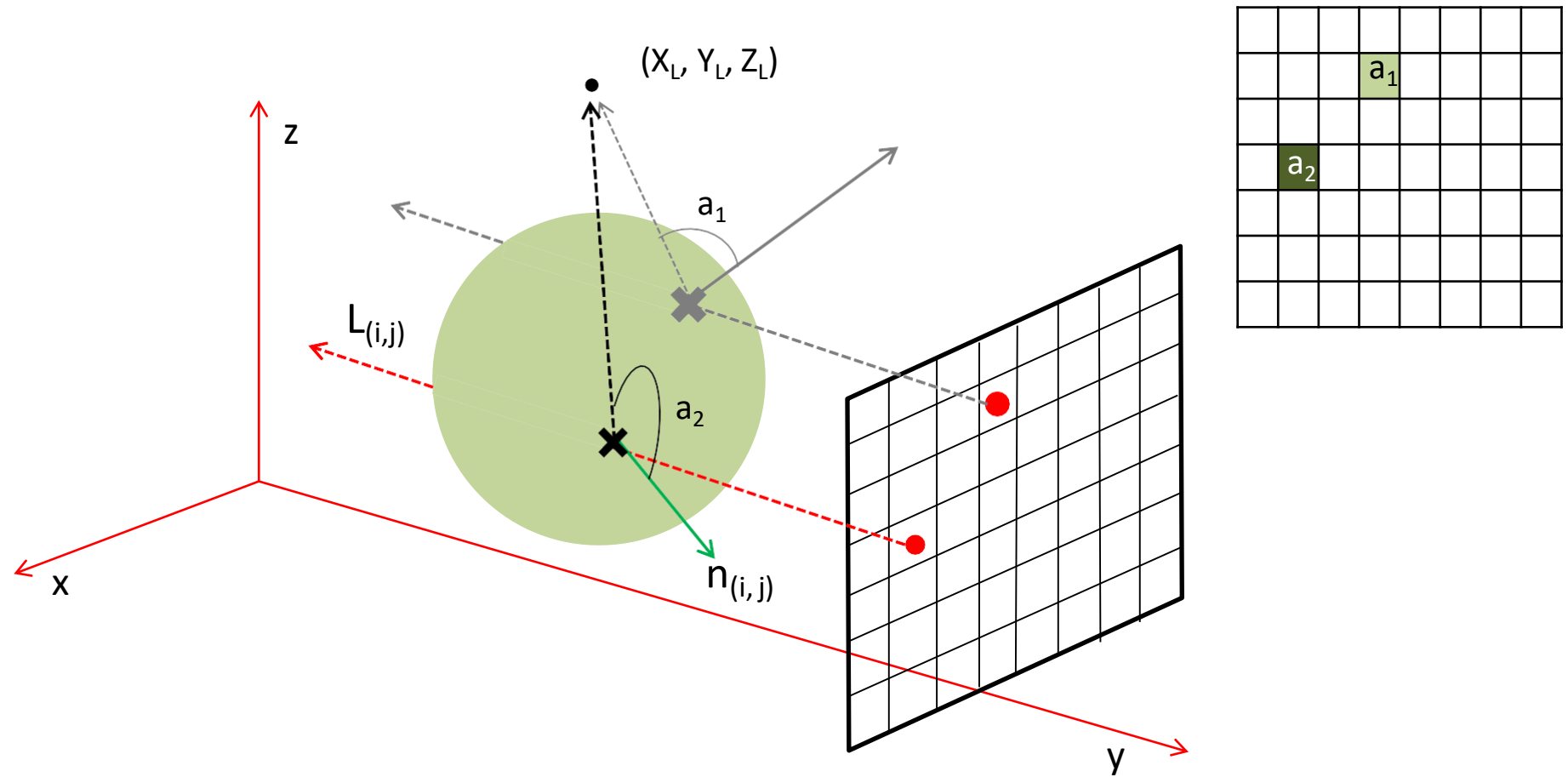
Ray-tracing Basics (12/15)



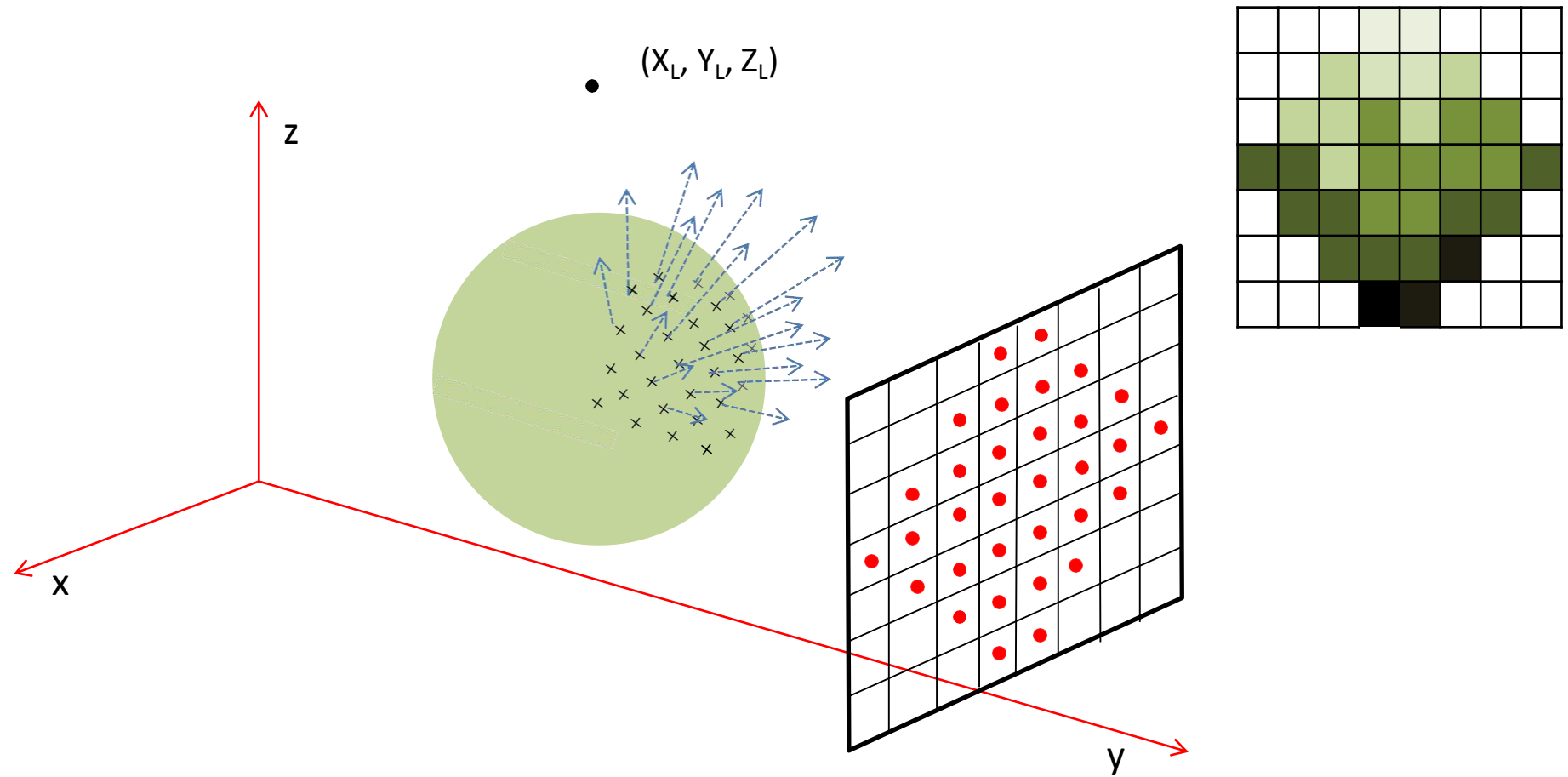
Ray-tracing Basics (13/15)



Ray-tracing Basics (14/15)



Ray-tracing Basics (15/15)



Ray-Tracing Algorithm (1/3)

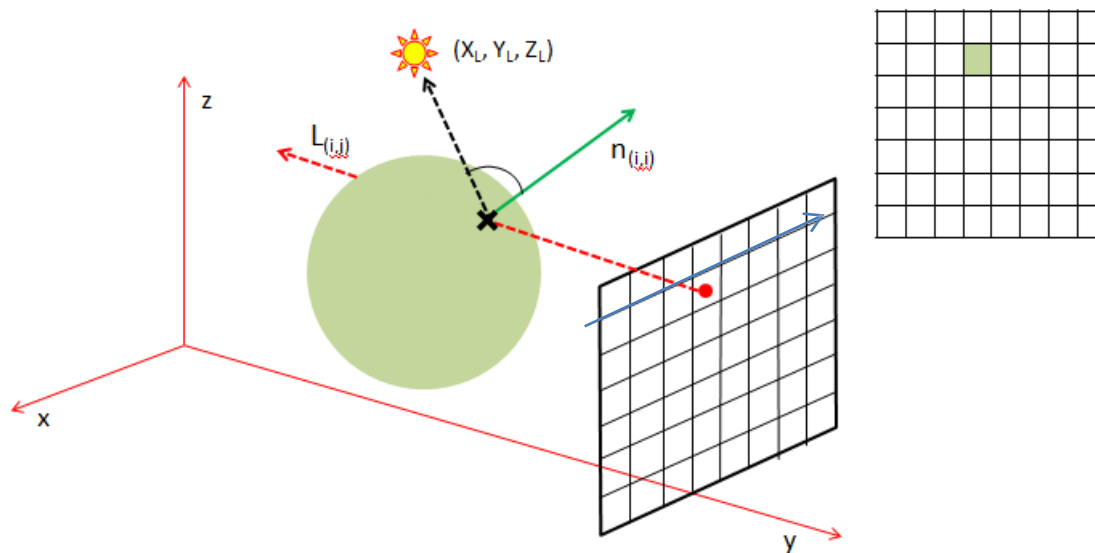


- A basic ray tracer therefore has three parts:
 - **ray generation:**
 - computes the origin and direction of each pixel's viewing ray.
 - **ray intersection:**
 - finds the closest object intersecting the viewing ray.
 - **shading:**
 - computes the pixel color based on the results of ray intersection.

Ray-Tracing Algorithm (2/3)

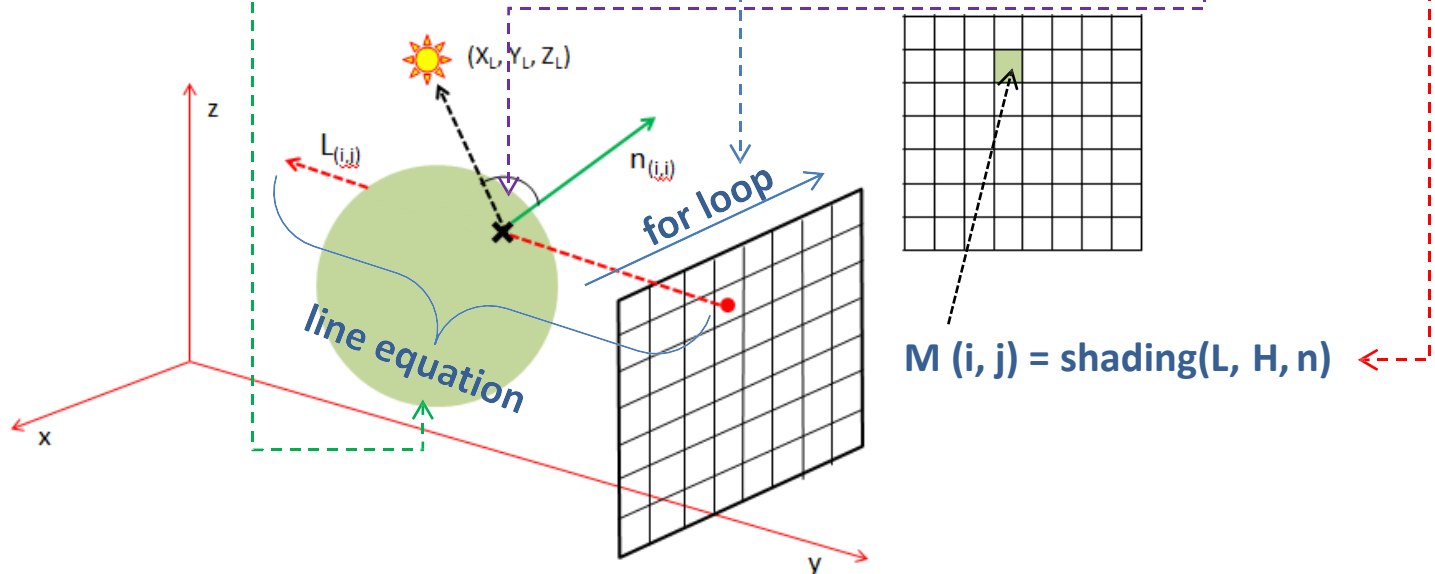
- **for each pixel do:**

- compute viewing ray
- find first object hit by ray and its surface normal $\mathbf{n}$
- set pixel color computed from hit point, light, and $\mathbf{n}$



Ray-Tracing Algorithm (3/3)

- **for each pixel do:**
 - compute **viewing ray**
 - find first object hit by ray and its **surface normal $\mathbf{n}$**
 - set pixel color computed from **hit point, light, and $\mathbf{n}$**



Practice Problems

1. Is the projected image on the image plane in the given example perspective?
2. Consider the following setup*:
 - Image plane: Situated at $y = 13$, parallel to ZX plane, (Resolution: 11×11), M is the corresponding array and Y -axis goes through $(6, 6)$.
 - Sphere: Center at $(0, 0, 0)$, $Radius = 6$.
 - Light: at $(0, 15, 0)$.

Now –

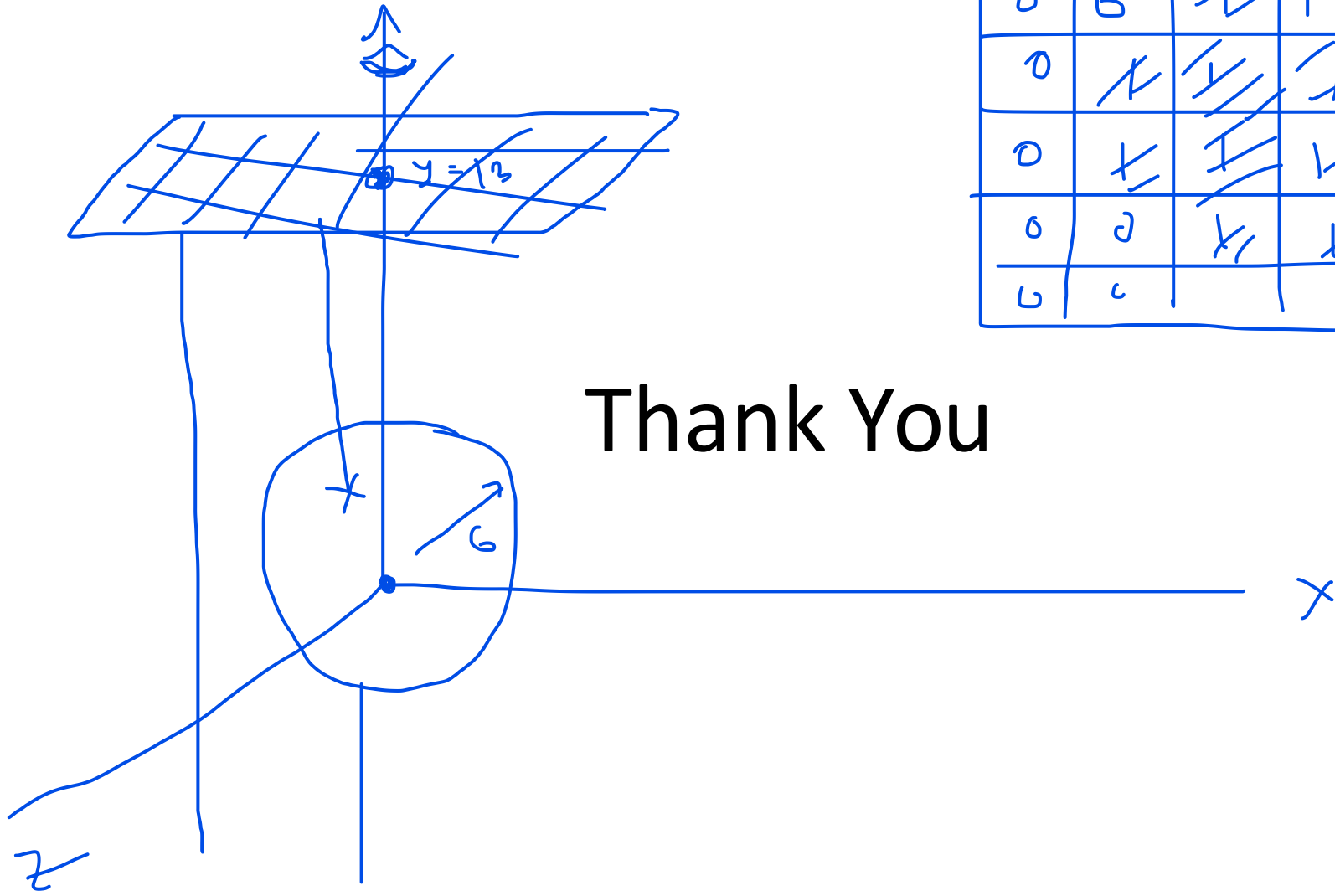
- a) Draw the ray-tracing setup showing two viewing rays (one hitting, another missing).
- b) Fill up the array (pixel) with 1 (for hitting) and 0 (for missing). Show the hitting/ missing mathematically for at least one pixel.
- c) Fill up the array (pixel) with angles between surface normal and viewing ray. Show the angle calculation for at least one pixel.

* This problem can be helpful for understanding basic ray-tracing algorithm from the scratch.

$(6, 6)$

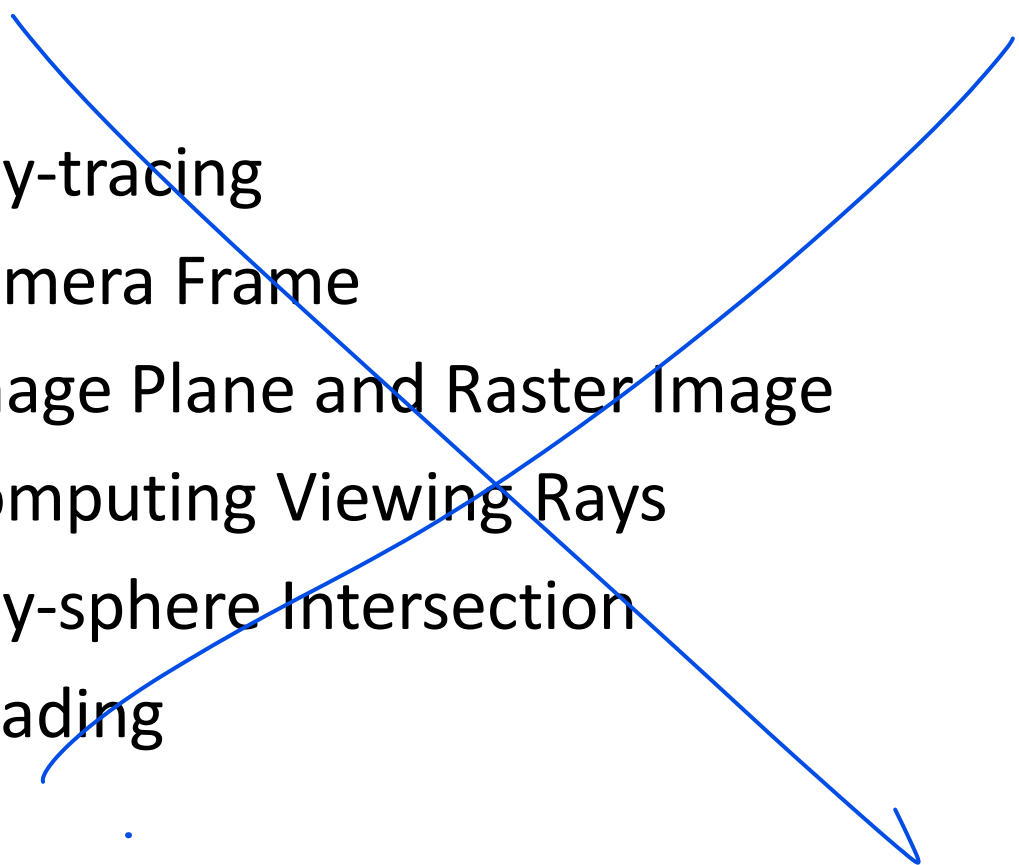
0	0	✓	✓	
0	✓	✓	✓	✓
0	✓	✓	✓	✓
0	0	✓	✓	✓
6	6			

Thank You



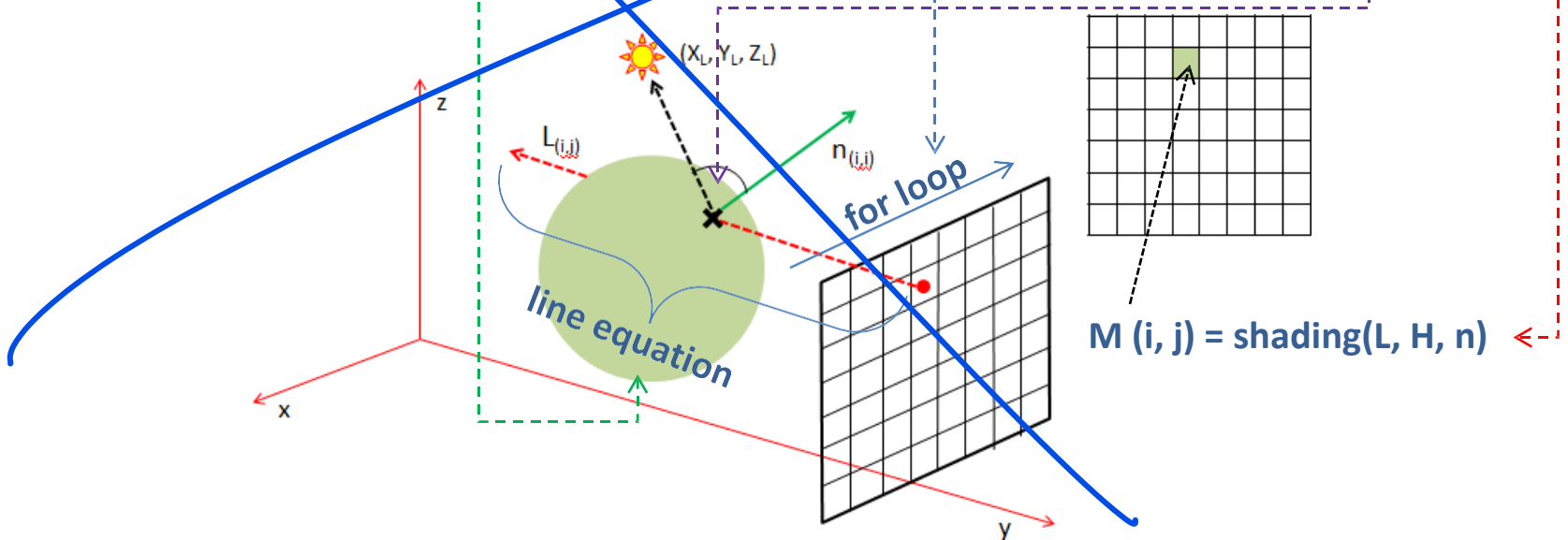
CSE4203: Computer Graphics
Chapter – 4 (part - C)
Ray Tracing

Outline

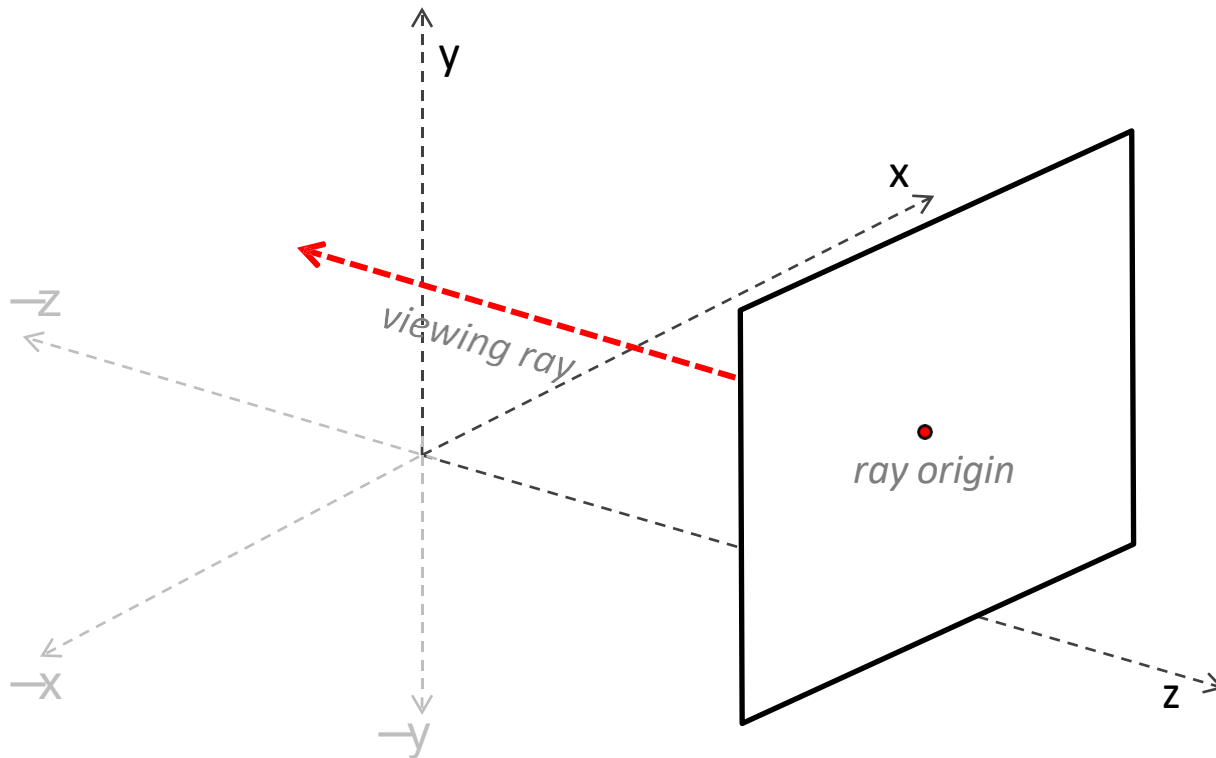
- Ray-tracing
 - Camera Frame
 - Image Plane and Raster Image
 - Computing Viewing Rays
 - Ray-sphere Intersection
 - Shading
- 

Ray-Tracing Algorithm

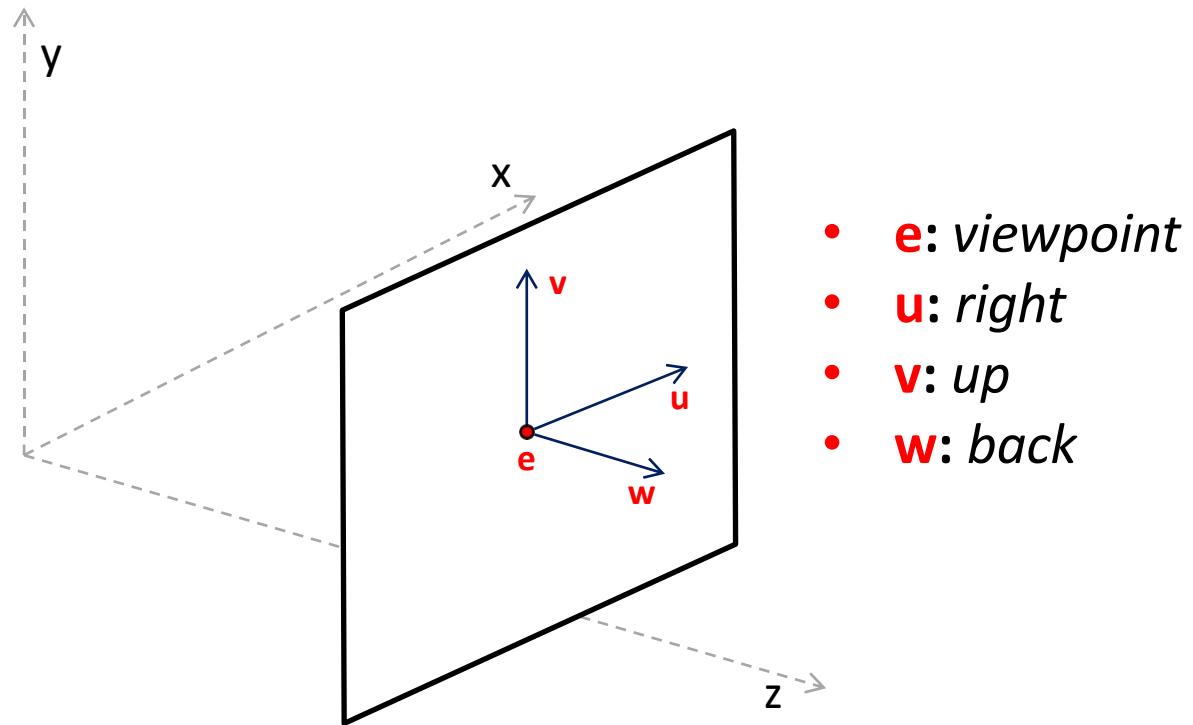
- **for each pixel do:**
 - compute **viewing ray**
 - find first object hit by ray and its **surface normal $\mathbf{n}$**
 - set pixel color computed from **hit point, light, and $\mathbf{n}$**



Camera Frame (1/6)

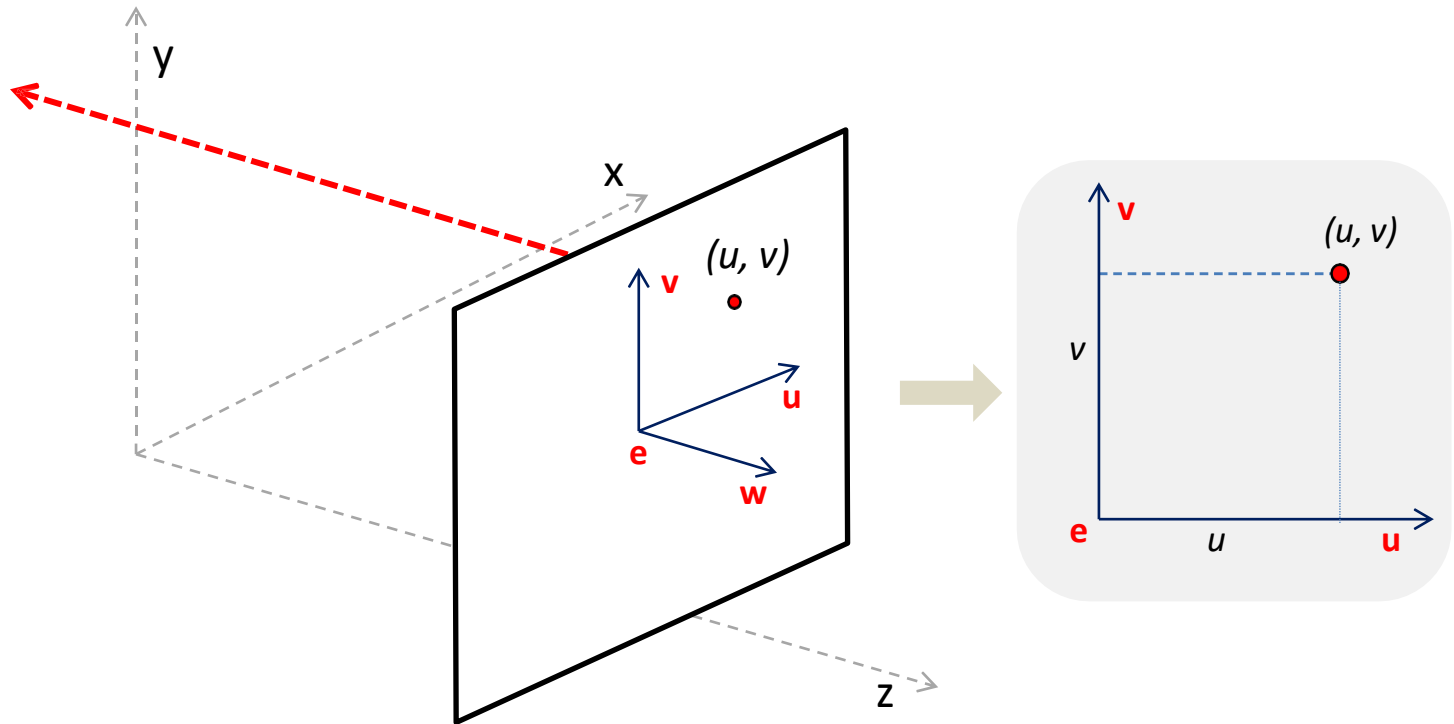


Camera Frame (2/6)

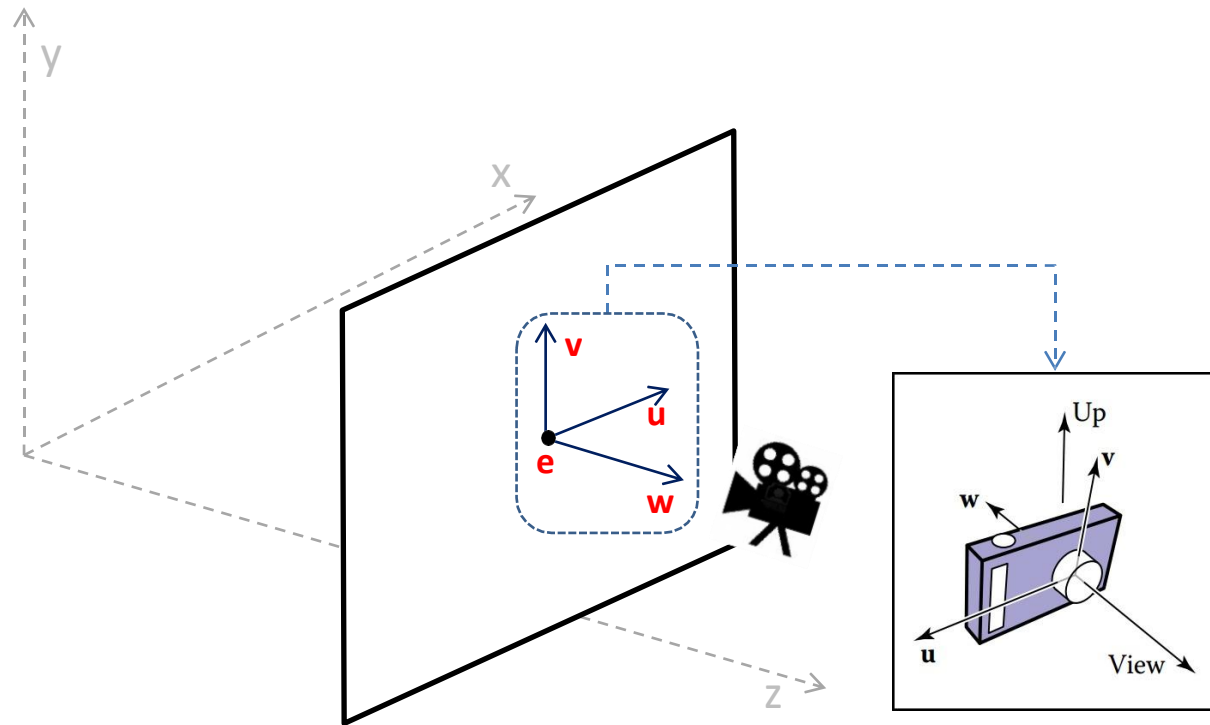


Camera Frame (3/6)

- ray origin = $\mathbf{e} + u \mathbf{u} + v \mathbf{v}$
 - ray direction = $-\mathbf{w}$

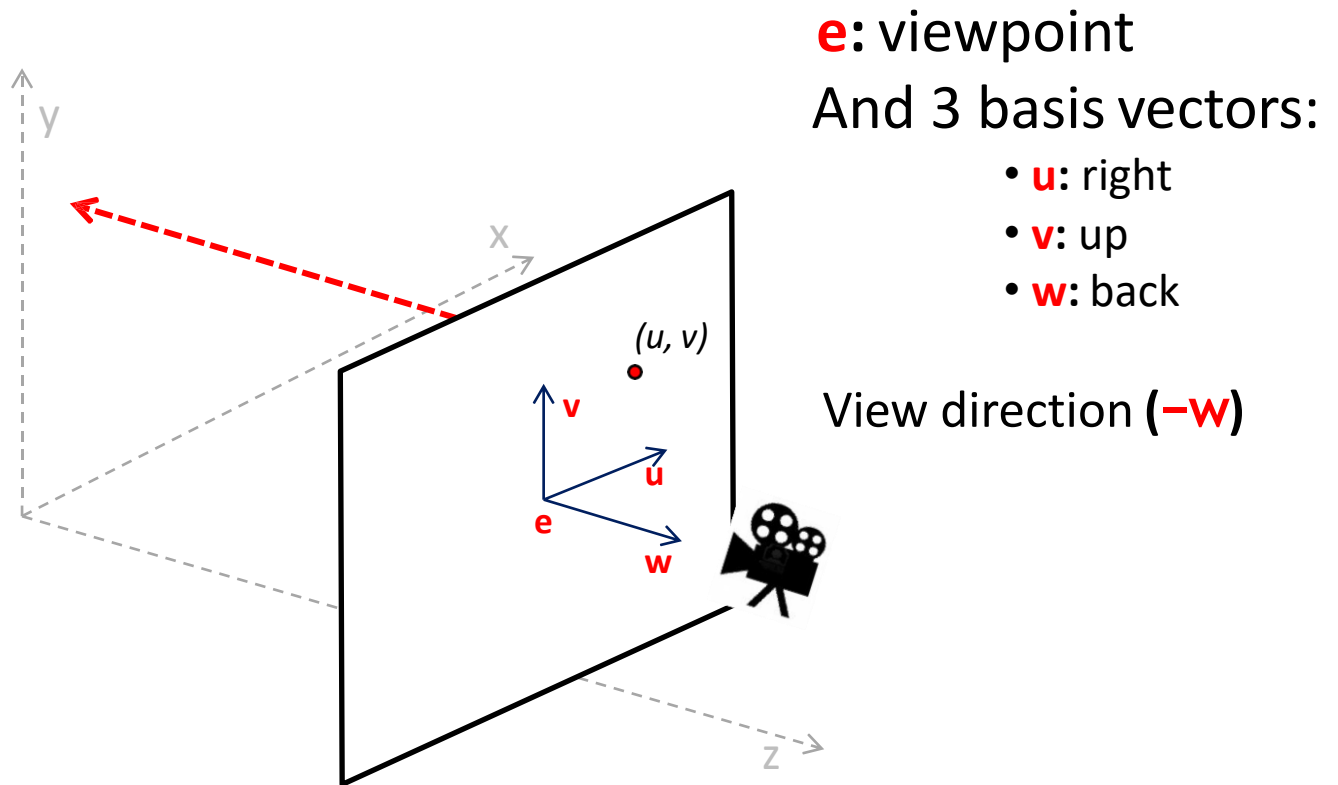


Camera Frame (4/6)



Camera Frame (5/6)

Camera frame: (Camera coordinate)



Camera Frame (6/6)

Orthographic:

- ray direction = $-\mathbf{w}$
- ray origin = $\mathbf{e} + u \mathbf{u} + v \mathbf{v}$

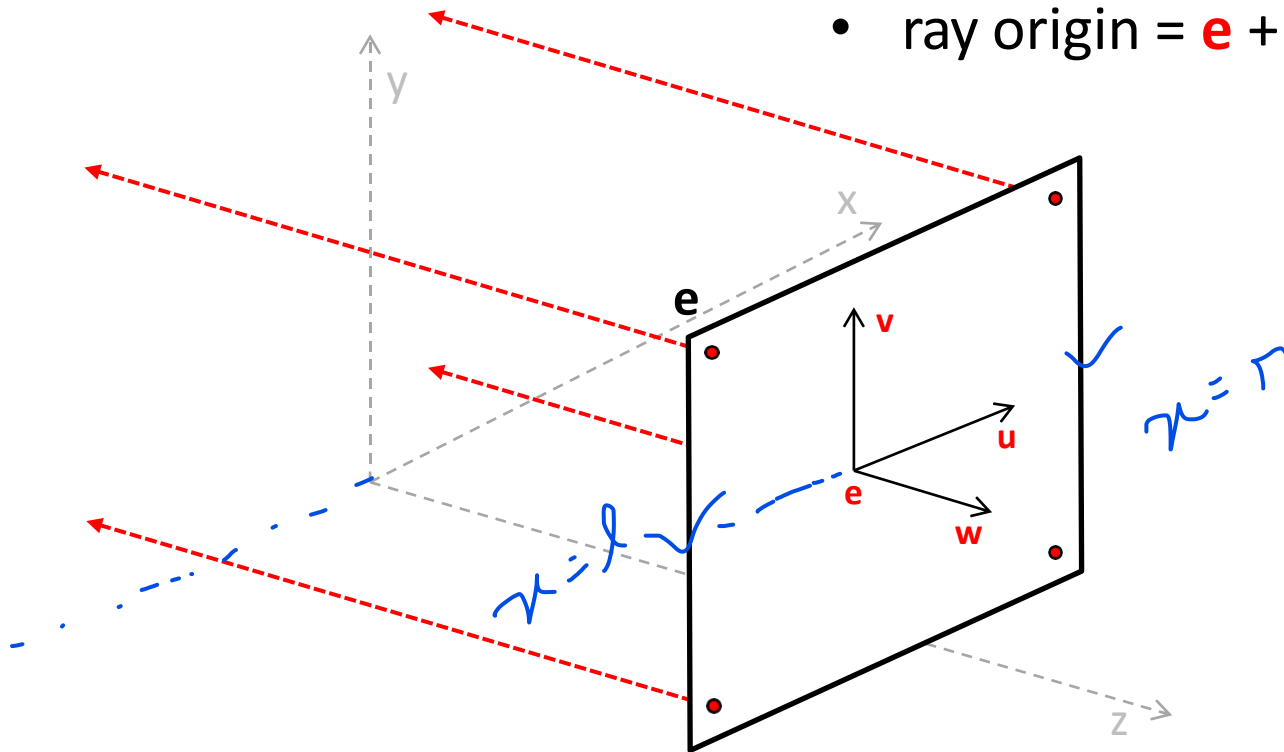


Image Plane (1/4)

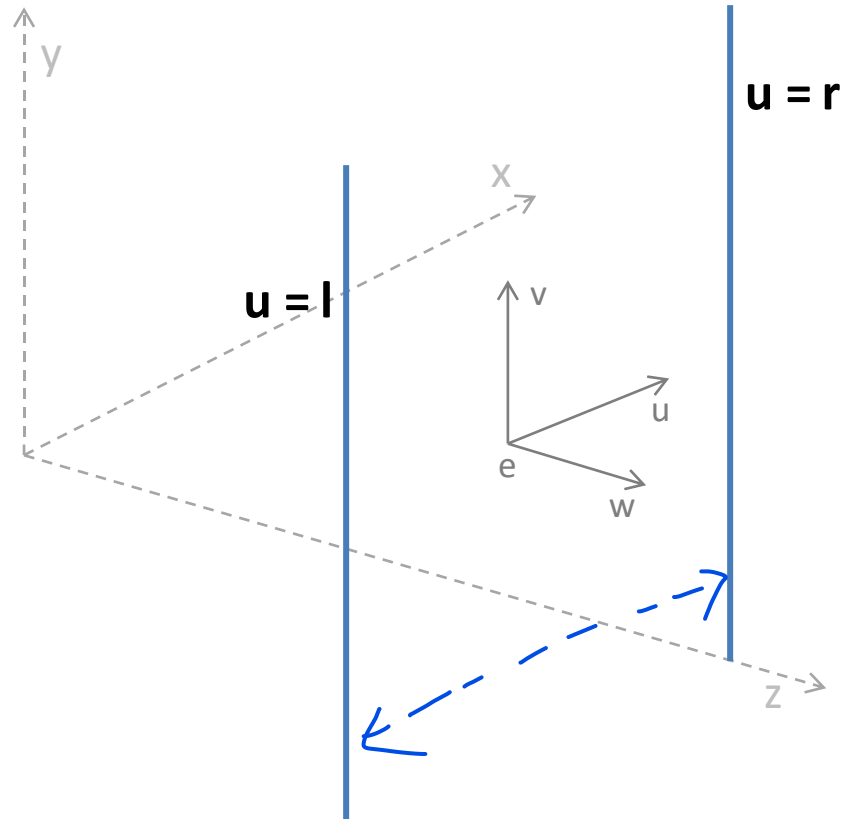


Image Plane (2/4)

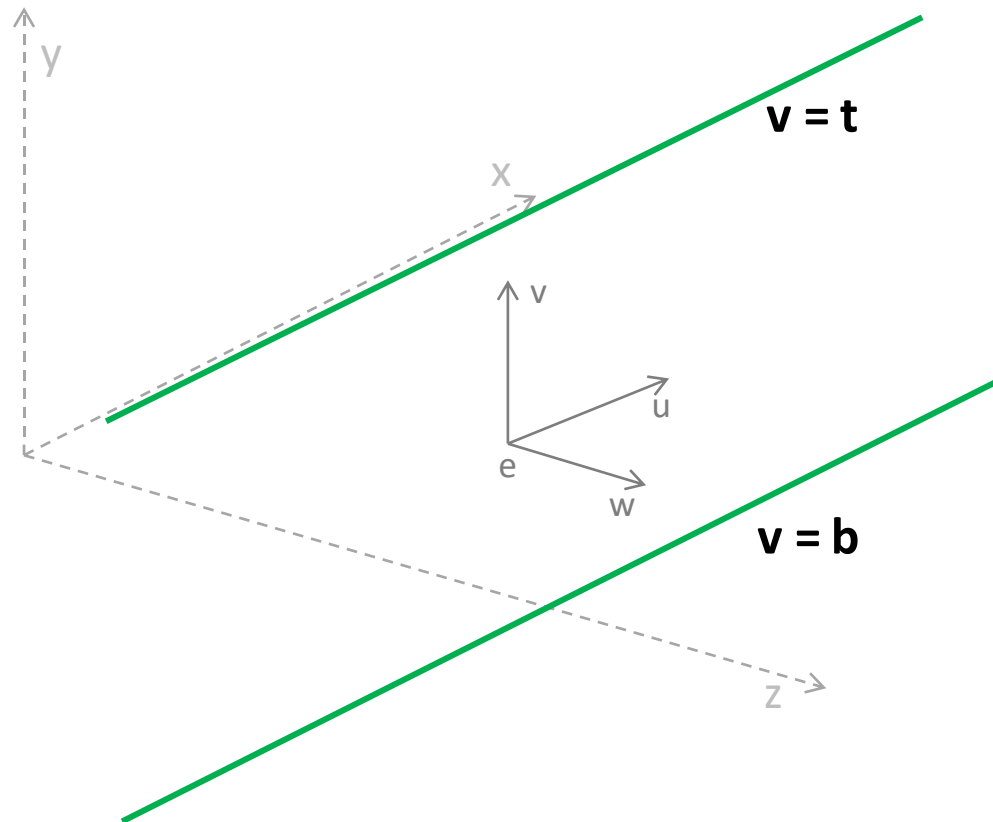


Image Plane (3/4)

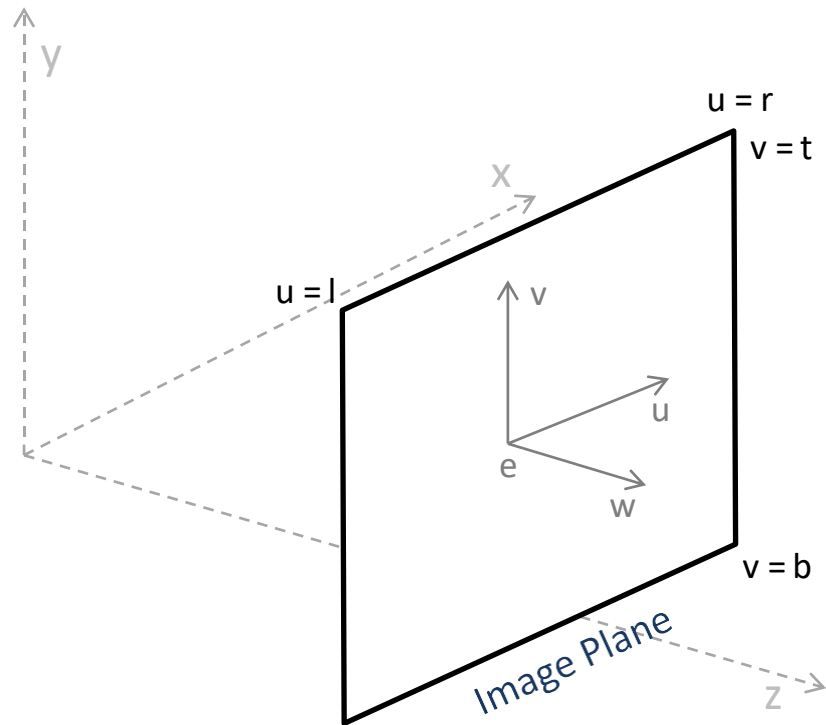
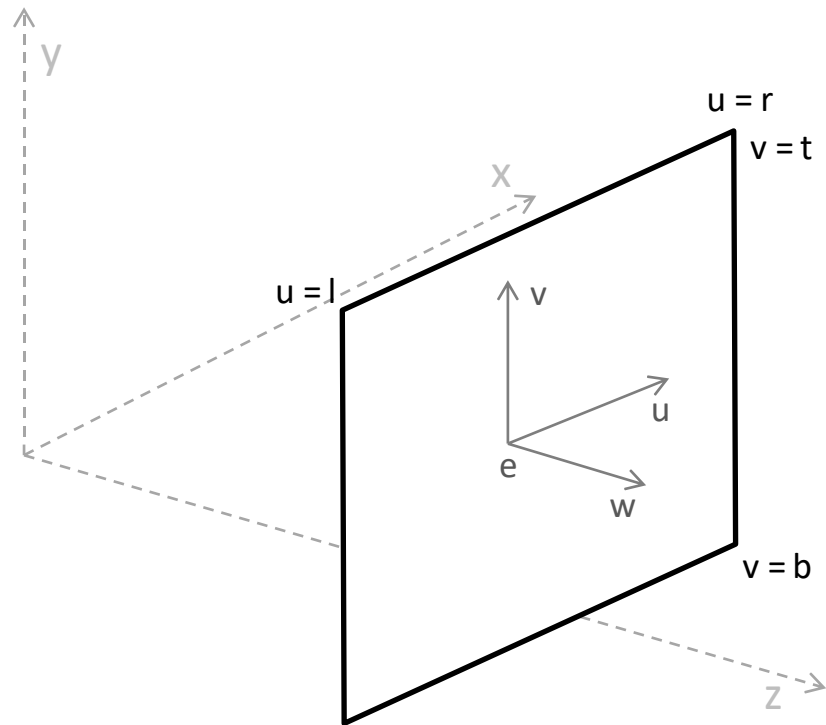
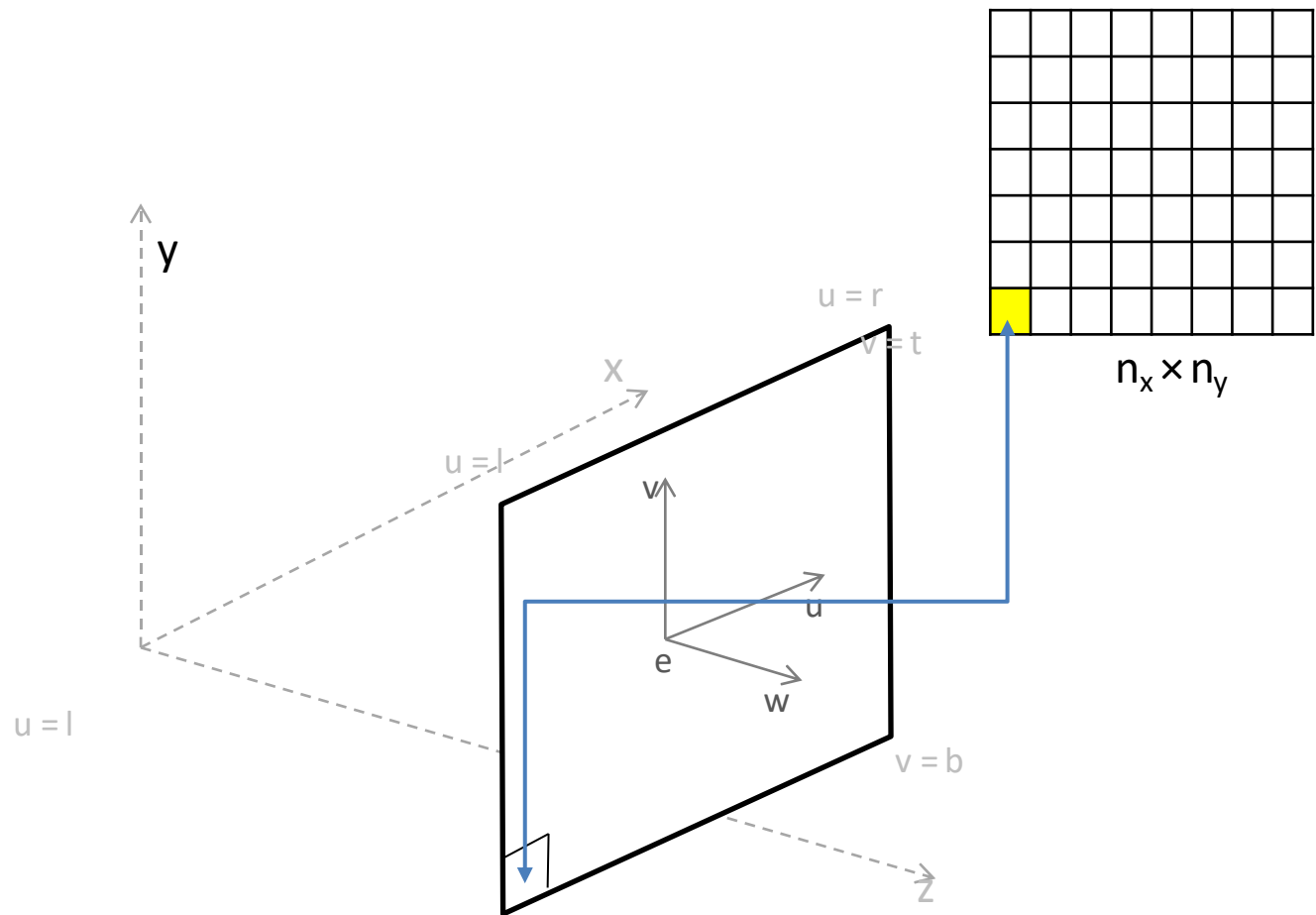


Image Plane (4/4)

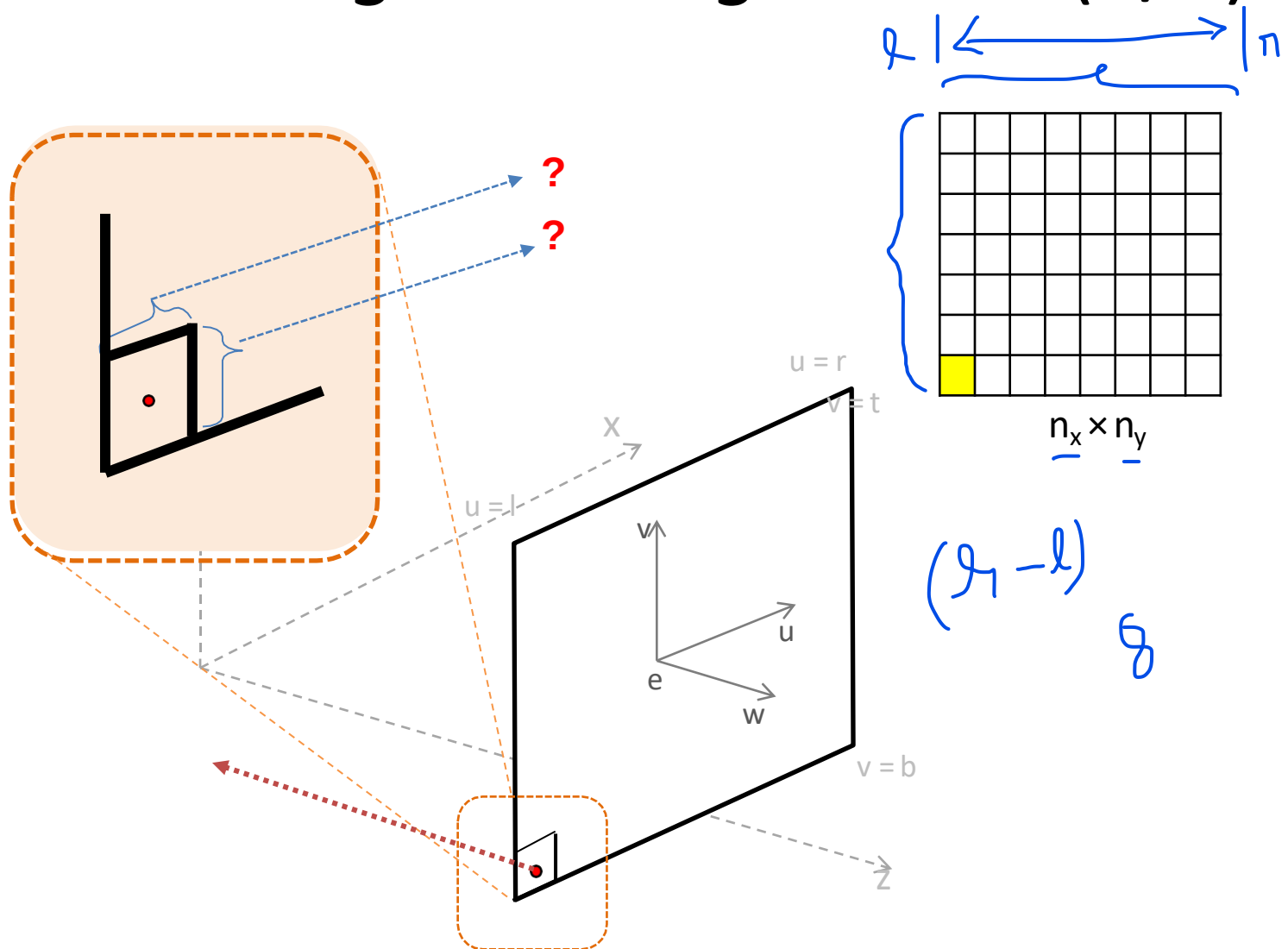
Q: determine the area of the image plane in terms of l , r , t and b .



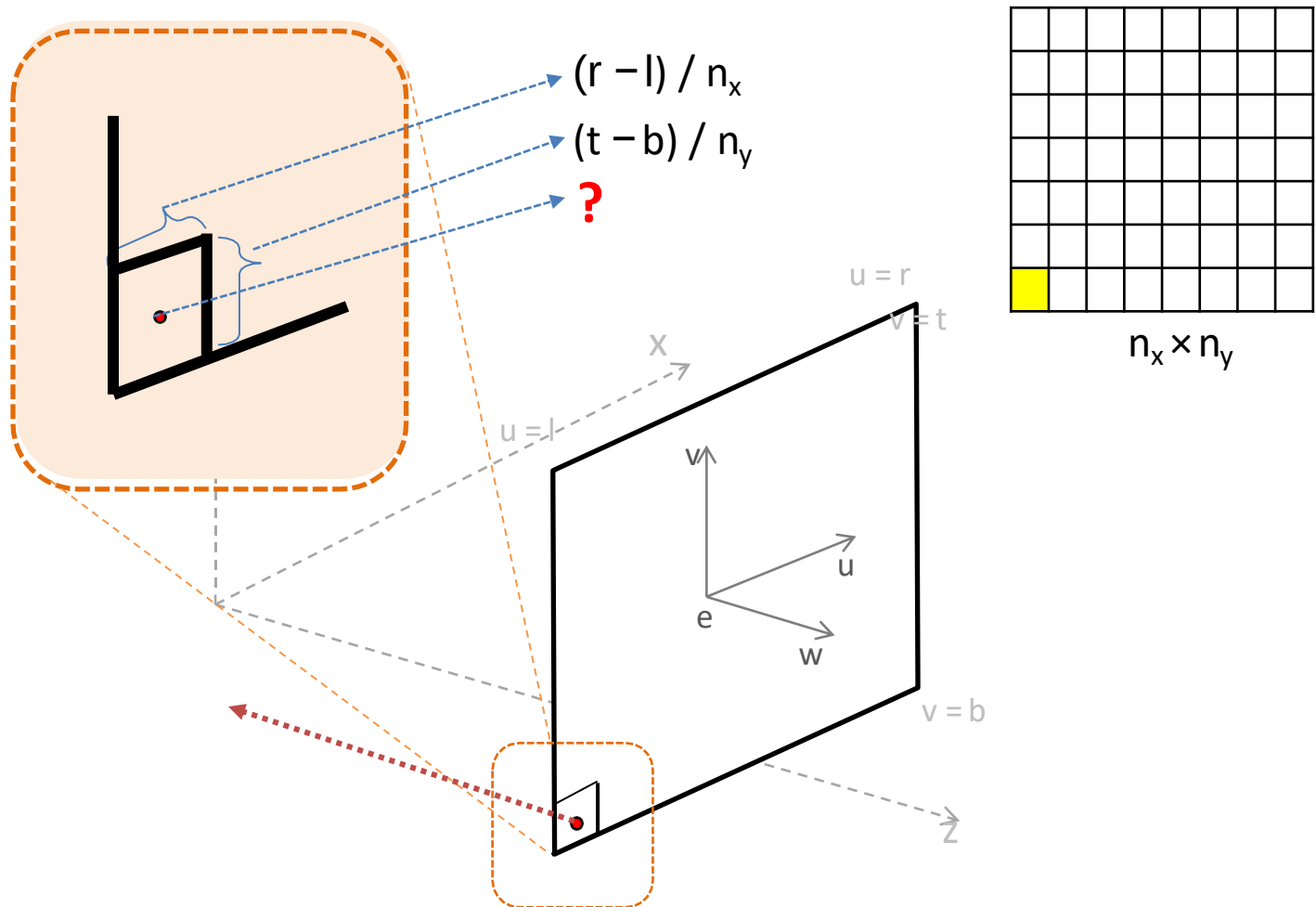
Raster Image $\leftrightarrow$ Image Plane (1/8)



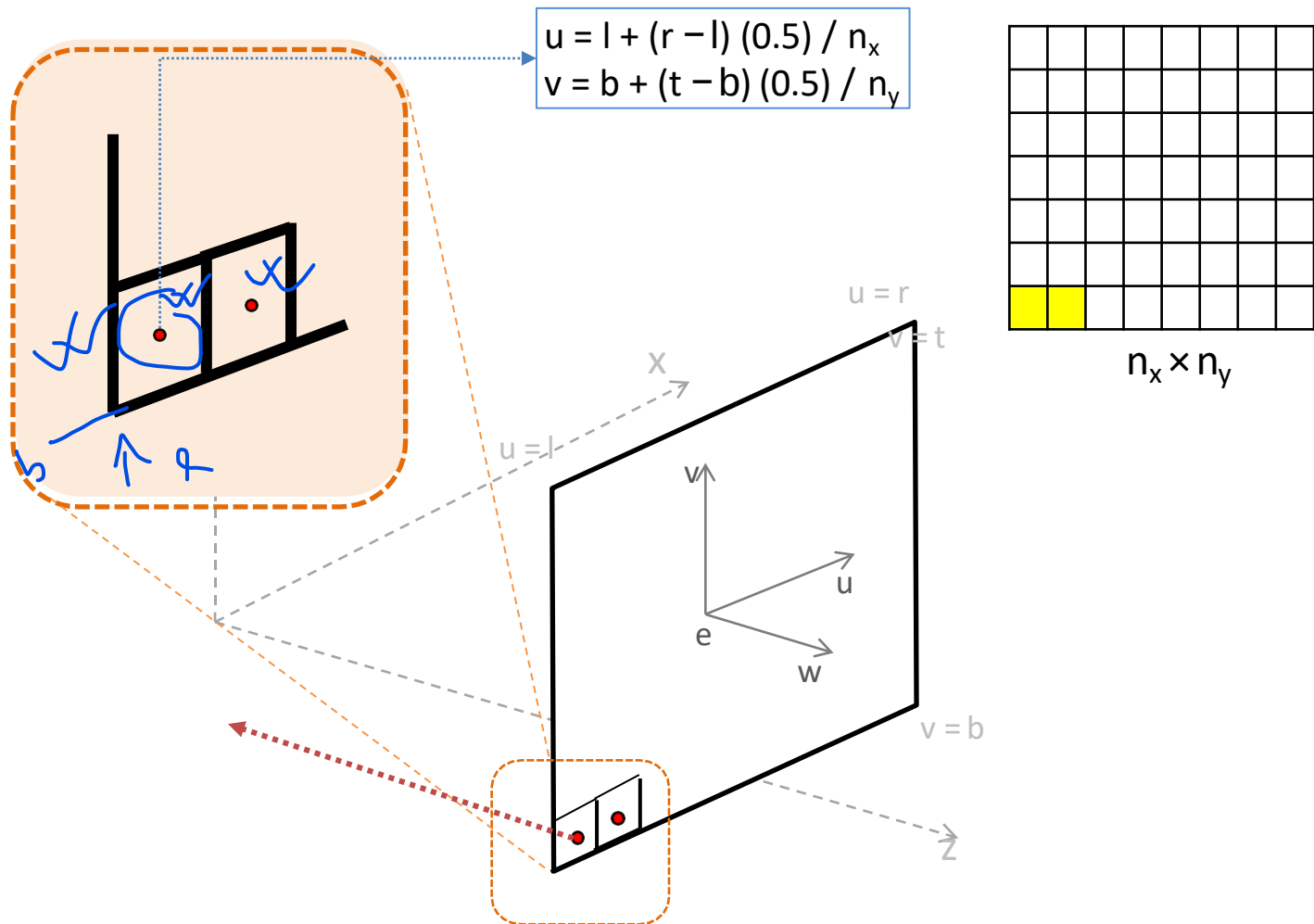
Raster Image $\leftrightarrow$ Image Plane (2/8)



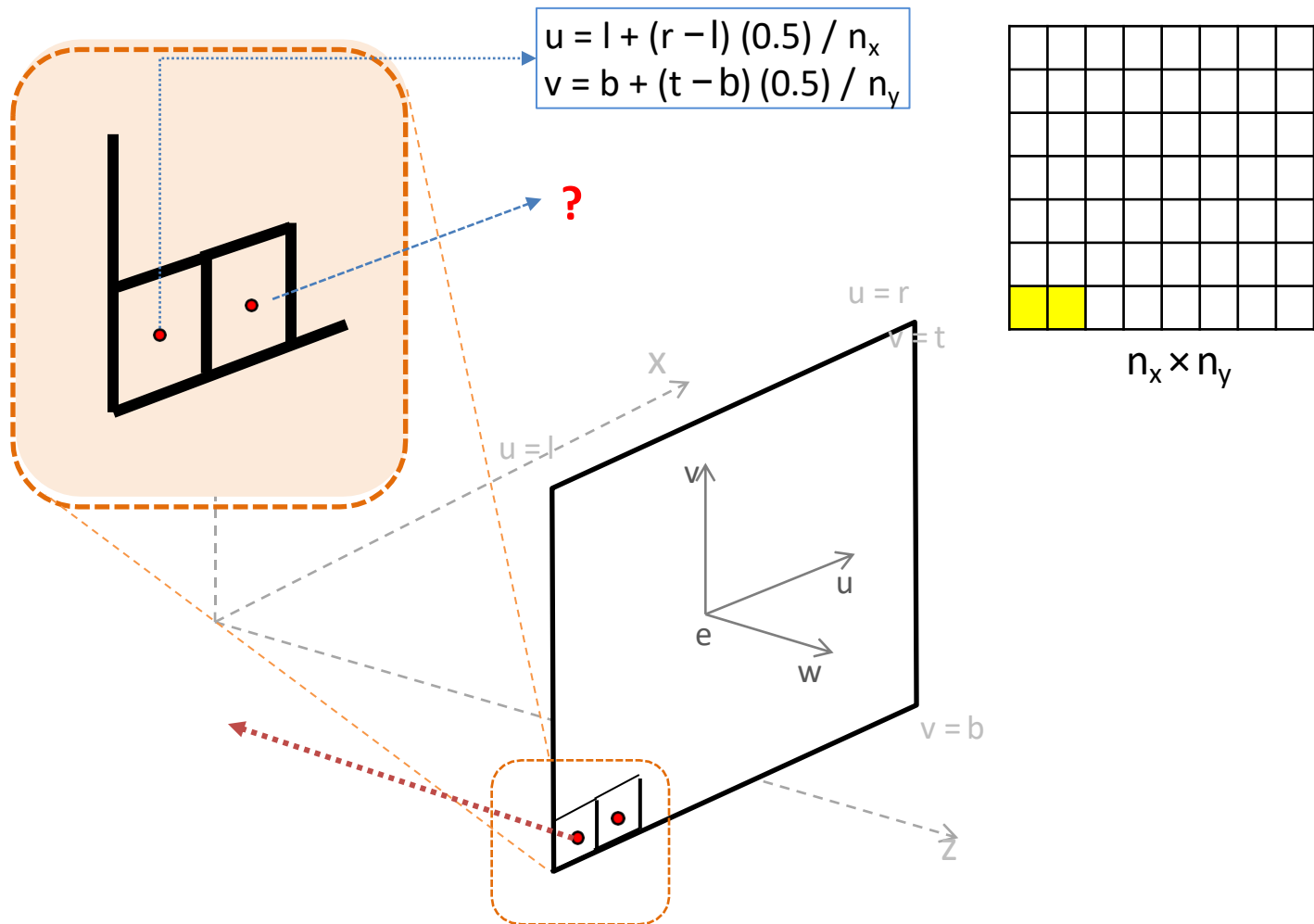
Raster Image $\leftrightarrow$ Image Plane (3/8)



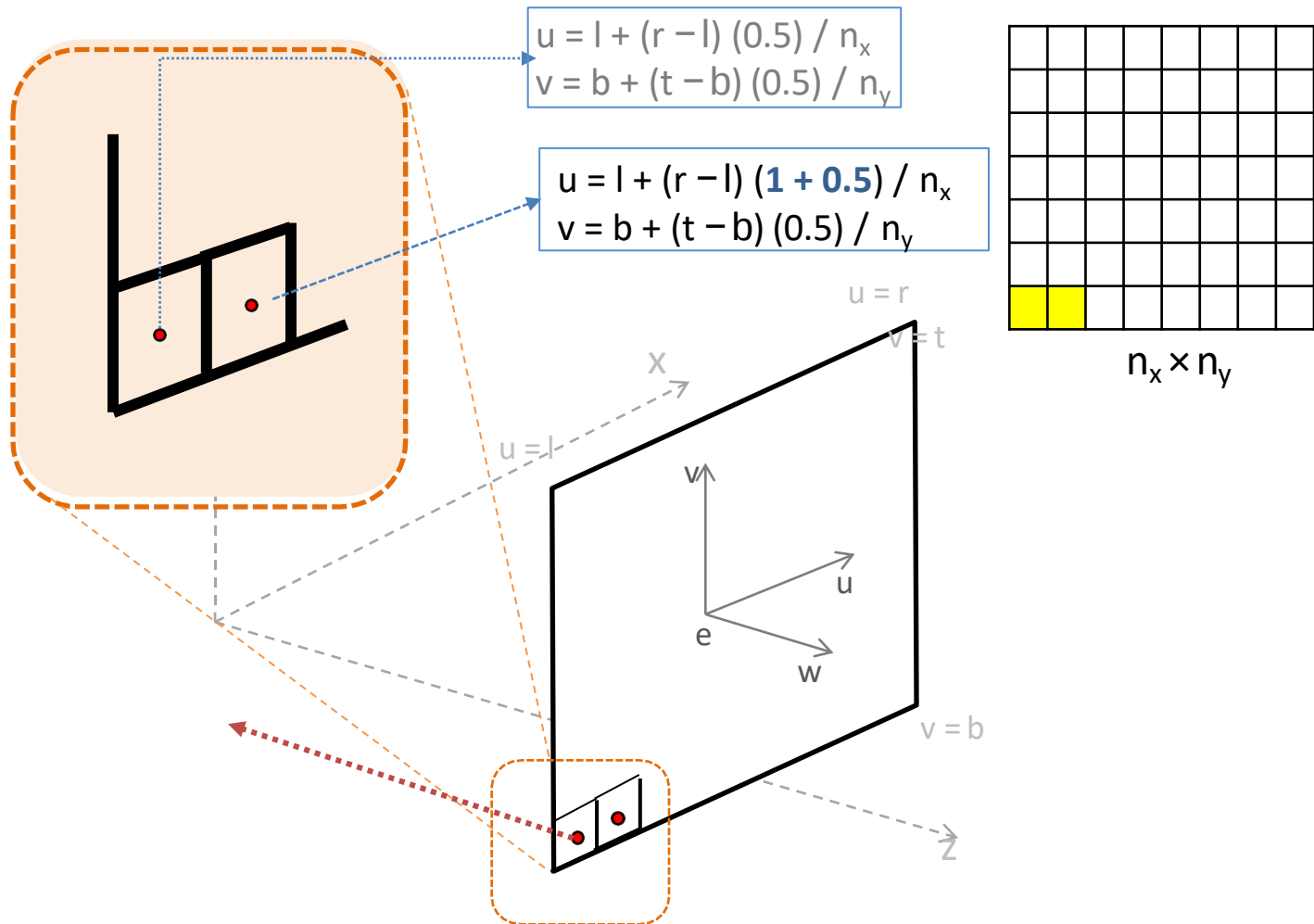
Raster Image $\leftrightarrow$ Image Plane (4/8)



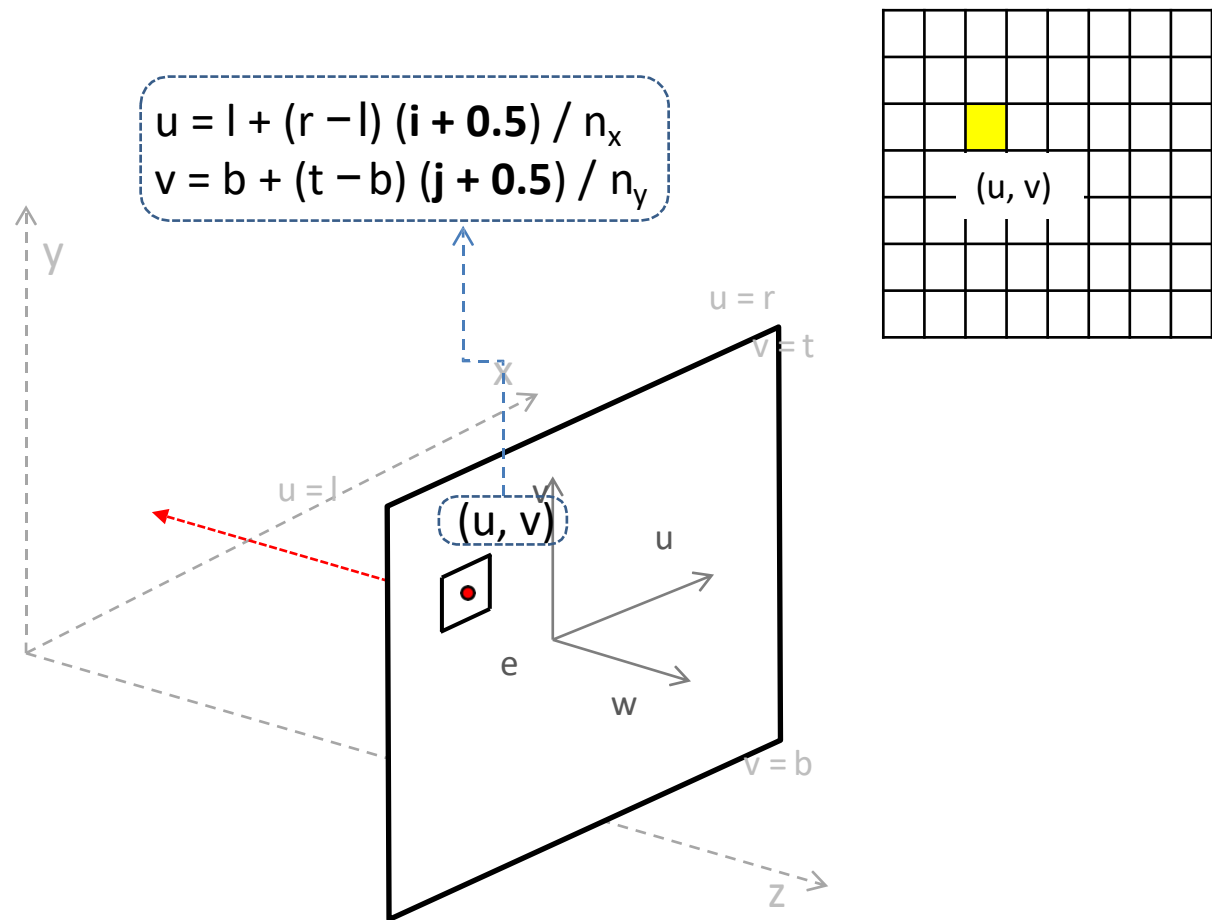
Raster Image $\leftrightarrow$ Image Plane (5/8)



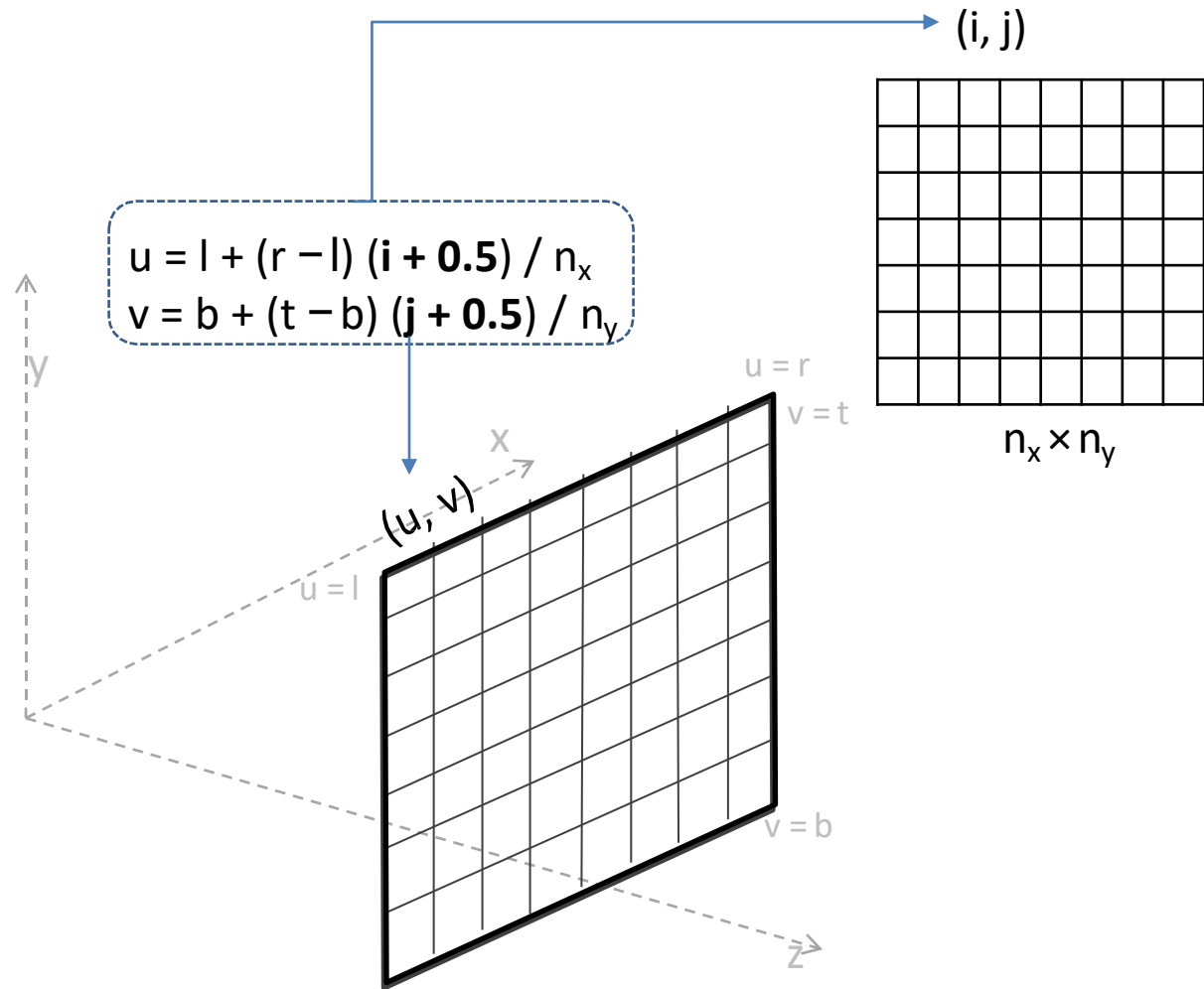
Raster Image $\leftrightarrow$ Image Plane (6/8)



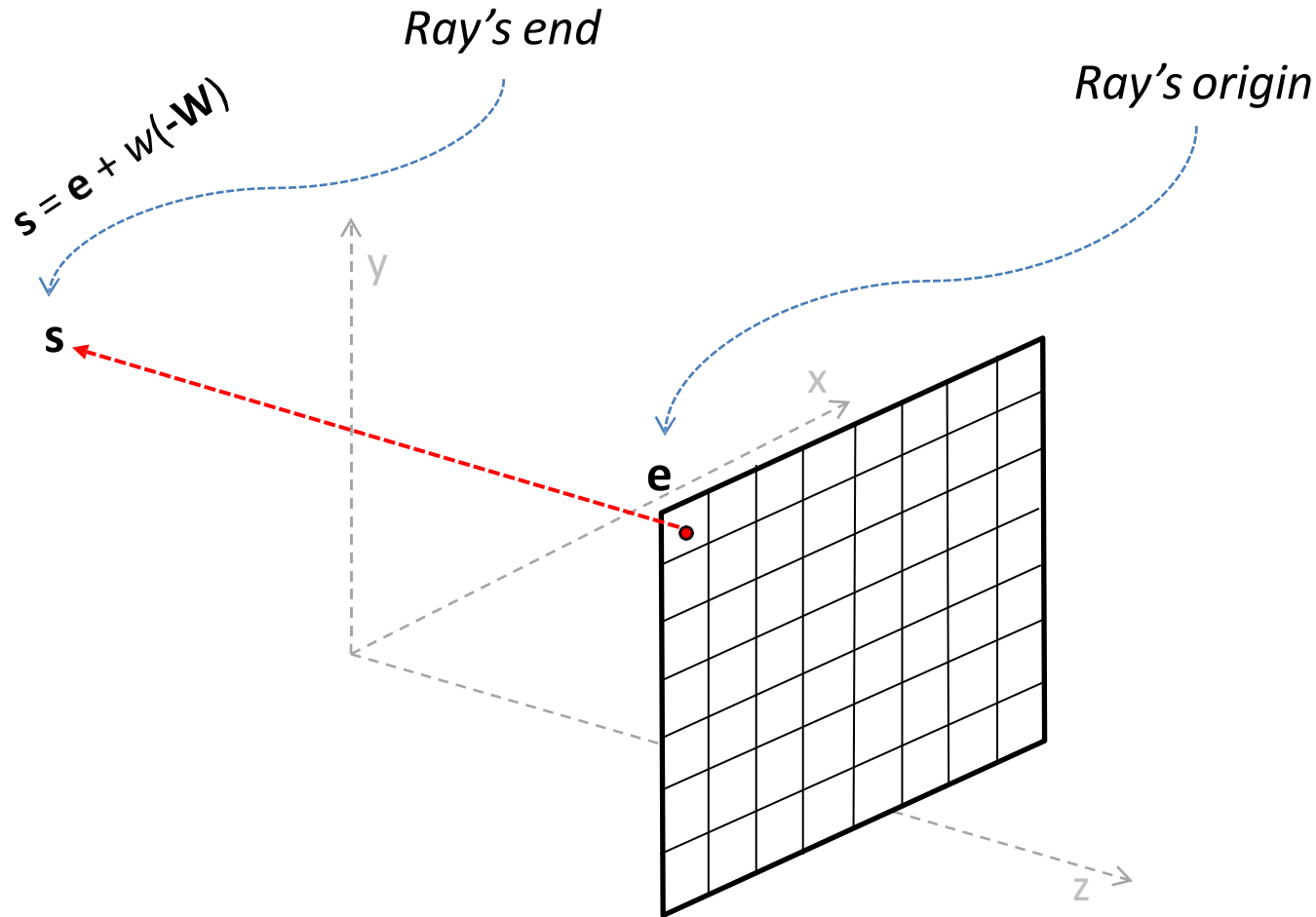
Raster Image $\leftrightarrow$ Image Plane (7/8)



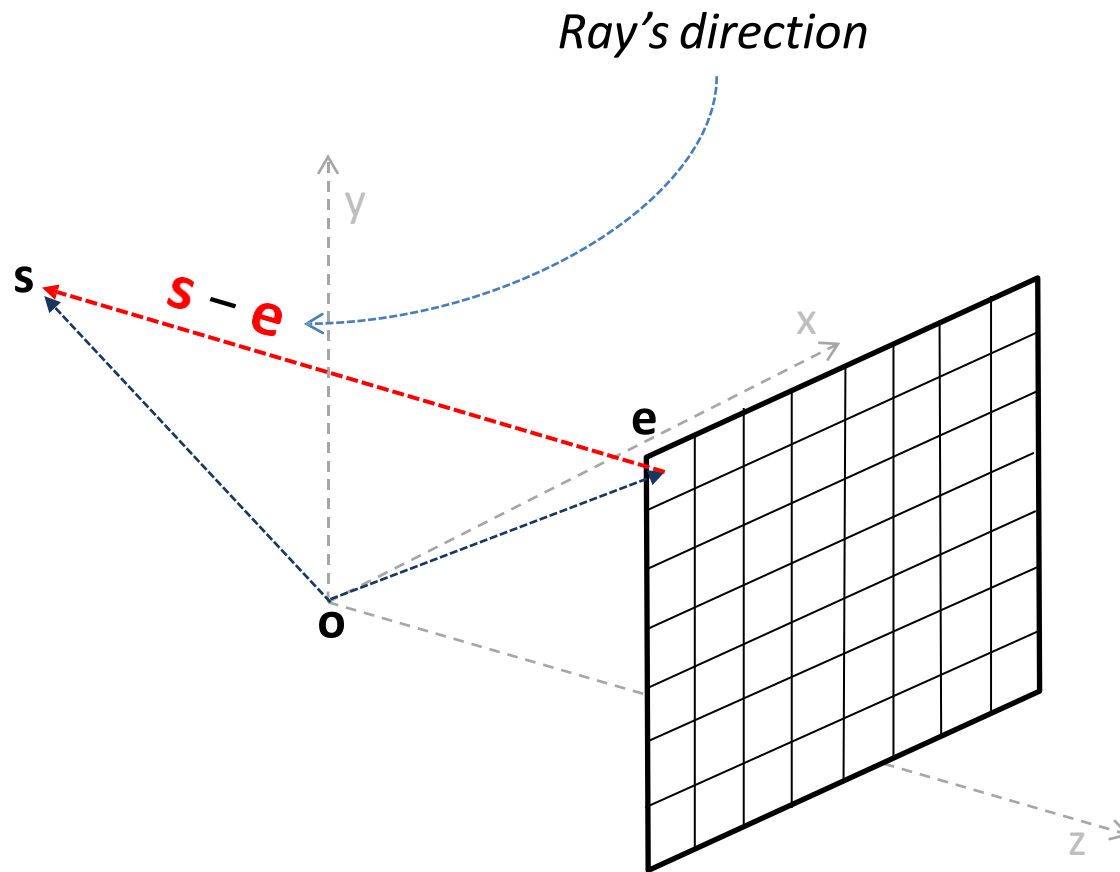
Raster Image $\leftrightarrow$ Image Plane (8/8)



Computing Viewing Rays (1/4)



Computing Viewing Rays (2/4)

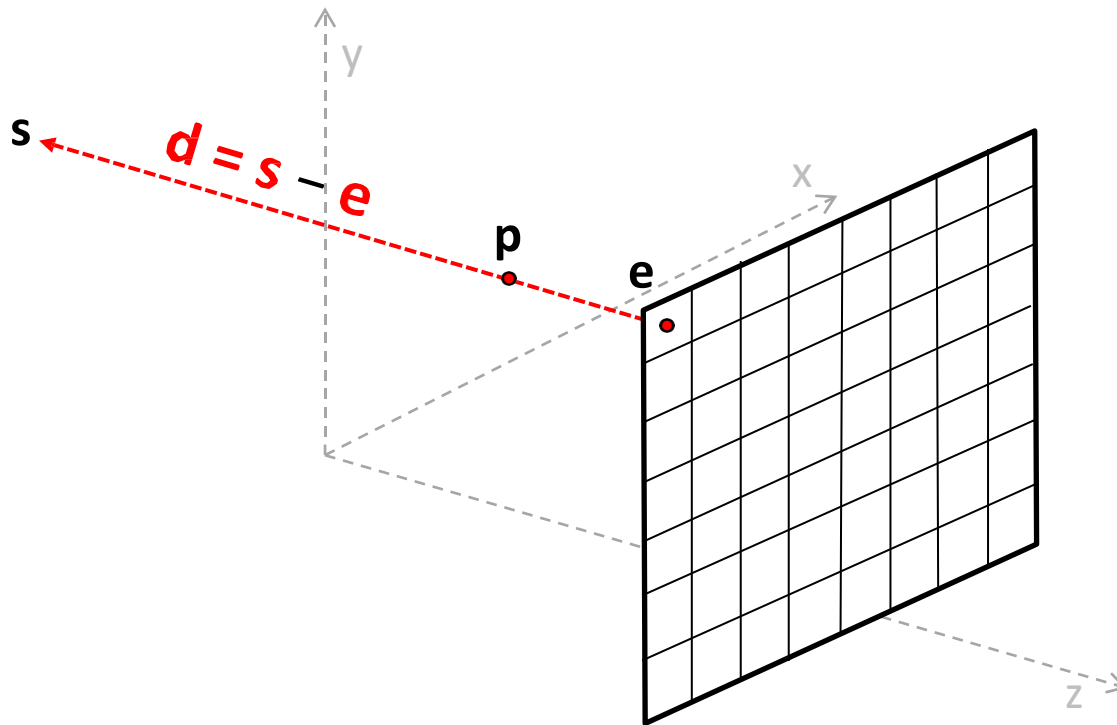


Computing Viewing Rays (3/4)

$$\begin{aligned}\mathbf{p} &= \mathbf{e} + t(\mathbf{s} - \mathbf{e}) \\ &= \mathbf{e} + t\mathbf{d}\end{aligned}$$

- Advancing from $\mathbf{e}$ along the vector $(\mathbf{s} - \mathbf{e})$

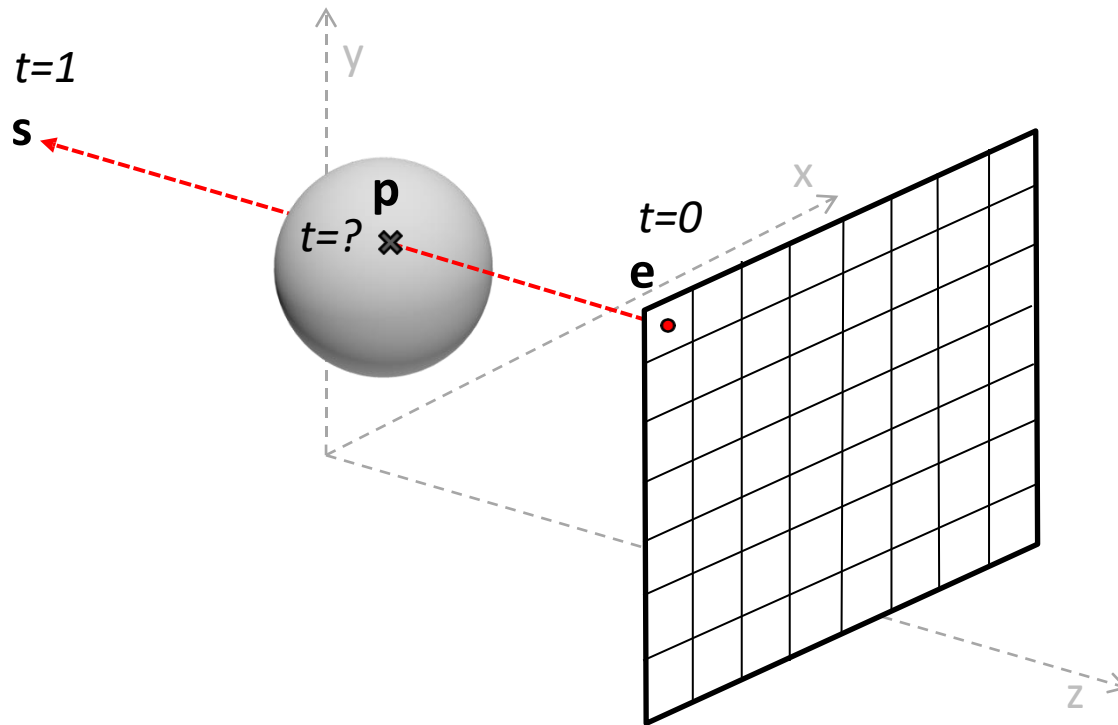
- With a fractional distance t to find the point $\mathbf{p}$



Computing Viewing Rays (4/4)

$$\begin{aligned}\mathbf{p} &= \mathbf{e} + t(\mathbf{s} - \mathbf{e}) \\ &= \mathbf{e} + t\mathbf{d}\end{aligned}$$

- We can use t to determine the intersection point $\mathbf{p}$

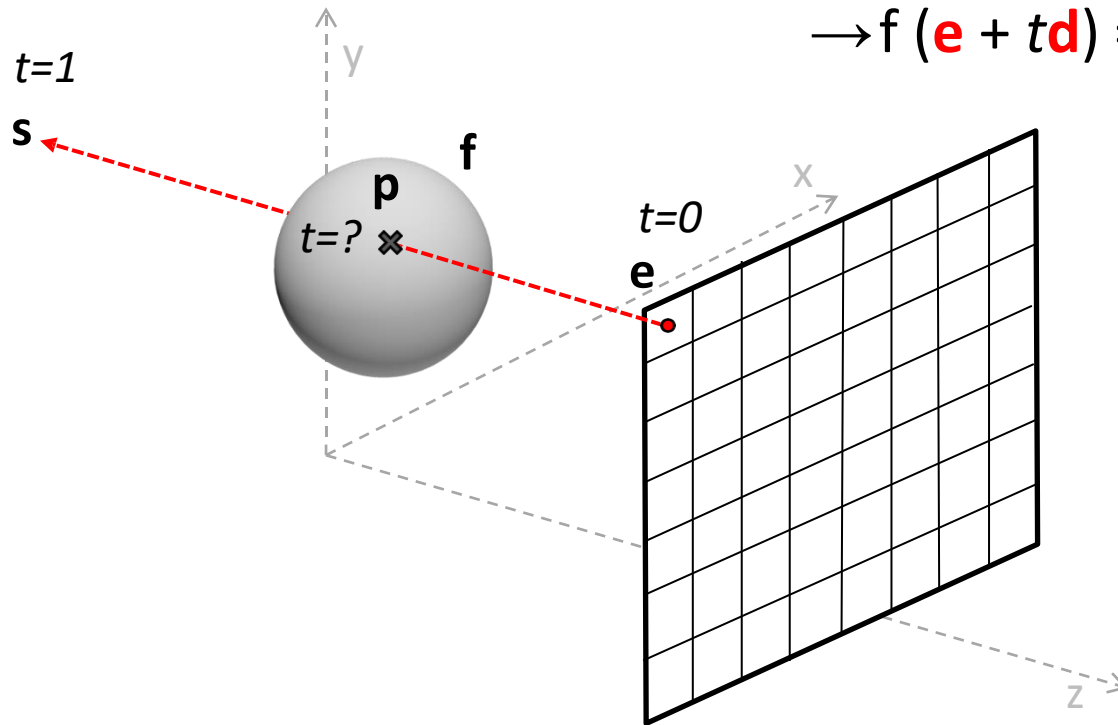


Ray - Sphere Intersection (1/8)

We have, $\mathbf{p} = \mathbf{e} + t(\mathbf{s} - \mathbf{e}) = \mathbf{e} + t\mathbf{d}$

$$\rightarrow f(\mathbf{p}) = 0$$

$$\rightarrow f(\mathbf{e} + t\mathbf{d}) = 0$$

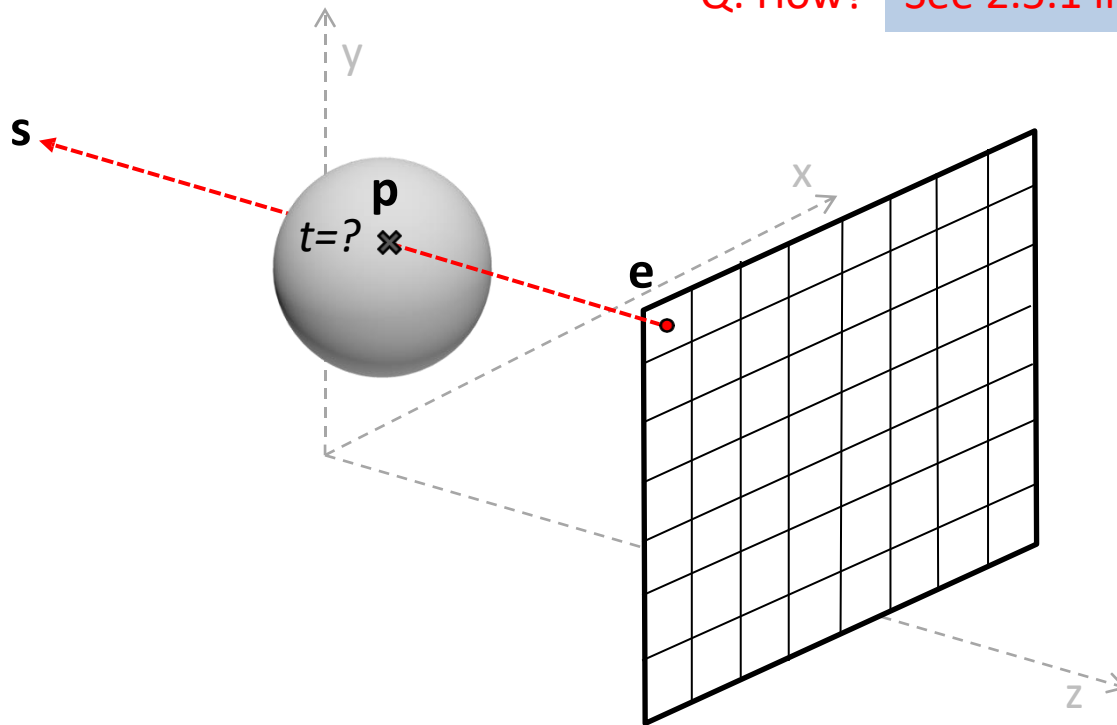


Ray - Sphere Intersection (2/8)

$$\rightarrow (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - R^2 = 0$$

$$\rightarrow (\mathbf{p} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) - R^2 = 0$$

Q: How? See 2.5.1 in Book



Ray - Sphere Intersection (3/8)

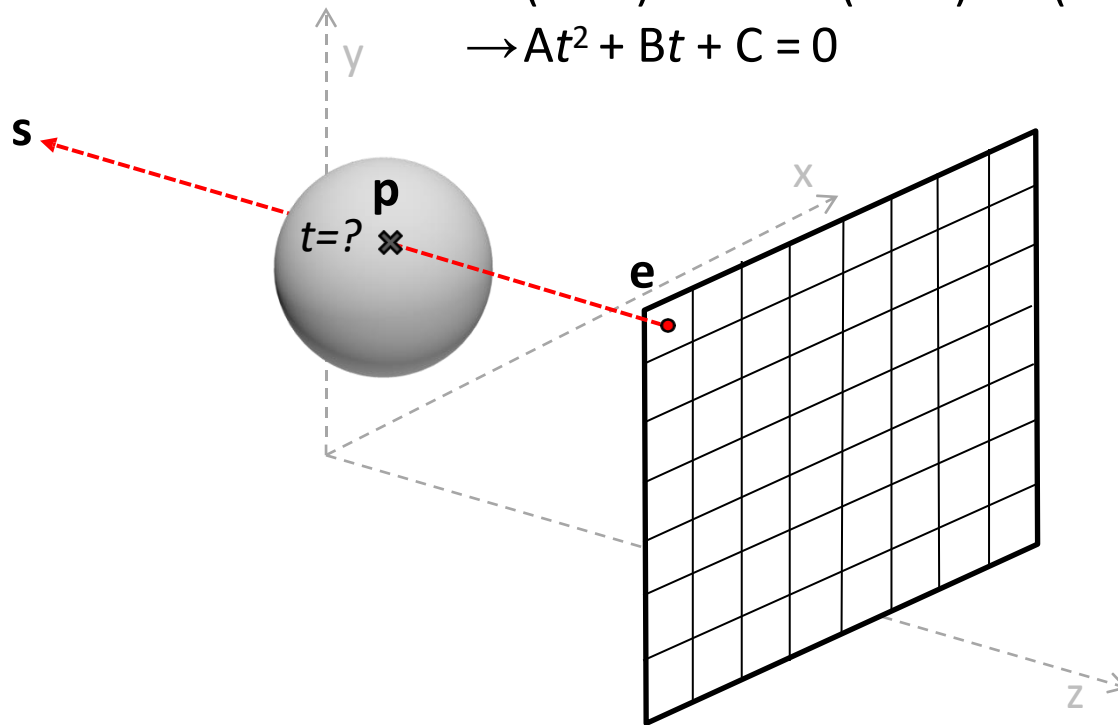
$$\rightarrow (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - R^2 = 0$$

$$\rightarrow (\mathbf{p} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) - R^2 = 0$$

$$\rightarrow (\mathbf{e} + t\mathbf{d} - \mathbf{c}) \cdot (\mathbf{e} + t\mathbf{d} - \mathbf{c}) - R^2 = 0$$

$$\rightarrow (\mathbf{d} \cdot \mathbf{d}) t^2 + 2\mathbf{d} \cdot (\mathbf{e} - \mathbf{c}) t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - R^2 = 0$$

$$\rightarrow At^2 + Bt + C = 0$$



Ray - Sphere Intersection (4/8)

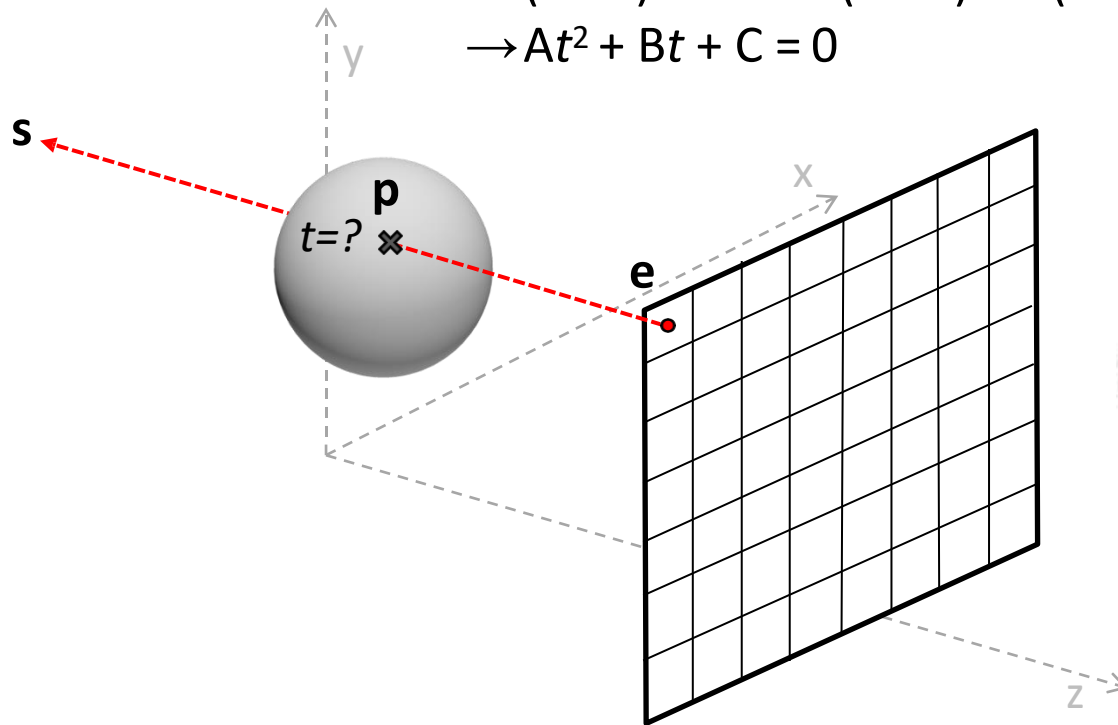
$$\rightarrow (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - R^2 = 0$$

$$\rightarrow (\mathbf{p} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) - R^2 = 0$$

$$\rightarrow (\mathbf{e} + t\mathbf{d} - \mathbf{c}) \cdot (\mathbf{e} + t\mathbf{d} - \mathbf{c}) - R^2 = 0$$

$$\rightarrow (\mathbf{d} \cdot \mathbf{d}) t^2 + 2\mathbf{d} \cdot (\mathbf{e} - \mathbf{c}) t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - R^2 = 0$$

$$\rightarrow At^2 + Bt + C = 0$$



$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

✓ Ray - Sphere Intersection (5/8)

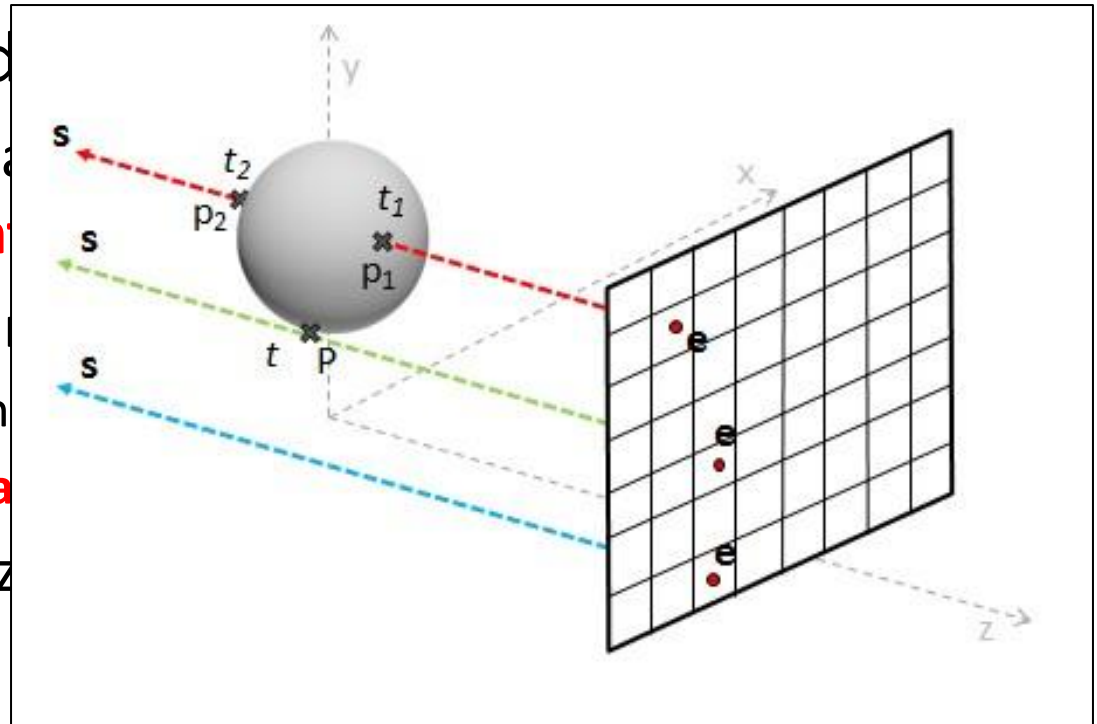
$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

- $B^2 - 4AC$, is called the discriminant and if it is –
 - **negative**: its square root is imaginary and the line and sphere **do not intersect**.
 - **positive**: there are two solutions –
 - one solution where the ray **enters** the sphere.
 - one where it **leaves**.
 - **zero**: the ray grazes the sphere, touching it at exactly **one point**.

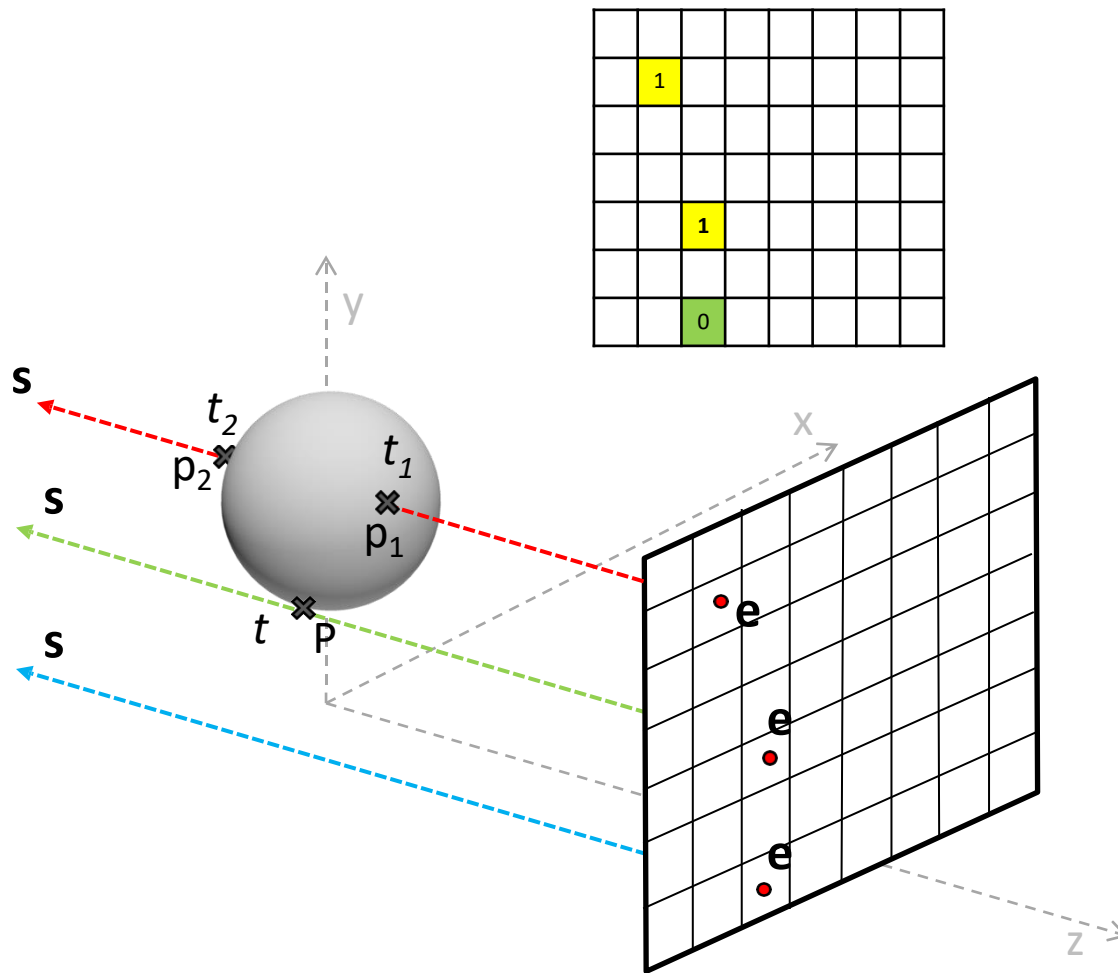
Ray - Sphere Intersection (5/8)

$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

- $B^2 - 4AC$, is called
 - **negative**: its square root is imaginary, sphere **do not intersect**
 - **positive**: there are two solutions
 - one solution where it **enters** the sphere
 - one where it **leaves** the sphere
 - **zero**: the ray grazes the sphere at a single **point**.



Ray - Sphere Intersection (6/8)



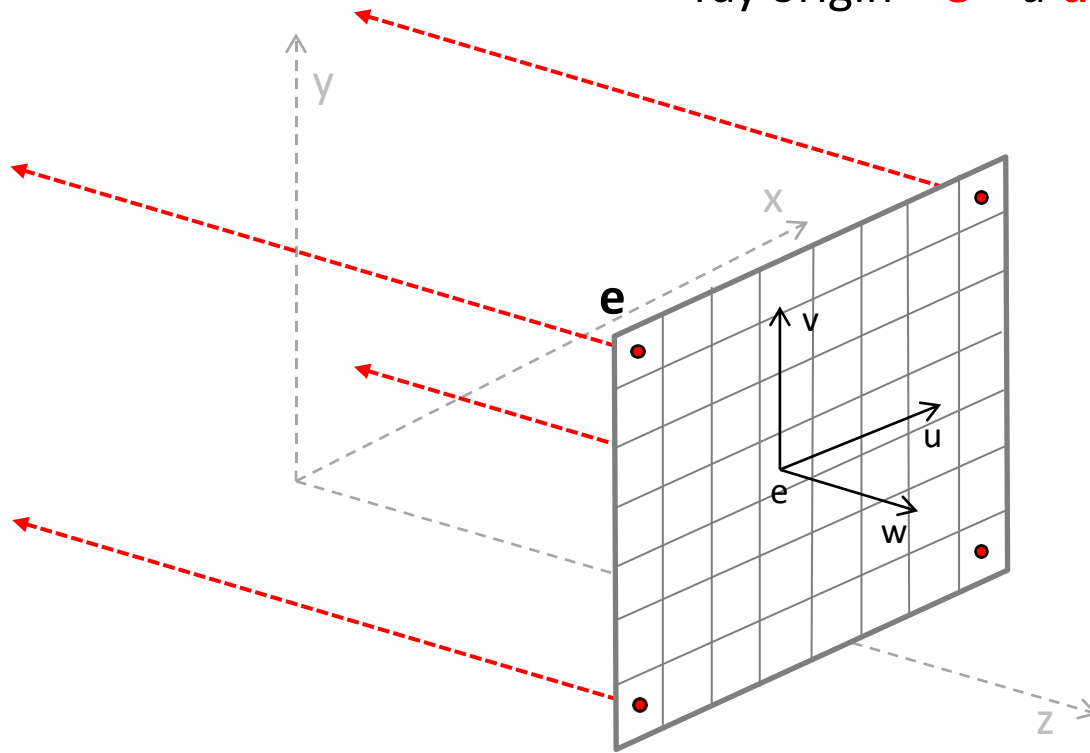
$$\begin{aligned} \mathbf{p} &= \mathbf{e} + t(\mathbf{s} - \mathbf{e}) \\ &= \mathbf{e} + t\mathbf{d} \end{aligned}$$

$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

Ray - Sphere Intersection (7/8)

Orthographic:

- ray direction = $-\mathbf{w}$
- ray origin = $\mathbf{e} + u \mathbf{u} + v \mathbf{v}$

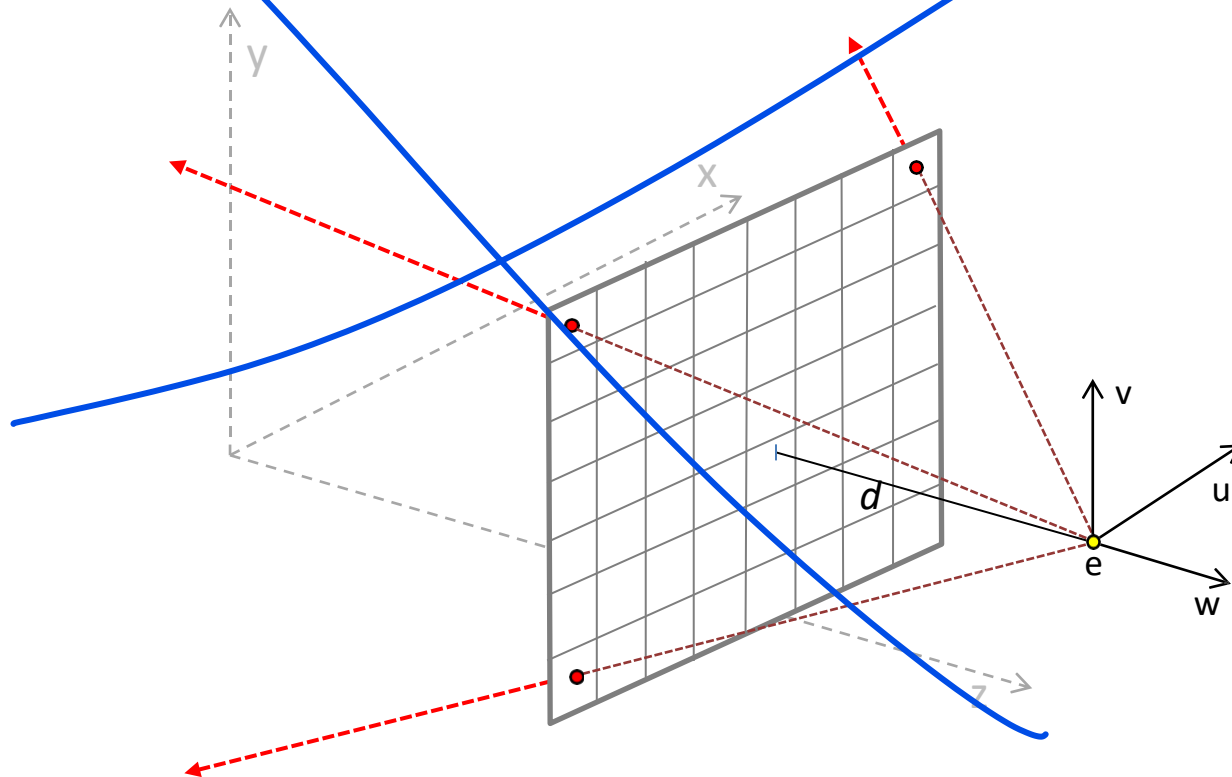


Ray - Sphere Intersection (8/8)

Perspective:

- ray direction = $-d\mathbf{w} + u\mathbf{u} + v\mathbf{v}$
- ray origin = $\mathbf{e}$

Q: why?

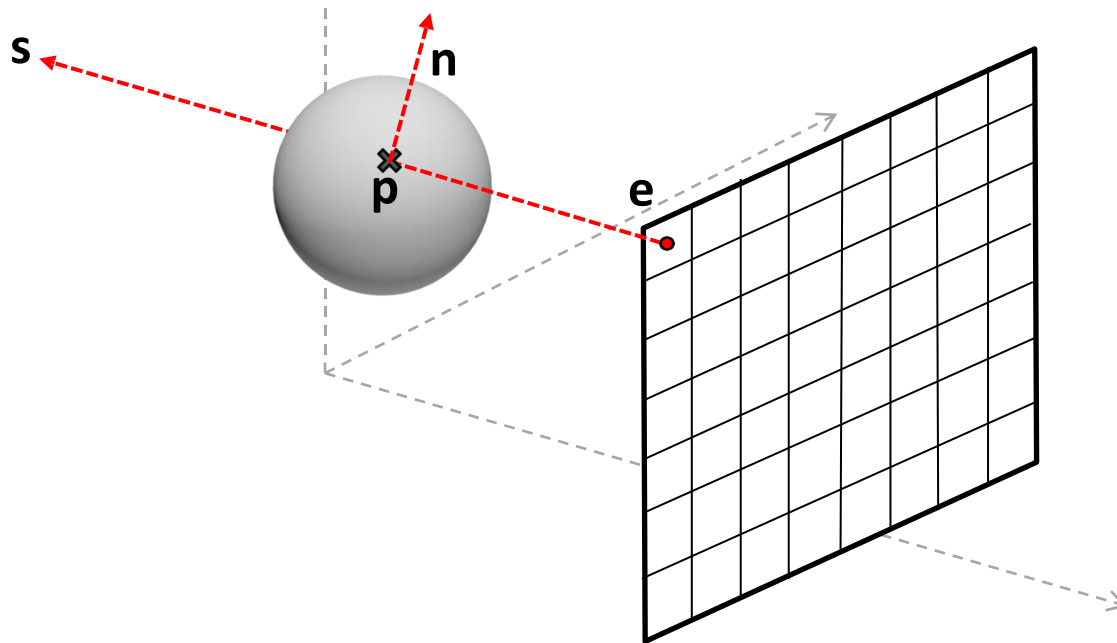


Shading (1/3)

Normal vector at point **p**:

- Gradient, $\mathbf{n} = 2 (\mathbf{p} - \mathbf{c})$.
- unit normal is $(\mathbf{p} - \mathbf{c})/R$.

[See section 2.5.4]

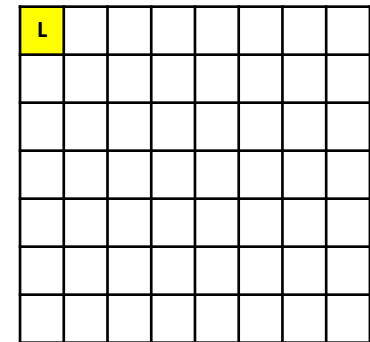
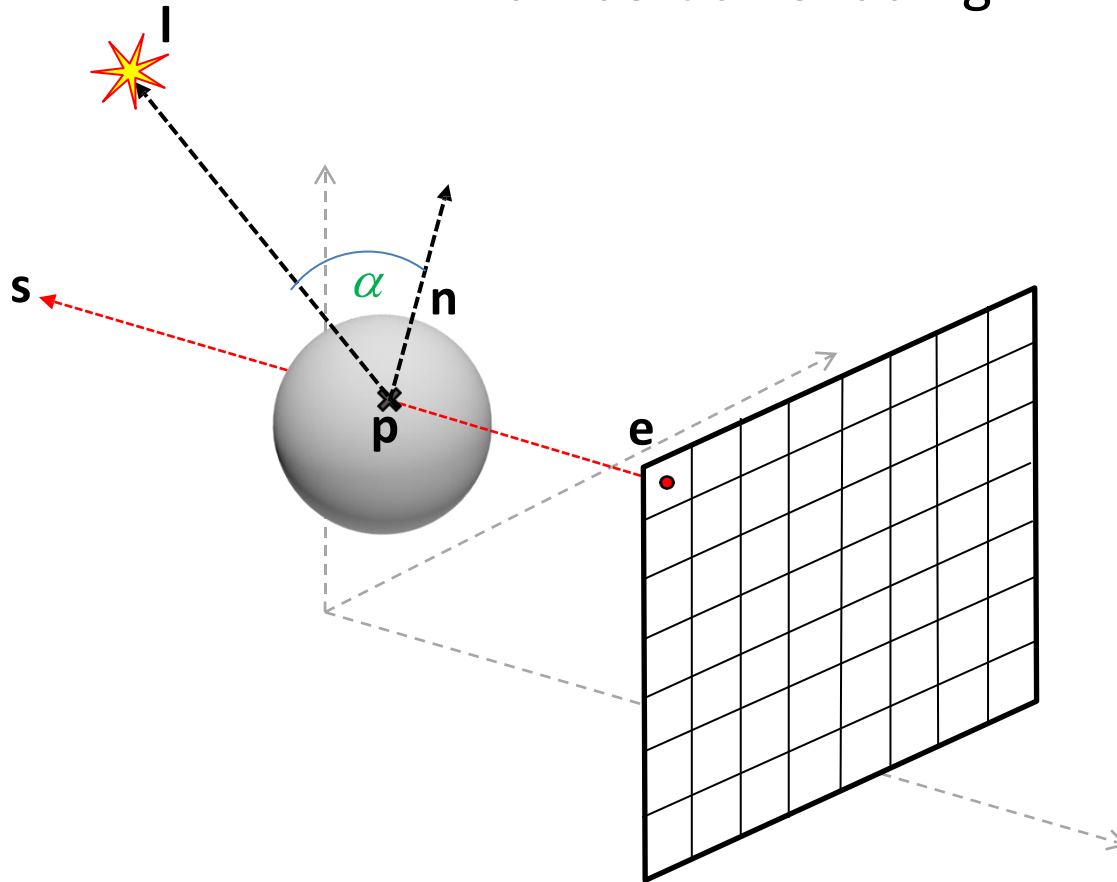


Shading (2/3)

Lambertian Shading: $L = k_d P \max(0, \mathbf{n} \cdot \mathbf{l})$

where,

- L = pixel color
- k_d = surface color
- P = intensity of the light source.



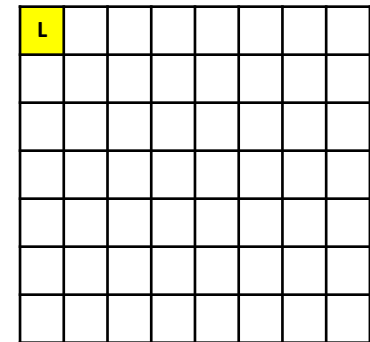
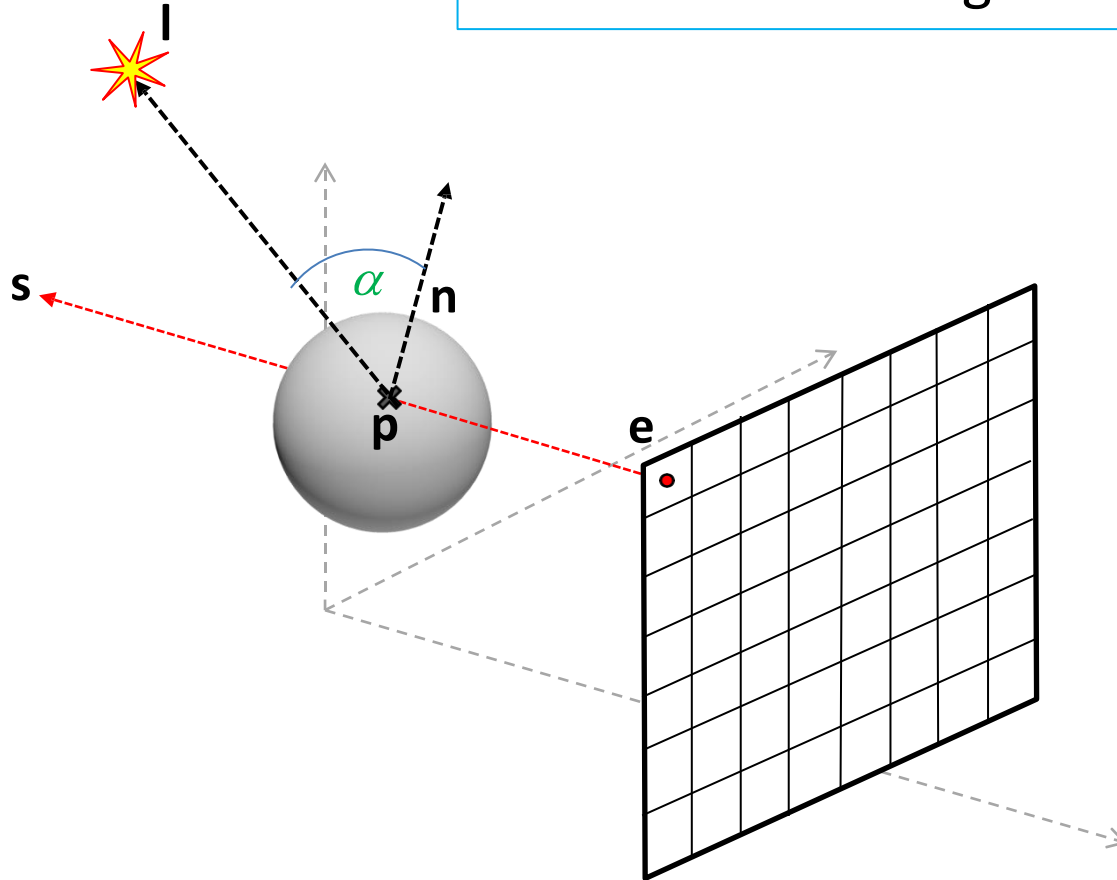
Shading (3/3)

Q: Are we considering angle in this formula? If yes – how?

Lambertian Shading: $L = k_d P \max(0, \mathbf{n} \cdot \mathbf{l})$

where,

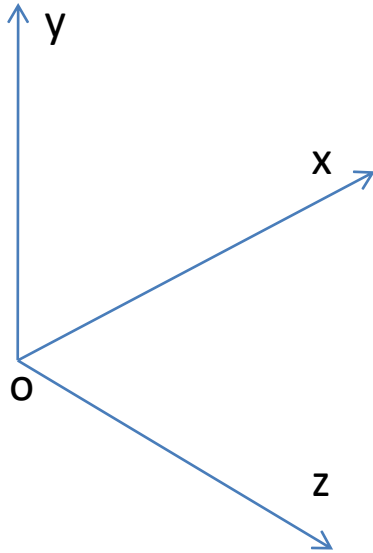
- L = pixel color
- k_d = surface color
- P = intensity of the light source.



Additional Reading

- 4.6: A Ray-Tracing Program

Practice Problem (1/3)



Camera frame (*orthographic*):

- $\mathbf{e} = [4, 4, 6]$; $\mathbf{u} = [1, 0, 0]$; $\mathbf{v} = [0, 1, 0]$; $\mathbf{w} = [0, 0, 1]$
 - Plot the camera frame on the given axis.

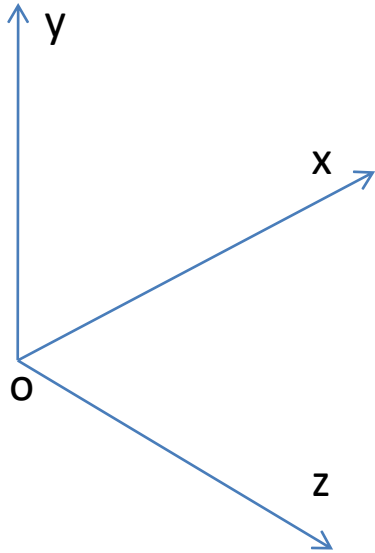
Viewing Ray:

- $\text{ray}_1.\text{origin} = \mathbf{e} + 2\mathbf{u} + 2\mathbf{v}$; $\text{ray}_1.\text{end} = [6, 6, 0]$
- $\text{ray}_2.\text{origin} = \mathbf{e} - 1\mathbf{u} + 1\mathbf{v}$; $\text{ray}_2.\text{end} = [4, 4, 0]$
 - Plot the origins for ray_1 and ray_2 .

Sphere: $f(x, y, z) = x^2 + y^2 + z^2 - (4)^2 = 0$

1. What are the intersecting points for ray_1 and ray_2 ?
2. Plot the intersecting points.

Practice Problem (2/3)



Camera frame (*orthographic*):

- $\mathbf{e} = [4, 4, 8]$; $\mathbf{u} = [1, 0, 0]$; $\mathbf{v} = [0, 1, 0]$; $\mathbf{w} = [0, 1, 0]$

Image Plane:

- left: $u = -5$; right: $u = 5$; top: $v = 4$; bottom: $v = -4$

1. Plot the image plane on the given axis.
2. For a 10×10 image matrix M , what is the position on the image plane for the ray origin at $M(4,3)$?
3. Will it intersect $f(x, y, z) = x^2 + y^2 + z^2 - 5^2 = 0$?

Practice Problem (3/3)

Consider the following parameters for an orthographic ray-tracing:

- *Camera frame:*

$$E = [-2, 7, 17]^T, U = [1, 0, 0]^T, V = [0, 1, 0]^T, W = [0, 0, 1]^T$$

- *Image plane:*

$$l = -15, r = 15, t = 10, b = -10$$

- *Raster image resolution:* 13×11
- *Sphere:* $(x+3)^2 + (y-5)^2 + (z-3)^2 = 64$

Determine the ray-sphere intersection point(s) for a ray (with *length* = 25) at the center of the raster image. Drawing figures is NOT mandatory.

Practice Problem (3/3)

Solution steps:

- Find u and v
 $u = l + (r - l)(i + 0.5) / nx$
 $v = b + (t - b)(j + 0.5) / ny$
- Determine the ray origin, $\mathbf{e} = \mathbf{E} + u\mathbf{U} + v\mathbf{V}$
- Find ray end point, $\mathbf{s} = \mathbf{e} + w(-\mathbf{W})$
- Determine, $\mathbf{d} = \mathbf{s} - \mathbf{e}$
- Determine, $D = B^2 - 4AC$
 $A = \mathbf{d} \cdot \mathbf{d}$
 $B = 2\mathbf{d} \cdot (\mathbf{e} - \mathbf{c})$
 $C = (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - R^2$
- Determine the intersection parameter, t
 $t_1 = (-B + \sqrt{D}) / (2A)$
 $t_2 = (-B - \sqrt{D}) / (2A)$
- Determine the intersection point,
 $\mathbf{P}_1 = \mathbf{e} + t_1(\mathbf{s} - \mathbf{e})$
 $\mathbf{P}_2 = \mathbf{e} + t_2(\mathbf{s} - \mathbf{e})$

$P > 0$: Positive hoile bujha jabe 2 ta point e intersect korse, kono solution thakbe

Exercise

- Textbook exercise
 - no: 1